
300,000 Streets of Melbourne Technical Implementation Book

*From Raw Data to Interactive Dashboard:
A Step-by-Step Developer's Guide*

Regen Melbourne × RMIT University
City of Darebin — School Streets Safety Analysis

Parts 1–7 + Appendices: Chapters 1–24

From Raw Data to Production Pipeline

Author:	Hishikesh Phukan
Academic supervisor:	Lawrence (RMIT University)
Industry partner:	Regen Melbourne
Schools assessed:	Reservoir HS, William Ruthven SC, Preston HS
LGA:	City of Darebin, Victoria
Data period:	2021–2025

This book is a living technical companion. It documents every design decision, data source, algorithm, and implementation detail so that any developer can recreate the pipeline from scratch.

Contents

Acknowledgements	viii
Preface	ix
I Foundations	1
1 Introduction and Project Context	2
1.1 The 300,000 Streets Initiative	2
1.2 Study Scope and School Selection	2
1.3 The Seven Deliverables	3
1.4 How to Use This Book	3
1.5 Project History and Timeline	4
1.6 A Note on Reproducibility	4
2 The Healthy Streets Framework	6
2.1 Origins and Purpose	6
2.2 The Ten Indicators	6
2.3 Scoring Each Indicator: 0 to 10	7
2.4 Severity Classification	8
2.5 Adapting the Framework for Secondary Schools	8
2.6 The Cycling Infrastructure Score (CIS)	9
2.7 Why Ten Indicators?	9
3 Development Environment Setup	10
3.1 Prerequisites	10
3.2 Cloning the Repository	10
3.3 Creating the Conda Environment	10
3.4 Installing Python Dependencies	11
3.5 Full Requirements Reference	11
3.6 Verifying the Installation	12
3.7 Setting Up the Outputs Directory	13
3.8 Coordinate Reference Systems	13
3.9 Running the Pipeline for the First Time	14
4 Repository Structure and Architecture	15
4.1 Directory Tree	15
4.2 The config.py Module	16
4.3 Data Flow Architecture	18
4.4 The Six-Step Main Pipeline	18
4.5 The src/ Package Structure	19
4.5.1 src/data/loader.py	19
4.5.2 src/scoring/	20
4.5.3 src/recommendations/engine.py	20

4.5.4	src/visualisation/	20
4.5.5	src/scenarios/	20
4.6	Naming Conventions	21
4.7	Git Workflow and Branch Strategy	21
4.8	Summary: What to Expect After a Full Pipeline Run	21
5	Study Area and School Selection	24
5.1	The City of Darebin	24
5.2	The Three Schools	24
5.3	School Selection Criteria	25
5.4	Socioeconomic Context: SEIFA IRSD	26
5.5	Catchment Definition	26
6	Field Survey Methodology	27
6.1	Why a Primary Survey?	27
6.2	The Google Forms Instrument	27
6.3	Column Rename Reference	27
6.4	Site Visit Protocol	29
6.5	The Data Loader: src/data/loader.py	30
7	OSM Data with osmnx	32
7.1	OpenStreetMap as Infrastructure Data	32
7.2	The osmnx Library	32
7.3	CRS and Projection	33
7.4	Buffer Strategy	33
7.5	Walking Network Features	34
7.6	Road Network Features	34
7.7	Crossings and Signals	35
7.8	Healthy Streets Extras at 400 m	36
7.9	Cycling Network Features	36
7.10	GeoPackage Export	37
7.11	Full Feature Inventory	37
8	Government Open Datasets	39
8.1	Overview	39
8.2	VicRoads Crash Data	39
8.2.1	Source and Download	39
8.2.2	Key Columns	39
8.2.3	School Hours Filter	40
8.2.4	Crash Trend Analysis	40
8.3	ABS SEIFA 2021 IRSD	41
8.3.1	Source and Download	41
8.3.2	seifa_darebin.csv Structure	41
8.3.3	Equity Analysis	41
8.4	EPA AirWatch PM2.5	42
8.4.1	Source	42
8.4.2	Preprocessing	42
8.5	Crime Statistics Victoria	42
8.5.1	Source	42
8.5.2	Preprocessing	42
8.5.3	Usage in the ML Model	43
8.6	Joining All Data Sources	43

9	The Healthy Streets Scoring Engine	46
9.1	Architecture of <code>src/scoring/</code>	46
9.2	How <code>main.py</code> Calls the Engine	46
9.3	HS1 — Pedestrians from All Walks of Life	47
9.4	HS2 — Easy to Cross	48
9.5	HS3 — Shade and Shelter (Spatial Indicator)	49
9.6	HS5 — Not Too Noisy (Deductive Pattern)	49
9.7	All Ten Indicators at a Glance	50
10	Severity Classification and Recommendations	52
10.1	The Severity Model	52
10.2	The Recommendation Rules: <code>src/recommendations/rules.py</code>	53
10.3	The Recommendation Engine: <code>run_recommendations()</code>	54
10.4	Recommendations CSV Structure	54
11	Machine Learning: Feature Engineering and Ridge Regression	56
11.1	Purpose and Context	56
11.2	Feature Engineering: <code>feature_engineering.py</code>	56
11.2.1	The 26-Feature Matrix	56
11.2.2	Data Joining Code	57
11.3	The Ridge Regression Model: <code>ml_model.py</code>	58
11.3.1	Why Ridge Regression?	58
11.3.2	Pipeline Structure	58
11.3.3	Leave-One-Out Cross-Validation	59
11.3.4	Full-Data Model and Serialisation	59
11.3.5	Three Output Charts	60
12	Scenario Analysis	61
12.1	What Scenario Analysis Does	61
12.2	Intervention Templates: <code>src/scenarios/interventions.py</code>	61
12.3	The Scenario Analyser CLI	63
12.3.1	Output Chart	64
12.3.2	Rank-All Mode	64
12.4	End-to-End Scenario Workflow	64
II	Analysis, Equity, and Dashboard Deployment	66
13	Crash Trend Analysis	68
13.1	Why Crash Data Matters	68
13.2	Loading and Filtering	68
13.2.1	Severity Labelling	69
13.2.2	School Hours Filter	69
13.3	Four-Panel Chart	69
13.3.1	Panel 1 — Darebin LGA Crashes by Year	69
13.3.2	Panel 2 — Our Three Schools by Year	70
13.3.3	Panel 3 — School Hours vs Off-Peak	70
13.3.4	Panel 4 — Time-of-Day Distribution	70
13.4	Key Findings Function	70
13.5	Integration with <code>run_all.py</code>	71
14	Equity Analysis	72
14.1	The Equity Hypothesis	72

14.2	SEIFA Data Pipeline	72
14.3	Data Join	73
14.4	Three-Panel Chart	73
14.4.1	Panel 1 — Side-by-Side Bars	73
14.4.2	Panel 2 — Scatter Plot with Quadrant Shading	73
14.4.3	Panel 3 — Indicator Heatmap	73
14.5	Pearson Correlation and Policy Interpretation	74
14.6	Running the Equity Analysis	74
15	Dashboard and Deployment Pipeline	75
15.1	Architecture Overview	75
15.2	prepare_data.py — The Data Builder	75
15.2.1	Required Inputs	75
15.2.2	build_schools()	75
15.2.3	build_scenarios()	76
15.2.4	Map Layer Builders	76
15.2.5	Chart Copying	77
15.2.6	Final data.json Structure	77
15.3	Dashboard Sections	78
15.3.1	Hero Banner	78
15.3.2	Interactive Map	78
15.3.3	School Tabs and Radar Charts	78
15.3.4	Scenario Explorer	79
15.3.5	Analysis Section	79
15.3.6	Recommendations Panel	79
15.4	run_all.py — The Master Pipeline Runner	79
15.4.1	Skip-if-Exists Logic	79
15.4.2	Pipeline Steps Summary	79
15.4.3	Flags	80
15.5	GitHub Pages Deployment	80
III	Data Ingestion, Environmental Features, and Core Pipeline	82
16	Crash Data Download and Spatial Join	84
16.1	Overview	84
16.2	VicRoads Open Data Schema	84
16.3	Downloading the Three Tables	84
16.4	Table Join and Date Filter	85
16.5	Pedestrian and Cyclist Filter	85
16.6	School Gate Lookup	86
16.7	Vectorised Haversine Distance	86
16.8	Running crash_analysis.py	87
17	Environmental Features: AQI and Crime	88
17.1	Why Environmental Data	88
17.2	Module Architecture	88
17.3	EPA AirWatch — Three-Tier Fallback	88
17.3.1	AQI to HS10 Score Conversion	89
17.4	Crime Statistics Agency — Single-Attempt Download	90
17.4.1	Crime Rate to HS7 Score Contribution	90
17.5	School-Suburb Mapping	90

17.6	Output CSV Structure	91
18	Core Pipeline Orchestration	92
18.1	Role of main.py	92
18.2	Data Loading Hierarchy	92
18.3	HS Scoring Loop	93
18.4	Chart Generation	94
18.4.1	Chart 1 — Radar Chart	94
18.4.2	Chart 2 — 10-Indicator Grid	94
18.4.3	Chart 3 — Per-School Breakdown	94
18.5	Visualisation Module	94
18.5.1	Interactive Map	95
18.5.2	KDE Heatmap	95
18.6	Graceful Degradation Pattern	95
18.7	Expected Outputs	95
IV	Feature Engineering, Socio-economic Analysis, and Demographics	97
19	Feature Engineering for Machine Learning	98
19.1	Purpose	98
19.2	Feature Groups	98
19.2.1	Spatial Features (OSM)	98
19.2.2	Environmental Features	99
19.2.3	Crash Statistics	99
19.3	Speed-Zone Sentinel Codes	99
19.4	School-Hours Crash Fraction	99
19.5	Four-Way Join	100
19.6	Column Order Convention	100
19.7	Console Summary	100
20	SEIFA Disadvantage Analysis	101
20.1	Purpose	101
20.2	ABS Download and Dynamic Column Discovery	101
20.3	Darebin Filtering	102
20.4	Curated Fallback Data	102
20.5	Population-Weighted Catchment Average	102
20.6	Disadvantage Labels	103
20.7	School Gate Coordinates	103
20.8	Key Finding	103
21	ABS Demographics Chart	105
21.1	Purpose	105
21.2	Script Structure	105
21.3	The Four Panels	105
21.4	School-Age Children Estimate	106
21.5	Connection to the Equity Argument	106
21.6	Pipeline Position	106
V	Synthesis, Extensions, and Conclusions	108
22	Pilot Study Results and Policy Implications	109

22.1 Overview	109
22.2 School-by-School Analysis	110
22.2.1 Reservoir High School — Overall 4.6 / 10 — Major	110
22.2.2 William Ruthven Secondary College — Overall 5.6 / 10 — Moderate	110
22.2.3 Preston High School — Overall 6.3 / 10 — Moderate	110
22.3 The Equity Finding	111
22.4 Top Recommendations by School	111
22.5 Implications for the 300,000 Streets Initiative	112
23 Extending the Pipeline to New Schools and LGAs	113
23.1 Design for Reuse	113
23.2 Five-Step Checklist for a New School	113
23.3 config.py as the Single Source of Truth	114
23.4 Scaling to a Full LGA	114
23.5 Common Errors and Solutions	115
24 Architecture, Limitations, and Lessons Learned	116
24.1 What the Architecture Got Right	116
24.2 Known Limitations	116
24.2.1 Leave-One-Out Cross-Validation with $n = 3$	116
24.2.2 OSM Data Completeness	117
24.2.3 Three-School Sample and the Equity Correlation	117
24.2.4 Hardcoded Fallback Data	117
24.3 Design Decisions Worth Documenting	117
24.4 Future Technical Directions	118
A Complete HS Scoring Formula Reference	119
B Recommendation Rules and Intervention Catalogue	122
C Pipeline and Output Quick Reference	124
What's Next	127

Acknowledgement of Country

I acknowledge the Wurundjeri Woi-wurrung people of the Kulin Nation as the Traditional Custodians of the land on which this research was conducted — the land we call Melbourne (Naarm), and the surrounding suburbs of Darebin, Reservoir, Preston, and Thornbury, all of which sit within Wurundjeri Woi-wurrung Country.

I pay my deepest respects to their Elders, past and present, and to all Aboriginal and Torres Strait Islander peoples whose connection to Country predates the streets we study and endures long after the data is gathered.

This land was never ceded.

— Hishikesh Phukan

Acknowledgements

This work is the product of extraordinary support from people and institutions across two continents. I am deeply grateful to all of them.

Australia — a country that gave me a chance.

I came to Australia to pursue a Master’s degree and left with skills, knowledge, and an understanding of what data-driven research can do for real communities. Australia opened that door. The public institutions that underpin this project — the ABS, EPA Victoria, VicRoads, Crime Statistics Agency, OpenStreetMap contributors, and the many councils whose open data made this analysis possible — reflect the kind of society that invests in shared infrastructure and shared knowledge. I am grateful to be a small part of that.

RMIT University.

RMIT gave me the academic home, the coursework, and the environment to grow as a researcher and engineer. The structure of COSC2667/COSC2777 pushed me to build something real, not just something theoretical. I am grateful to my academic supervisor Lawrence for the guidance, the critique, and the patience throughout this project.

Regen Melbourne.

The 300,000 Streets initiative is Regen Melbourne’s vision: that every child in metropolitan Melbourne deserves a safe street to walk to school. Partnering with Regen Melbourne gave this project a purpose beyond the grade. Their commitment to equity-centred urban planning kept me focused on what the data is actually for: the students walking along Plenty Road every morning.

My parents.

Nothing I have done would have been possible without my parents, who pushed me hard, believed in me without condition, and supported me through every difficulty of a Master’s degree on the other side of the world. They did not let me quit when it was hard, and they celebrated every small win as if it were the biggest. This book is for them.

*Hishikesh Phukan
Melbourne, Victoria, Australia
2026*

Preface

This book documents the complete technical implementation of the **300,000 Streets of Melbourne** pilot study — a data-driven assessment of pedestrian safety conditions around three secondary schools in the City of Darebin. It is intended as a companion to the formal project report and is written so that any developer with intermediate Python experience can recreate every output from scratch.

Who should read this book. This book is for:

- Data scientists and engineers who want to extend the pipeline to more schools or other local government areas (LGAs);
- Urban planners and researchers who want to understand the methodology behind the Healthy Streets scoring engine;
- Students who want a worked example of an end-to-end spatial data science project, from field survey design through to a deployed web dashboard.

What this book covers. Chapters 1–4 cover the project context, the Healthy Streets framework, environment setup, and the overall software architecture. Later chapters (to be released as Parts 2, 3, and beyond) will cover data collection, scoring, machine learning, scenario analysis, equity analysis, crash analysis, and dashboard deployment.

Conventions.

- **Code listings** show exact file contents from the project. Line numbers align with the actual source files.
- **Monospace text** in the body refers to file names, function names, variable names, or shell commands.
- Callout boxes highlight key concepts (**blue**), tips (**green**), and warnings (**amber**).
- All paths are relative to the project root unless stated otherwise.

Part I

Foundations

Chapter 1

Introduction and Project Context

1.1 The 300,000 Streets Initiative

The **300,000 Streets of Melbourne** initiative is a collaboration between **Regen Melbourne** — a not-for-profit urban sustainability organisation — and **RMIT University**. Its goal is ambitious: systematically assess pedestrian and cyclist safety conditions around every school in metropolitan Melbourne and use that evidence base to advocate for infrastructure investment in the most disadvantaged communities.

The name derives from an estimate of the total lane-kilometres of local streets in metropolitan Melbourne. Most of these streets have never been assessed for walkability or child pedestrian safety; they lack footpath audits, crossing inventories, or any measure of how welcoming the street environment is for someone arriving on foot or by bicycle.

The pilot study documented in this book focuses on **three secondary schools in the City of Darebin**, a northern-inner-ring LGA whose school zone population spans a wide socioeconomic gradient. The pilot is designed to prove out the methodology before scaling to the full metropolitan network.

1.2 Study Scope and School Selection

Three schools were selected to represent the socioeconomic diversity of the Darebin LGA, using the ABS 2021 SEIFA Index of Relative Socio-economic Disadvantage (IRSD):

Table 1.1: Schools assessed in the pilot study

School	Address	SEIFA Decile	Suburb
Reservoir High School	855 Plenty Rd, Reservoir VIC 3073	D4	Reservoir
William Ruthven Secondary College	60 Merrilands Rd, Reservoir VIC 3073	D6	Reservoir
Preston High School	2–16 Cooma St, Preston VIC 3072	D7	Preston

SEIFA decile 1 indicates the most socioeconomically disadvantaged, decile 10 the least. All three schools fall in deciles 4–7, covering the lower-middle to middle range of disadvantage. A key research hypothesis is that disadvantage correlates with worse pedestrian safety — and the pilot provides early evidence that this is indeed the case ($r = 0.84$).

1.3 The Seven Deliverables

The project was contracted around seven concrete deliverables. Every technical component in this book serves at least one of them:

D1 — Field Assessment Report

A written report documenting observations at each school gate, including hazard descriptions, photos, and qualitative commentary. This feeds the raw `school_data.csv` dataset.

D2 — Healthy Streets Scores

Quantitative scores for all ten Healthy Streets indicators (HS1–HS10) at each school, plus an overall severity classification. Output: `outputs/hs_scores.csv`.

D3 — Visualisation Suite

Three static charts — a radar chart, a per-indicator bar chart, and a per-school breakdown chart. Output: `outputs/chart1_safety_scores.png`, `chart2_hazard_severity.png`, `chart3_score_breakdown.png`.

D4 — Equity Analysis

An analysis of whether SEIFA disadvantage predicts lower Healthy Streets scores. Output: `outputs/chart_equity_seifa.png`.

D5 — Crash Trend Analysis

A deep-dive into VicRoads crash data (2021–2025) showing pedestrian and cyclist crash trends, school-hours breakdowns, and time-of-day distributions. Output: `outputs/chart_crash_trends.png`.

D6 — Machine Learning Pipeline

A Ridge regression model trained on open data (OSM, AQI, crime, crash statistics) to predict HS indicator scores at new schools without field surveys. Output: `outputs/hs_predictor.pkl` and associated charts.

D7 — Interactive Dashboard

A GitHub Pages static web dashboard with scenario explorer, hero stats, and equity/crash charts. Deployed via `docs/` directory.

1.4 How to Use This Book

This book is **sequential**. Each chapter builds on the previous one. We recommend working through it in order, running each script as it is introduced and inspecting the outputs before moving on.

A rough reading and execution guide:

Table 1.2: Reading and execution guide

Part	Chapters	Time estimate
1	Ch. 1–4: Foundations	2–3 hours (reading + environment setup)
2	Ch. 5–8: Data Collection	4–6 hours (field survey + API calls can be slow)
3	Ch. 9–12: HS Scoring Engine	2–3 hours
4	Ch. 13–15: Spatial Feature Engineering	2–3 hours
5	Ch. 16–18: Machine Learning	1–2 hours
6	Ch. 19–21: Scenario & Equity Analysis	2 hours
7	Ch. 22–24: Visualisation & Dashboard	3–4 hours

Before you begin

You will need: Python 3.10 or higher, an internet connection for Overpass API calls to OpenStreetMap, access to the VicRoads open data portal, and approximately 2 GB of free disk space for intermediate outputs. Chapter 3 covers environment setup in full detail.

1.5 Project History and Timeline

The project ran over twelve weeks during Semester 1, 2025. The pipeline was built in four sprints:

- 1. **Sprint 1 (Weeks 1–3):** Project scoping, school selection, field survey protocol design, and initial data collection.
- 2. **Sprint 2 (Weeks 4–6):** HS scoring engine, static visualisations, and crash data integration.
- 3. **Sprint 3 (Weeks 7–9):** OSM spatial feature extraction, feature engineering, and Ridge regression ML model.
- 4. **Sprint 4 (Weeks 10–12):** Scenario engine, equity analysis, interactive dashboard, and report writing. Sprint 4 absorbed significant unplanned scope from upstream delays in OSM data quality issues.

The twelve-week, sequentially dependent pipeline left no buffer for upstream delays — a key limitation noted in the report. Despite this, all seven deliverables were completed on time.

1.6 A Note on Reproducibility

The OSM data used in this project is live: the Overpass API returns whatever is currently mapped at the time of the query. Our spatial features were extracted in April 2025. If you re-run `spatial_features.py` today, OSM may have changed, leading to slightly different

feature values and therefore slightly different ML predictions. The VicRoads crash dataset is updated quarterly; similarly, running the crash analysis scripts at a later date will produce updated trend charts.

Non-determinism

ML results and some chart values will differ slightly depending on when you pull OSM data. For exact replication of the published results, use the CSV files in `outputs/` that are committed to the repository rather than re-running the data collection scripts.

Chapter 2

The Healthy Streets Framework

2.1 Origins and Purpose

The **Healthy Streets** framework was developed by **Dr Lucy Saunders** for Transport for London (TfL) in 2015. It provides a structured, evidence-based approach to assessing how well a street supports the physical and mental health of people who walk, cycle, or use public transport. The framework defines ten indicators — each scored from 0 to 10 — that together capture whether a street environment is genuinely hospitable to active travel.

The original TfL framework was designed for general urban streets. This project adapts it specifically for the **school gate context**: a 400-metre radius around the main pedestrian entrance of a secondary school, assessed during the school drop-off and pick-up periods (7:30–9:00 am and 2:30–4:30 pm on weekdays).

Core principle of the Healthy Streets framework

A “healthy street” is one that enables and encourages people to walk, cycle, and use public transport as part of their everyday lives — not just for recreation, but for the journeys they need to make. The framework asks: would a person of any age, ability, or background choose to walk or cycle on this street?

2.2 The Ten Indicators

Each of the ten indicators addresses a distinct dimension of street quality. Together they cover physical infrastructure, environmental conditions, and the subjective experience of being on the street.

Table 2.1: Healthy Streets indicators HS1–HS10

Code	Name	Description
HS1	Pedestrians from all walks of life	Does the street infrastructure enable people of all ages, abilities, and mobility needs to walk safely? This covers footpath presence, width, continuity, condition, kerb ramps, and obstacles.
HS2	Easy to cross	Can a pedestrian cross the road safely and conveniently at or near the school gate? This covers the presence of a signalised or marked crossing, its distance from the gate, visibility, condition, and whether there are pedestrian countdown timers.

Code	Name	Description
HS3	Shade and shelter	Is there shade from trees, or shelter from rain, for people waiting near the school gate? Covers tree canopy density within 100 m, bus shelters within 200 m, and green space percentage within 400 m.
HS4	Places to stop and rest	Are there benches or other seating opportunities within 200 m of the gate? Covers bench count (OSM) and park count within 400 m.
HS5	Not too noisy	Is the noise environment comfortable for pedestrians? Covers speed limit, traffic volume, heavy vehicle presence, arterial road percentage, and presence of traffic calming measures.
HS6	People choose to walk, cycle, or use PT	Does the street encourage active and sustainable travel? Covers cycling infrastructure quality (using the Cycling Infrastructure Score, CIS), public transport stop count within 400 m, and presence of a school zone.
HS7	People feel safe	Do people feel safe from crime and threatening behaviour? Covers suburb-level crime rate (incidents per 100,000 population) from Crime Statistics Victoria.
HS8	Things to see and do	Is there visual interest and activity along the street? Covers amenity count (cafes, shops, community facilities) and street furniture within 400 m.
HS9	People feel relaxed	Is the overall street environment calming rather than stressful? Derived from a composite of traffic stress indicators: speed limit, traffic volume, parking conflicts, and lighting quality.
HS10	Clean air	Is air quality acceptable for sustained outdoor activity? Covers EPA AirWatch annual average PM2.5 concentration for the monitoring station nearest the school.

2.3 Scoring Each Indicator: 0 to 10

Each indicator is scored on a continuous scale from **0** (worst) to **10** (best). The scoring functions are defined in `src/scoring/` (one Python file per indicator). The following principles apply across all indicators:

1. **Component scores are weighted and summed.** Each indicator draws on 2–8 measurable components. Each component is scored 0–1 (or 0–10 in some cases), then combined with a defined weight.
2. **Scores are clipped to [0, 10].** Intermediate calculations can exceed this range; the final score is always clipped.
3. **Missing data is handled gracefully.** If a required data field is absent (e.g. OSM has no bench tags near a school), the component is scored at 0 or at the dataset mean, depending on the indicator.

4. **Spatial data augments field data.** Indicators HS3, HS4, HS6, and HS8 combine field observations with OSM-derived spatial features, because these are difficult to observe completely during a single site visit.

Which indicators need field surveys?

HS1, HS2, HS5, HS7, and HS9 rely primarily on field observations (footpath width, crossing visibility, traffic volume, etc.). HS3, HS4, HS6, and HS8 rely primarily on OSM data and can therefore be computed for any school without a site visit, making them key candidates for the ML triage pipeline. HS10 comes from EPA monitoring data and requires no field work at all.

2.4 Severity Classification

After computing all ten indicator scores, the pipeline assigns each school a **severity classification** — a single label that prioritises schools for intervention:

Table 2.2: Severity classification rules

Severity	Rule
Major	HS2 < 4.0 OR HS1 < 4.0 OR HS5 < 3.0
Moderate	Two or more indicators < 6.0 OR overall HS score < 5.0
Minor	All other cases

The thresholds are set deliberately conservative: a school is classified as **Major** if any single critical indicator — crossing safety (HS2), footpath accessibility (HS1), or noise (HS5) — falls below 4.0. This reflects the view that a missing or very poor crossing near a school gate is a stand-alone safety crisis regardless of how well the school performs on other indicators.

In our pilot, Preston HS was classified as **Major** (HS2 = 0.4, no signalised crossing within 400 m of the gate). Both Reservoir HS and William Ruthven SC were classified as **Moderate**.

2.5 Adapting the Framework for Secondary Schools

The original TfL framework was designed for general urban contexts, not specifically for schools. Several adaptations were made for this project:

School zone recognition (HS6). The presence of an active school zone (as per VicRoads school zone data) is treated as a positive indicator for HS6, because it signals that infrastructure investment has already been made for active travel near the school.

Gate-based spatial analysis. All buffer radii (200 m, 400 m, 800 m) are measured from the main pedestrian gate of each school, not from the centroid of the school parcel.

School-hours crash filtering (D5). The crash analysis (Chapter 21 in the full book) filters crashes by time of day: 7:30–9:00 am and 2:30–4:30 pm, Monday–Friday, to focus on the highest-risk periods for students.

SEIFA equity layer (D4). The equity analysis overlays SEIFA disadvantage on top of HS scores to test whether disadvantaged school catchments also have the worst pedestrian safety — a finding with direct policy implications.

2.6 The Cycling Infrastructure Score (CIS)

HS6 uses a custom ordinal scale called the **Cycling Infrastructure Score (CIS)** to encode the type of cycling infrastructure present near each school. The CIS is derived from the Level of Traffic Stress (LTS) framework (Mekuria, Furth & Nixon, 2012) and from VicRoads Traffic Engineering Manual Volume 3, Part 218.

Table 2.3: Cycling Infrastructure Score (CIS) mapping

Infrastructure type observed	CIS score
Separated bike lane (physical barrier)	9.0
Shared path or greenway	8.0
Painted bike lane (on-road, no barrier)	4.5
Advisory lane or shared road	2.0
No cycling infrastructure	1.0

This mapping is defined in `config.py` as `CIS_MAP` and is imported by the HS6 scoring function.

2.7 Why Ten Indicators?

The ten-indicator structure allows the framework to be *disaggregated*: a single overall score hides the fact that a school may score well on air quality but very badly on crossing safety. By examining all ten indicators, decision-makers can pinpoint exactly which infrastructure interventions will have the greatest impact on the overall score.

In our ML pipeline (Chapters 16–18 of the full book), each indicator is predicted *independently*, allowing the scenario engine to show how a single intervention — such as installing a traffic signal — affects HS2 and HS9 specifically, rather than just changing the aggregate score.

Chapter 3

Development Environment Setup

3.1 Prerequisites

Before you can run any code in this project, you need the following installed on your system:

- **Python 3.10 or higher.** The project uses f-strings with = (Python 3.8+), `pathlib.Path` idioms, and type annotations that require at least 3.10.
- **Git.** To clone the repository and track changes.
- **Conda or pip with virtualenv.** We strongly recommend using a Conda environment to avoid dependency conflicts, particularly between `geopandas`, `osmnx`, and `shapely`.
- **Internet access.** The `spatial_features.py` script makes live calls to the OpenStreetMap Overpass API. Each school takes 1–2 minutes. Without internet access, you must use the pre-computed CSVs.
- **≈2 GB free disk space.** The intermediate outputs (CSVs, PNGs, HTML files, the GeoPackage) accumulate to around 1.5 GB.

3.2 Cloning the Repository

```
1 # Clone via HTTPS (no SSH key required)
2 git clone https://github.com/s4111078-sidd/300-000-School-Streets.git
3
4 # Navigate into the project root
5 cd 300-000-School-Streets
6
7 # Verify the directory structure
8 ls -la
```

Listing 3.1: Clone the project repository

You should see, among other files: `main.py`, `config.py`, `spatial_features.py`, `ml_model.py`, `feature_engineering.py`, `scenario_analyzer.py`, `equity_analysis.py`, `crash_trend_analysis.py`, and a `src/` directory.

3.3 Creating the Conda Environment

We strongly recommend Conda because it handles the binary dependencies of `geopandas` (GDAL, GEOS, PROJ) without requiring you to install system-level libraries manually.

```
1 # Create a new environment named "streets" with Python 3.11
2 conda create -n streets python=3.11 -y
```

```

3
4 # Activate it
5 conda activate streets
6
7 # Install geospatial dependencies first via conda-forge
8 # (these have pre-compiled binaries --- much easier than pip)
9 conda install -c conda-forge \
10     geopandas \
11     osmnx \
12     shapely \
13     fiona \
14     pyproj \
15     networkx \
16     -y

```

Listing 3.2: Create and activate the Conda environment

Why conda-forge for geopandas?

Installing `geopandas` via `pip` on Windows or macOS often fails because it requires compiled C libraries (GDAL, GEOS, PROJ). The `conda-forge` channel provides pre-built binaries for all platforms. Always install `geopandas` and `osmnx` via `conda` before installing the remaining packages via `pip`.

3.4 Installing Python Dependencies

Once the geospatial stack is installed via Conda, install the remaining dependencies using `pip`:

```

1 # Install all remaining dependencies
2 pip install \
3     pandas==2.1.4 \
4     numpy==1.26.4 \
5     matplotlib==3.8.2 \
6     seaborn==0.13.2 \
7     scikit-learn==1.4.0 \
8     scipy==1.12.0 \
9     folium==0.15.1 \
10    jinja2==3.1.3
11
12 # Verify key packages installed correctly
13 python -c "import geopandas, osmnx, sklearn, folium; print('All OK!')"

```

Listing 3.3: Install remaining Python packages via `pip`

Using `pip` only (no Conda)

If you cannot use Conda (e.g. on a server), try: `pip install geopandas` after first running `pip install GDAL`. On Ubuntu/Debian, run `sudo apt-get install gdal-bin libgdal-dev python3-gdal` first. On macOS with Homebrew: `brew install gdal`.

3.5 Full Requirements Reference

The following table lists all Python packages used across the project, their purpose, and the modules that import them:

Table 3.1: Python package requirements

Package	Version	Used by
geopandas	0.14+	spatial_features.py, interactive map
osmnx	1.9+	spatial_features.py (OSM API calls)
shapely	2.0+	Geometry operations (buffers, intersections)
pyproj	3.6+	CRS transformations (WGS84 → GDA2020)
networkx	3.2+	Graph analysis (osmnx dependency)
pandas	2.1+	All scripts (data manipulation)
numpy	1.26+	All scripts (numerical operations)
matplotlib	3.8+	main.py, all chart scripts
seaborn	0.13+	ml_model.py (correlation heatmap)
scikit-learn	1.4+	ml_model.py (Ridge, Pipeline, LOO-CV)
scipy	1.12+	KDE heatmap, statistical tests
folium	0.15+	Interactive map (Leaflet.js wrapper)
jinja2	3.1+	Dashboard HTML templating
fiona	1.9+	GeoPackage read/write (osmnx dependency)

3.6 Verifying the Installation

Run the following verification script to confirm that all critical imports work before touching any project code:

```

1  """
2  verify_install.py
3  Run this before any project script to confirm the environment is correct.
4  """
5  import sys
6
7  checks = []
8
9  def check(name, fn):
10     try:
11         fn()
12         checks.append((name, True, None))
13     except Exception as e:
14         checks.append((name, False, str(e)))
15
16  check("Python >= 3.10", lambda: assert_version())
17  check("pandas", lambda: __import__("pandas"))
18  check("numpy", lambda: __import__("numpy"))
19  check("matplotlib", lambda: __import__("matplotlib"))
20  check("seaborn", lambda: __import__("seaborn"))
21  check("sklearn", lambda: __import__("sklearn"))
22  check("geopandas", lambda: __import__("geopandas"))
23  check("osmnx", lambda: __import__("osmnx"))
24  check("shapely", lambda: __import__("shapely"))
25  check("pyproj", lambda: __import__("pyproj"))
26  check("folium", lambda: __import__("folium"))
27
28  def assert_version():
29     v = sys.version_info
30     assert v.major == 3 and v.minor >= 10, f"Need Python 3.10+, got {v}"

```



```

31
32 print(f"\n{'Package':<20} {'Status'}")
33 print("-" * 35)
34 all_ok = True
35 for name, ok, err in checks:
36     status = "OK" if ok else f"FAIL: {err}"
37     print(f"    {name:<18} {status}")
38     if not ok:
39         all_ok = False
40
41 print()
42 if all_ok:
43     print("    All checks passed. Environment is ready.")
44 else:
45     print("    Some checks failed. See errors above.")
46     sys.exit(1)

```

Listing 3.4: `verify_install.py` — run this first to confirm your environment

Run it with:

```
1 python verify_install.py
```

3.7 Setting Up the Outputs Directory

All scripts write their outputs to the `outputs/` directory in the project root. This directory is created automatically by each script if it does not exist, but you can create it manually:

```
1 mkdir -p outputs
```

The Git repository is configured to track the `outputs/` directory but to ignore large binary files. Check `.gitignore` if you are adding new output types.

3.8 Coordinate Reference Systems

This is one of the most important setup concepts in the entire project. We use two coordinate reference systems (CRS):

EPSG:4326 — WGS84 (geographic). Latitude and longitude in decimal degrees. This is the default CRS of GPS receivers, OSM data, and most web APIs. All school gate coordinates in `config.py` are stored in WGS84.

EPSG:7855 — GDA2020 / MGA Zone 55 (projected, metric). A Cartesian coordinate system in metres, covering eastern Victoria. This is the Australian standard for precise local-scale spatial analysis. All buffer distances (200 m, 400 m, 800 m) are computed in this CRS.

CRS mismatch is a silent killer

If you compute a 400 m buffer in WGS84 degrees rather than EPSG:7855 metres, you will get a buffer of the wrong size — and it will *not* raise an error. The project uses `pyproj`'s `Transformer` to explicitly reproject all geometries before every spatial operation. Never skip this step when adapting the code for a new school or LGA.

The `Transformer` is created once at module load time in `spatial_features.py`:

```

1 from pyproj import Transformer
2
3 CRS_METRIC = 'EPSG:7855' # GDA2020 / MGA zone 55
4 CRS_GEO    = 'EPSG:4326' # WGS84
5
6 _transformer = Transformer.from_crs(CRS_GEO, CRS_METRIC, always_xy=True)
7
8 def gate_to_projected_point(lat, lon):
9     """Convert WGS84 lat/lon to a Shapely Point in EPSG:7855."""
10    x, y = _transformer.transform(lon, lat)
11    return Point(x, y)

```

Listing 3.5: CRS transformation setup in spatial_features.py

The `always_xy=True` parameter is critical: without it, pyproj may swap latitude and longitude, producing incorrect coordinates.

3.9 Running the Pipeline for the First Time

Once your environment is set up and verified, run the scripts in this order:

```

1 # Step 1: Extract OSM spatial features (takes ~5 minutes)
2 python spatial_features.py
3
4 # Step 2: Extract environmental features (AQI + crime)
5 python environmental_features.py
6
7 # Step 3: Download and filter crash data (VicRoads)
8 python crash_analysis.py
9
10 # Step 4: Run the main scoring pipeline
11 python main.py
12
13 # Step 5: Build the ML feature matrix
14 python feature_engineering.py
15
16 # Step 6: Train the ML model (LOO-CV)
17 python ml_model.py
18
19 # Step 7: Equity and SEIFA analysis
20 python equity_analysis.py
21
22 # Step 8: Crash trend analysis charts
23 python crash_trend_analysis.py
24
25 # Step 9: Run a scenario (optional)
26 python scenario_analyzer.py --school "Preston HS" \
27     --interventions pedestrian_crossing speed_reduction

```

Listing 3.6: Full pipeline execution sequence

At the end of Step 4, check that `outputs/hs_scores.csv` exists and contains three rows (one per school) with all ten HS indicator columns populated.

Using pre-computed outputs

If you want to skip the slow OSM API calls (Steps 1–3), the repository includes pre-computed CSVs in `outputs/`. You can start directly from Step 4 (`main.py`) using those cached inputs.

Chapter 4

Repository Structure and Architecture

4.1 Directory Tree

The project root contains the following structure. Understanding where each file lives is essential before reading the code chapters.

```
1 300-000-School-Streets/
2 |
3 |-- config.py           # Shared constants and lookup tables
4 |-- main.py             # Master pipeline (Steps 1-6)
5 |-- spatial_features.py # OSM feature extraction
6 |-- feature_engineering.py # Build ML feature matrix
7 |-- ml_model.py         # Ridge regression + LOO-CV
8 |-- scenario_analyzer.py # What-if scenario CLI tool
9 |-- equity_analysis.py  # SEIFA equity charts
10 |-- crash_trend_analysis.py # VicRoads crash trend charts
11 |-- environmental_features.py # AQI + crime data collection
12 |-- crash_analysis.py  # Raw crash data download + filter
13 |-- school_data.csv    # Field observation data (raw input)
14 |-- demographics_darebin.csv # ABS suburb-level demographics
15 |
16 |-- src/               # Modular source packages
17 | | |-- data/
18 | | | |-- loader.py    # CSV loading + column renaming
19 | | | |-- scoring/
20 | | | | |-- __init__.py # Exports all HS compute functions
21 | | | | |-- hs1.py ... hs10.py # One file per HS indicator
22 | | | | |-- severity.py # Severity classification
23 | | | |-- recommendations/
24 | | | | |-- engine.py  # Rule-based recommendations
25 | | | |-- visualisation/
26 | | | | |-- heatmap.py # KDE heatmap (static + interactive)
27 | | | | |-- interactive_map.py # Leaflet.js interactive map
28 | | | |-- scenarios/
29 | | | | |-- engine.py  # Delta-method scenario runner
30 | | | | |-- interventions.py # Intervention definitions
31 |
32 |-- outputs/           # All generated outputs (git-tracked)
33 | | |-- hs_scores.csv  # School-level HS1-HS10 scores
34 | | |-- spatial_features.csv # OSM feature matrix
35 | | |-- ml_school_features.csv # Combined ML feature matrix
36 | | |-- ml_predictions.csv # LOO-CV predictions
37 | | |-- hs_predictor.pkl # Trained Ridge model
38 | | |-- seifa_darebin.csv # SEIFA scores per school
39 | | |-- chart*.png     # All static charts
40 | | |-- map_interactive.html # Interactive hazard map
41 | | |-- map_heatmap.html # KDE heatmap
```

```

42 | |-- networks.gpkg          # GeoPackage: OSM network layers
43 |
44 | -- docs/                  # GitHub Pages dashboard
45 | |-- index.html           # Dashboard entry point
46 | |-- data/
47 |     |-- data.json        # Aggregated data for charts
48 |
49 | -- report/
50 | |-- report.tex           # LaTeX report source
51 | |-- book.tex             # This book (LaTeX source)
52 |
53 | -- build_presentation.py  # Presentation slide builder
54 | -- .gitignore
55 | -- README.md

```

Listing 4.1: Project directory structure

4.2 The config.py Module

`config.py` is the single source of truth for every constant, path, and lookup table in the project. *Every* other module imports from it — never hardcodes its own copies of school names, gate coordinates, or indicator lists. This ensures consistency across the pipeline.

Here is the complete, annotated source:

```

1  """
2  Configuration -- paths, constants, and lookup tables shared across all
3  modules. All modules should import constants from here rather than
4  defining their own.
5  """
6  from pathlib import Path
7
8  PROJECT_ROOT = Path(__file__).parent
9
10 # -- Paths -----
11 # All paths are derived from PROJECT_ROOT so the code works regardless
12 # of where the user clones the repository.
13 CSV_FILE      = PROJECT_ROOT / 'school_data.csv'
14 DEMO_FILE     = PROJECT_ROOT / 'demographics_darebin.csv'
15 OUT_DIR       = PROJECT_ROOT / 'outputs'
16 SPATIAL_CSV   = OUT_DIR / 'spatial_features.csv'
17 ENV_CSV       = OUT_DIR / 'environmental_features.csv'
18 HS_CSV        = OUT_DIR / 'hs_scores.csv'
19
20 # -- Healthy Streets indicators -----
21 # 10 indicators (Lucy Saunders / TfL framework, adapted for school
22 # streets). Each tuple is (code, display_label). The label uses \n
23 # for multi-line chart labels in matplotlib polar charts.
24 HS_INDICATORS = [
25     ('HS1', 'Pedestrians from\nall walks of life'),
26     ('HS2', 'Easy to\ncross'),
27     ('HS3', 'Shade and\nshelter'),
28     ('HS4', 'Places to stop\nand rest'),
29     ('HS5', 'Not too\nnoisy'),
30     ('HS6', 'People choose to\nwalk / cycle / PT'),
31     ('HS7', 'People\nfeel safe'),
32     ('HS8', 'Things to\nsee and do'),
33     ('HS9', 'People\nfeel relaxed'),
34     ('HS10', 'Clean\nair'),
35 ]
36 HS_CODES      = [h[0] for h in HS_INDICATORS] # ['HS1', ..., 'HS10']
37 HS_LABELS     = [h[1] for h in HS_INDICATORS] # display labels

```

```

38
39 # -- School gates -----
40 # Main gate coordinates for all assessed schools, keyed by full name.
41 # These are used when we need the address for display purposes.
42 SCHOOL_GATES = {
43     'Reservoir High School': {
44         'lat': -37.7224, 'lon': 145.0294,
45         'addr': '855 Plenty Rd, Reservoir VIC 3073',
46     },
47     'William Ruthven Secondary College': {
48         'lat': -37.69654, 'lon': 145.00299,
49         'addr': '60 Merrilands Rd, Reservoir VIC 3073',
50     },
51     'Preston High School': {
52         'lat': -37.7417, 'lon': 145.0071,
53         'addr': '2-16 Cooma St, Preston VIC 3072',
54     },
55 }
56
57 # Full name -> short display name (chart labels, CSV matching).
58 # Charts use short names to avoid label overflow.
59 SCHOOL_SHORT_NAMES = {
60     'Reservoir High School': 'Reservoir HS',
61     'William Ruthven Secondary College': 'William Ruthven SC',
62     'Preston High School': 'Preston HS',
63 }
64
65 # Short name -> gate coords.
66 # Used by crash_analysis and spatial_features which key their gate
67 # dicts by short name rather than full name.
68 SCHOOL_GATES_BY_SHORT = {
69     'Reservoir HS': {'lat': -37.7224, 'lon': 145.0294},
70     'William Ruthven SC': {'lat': -37.69654, 'lon': 145.00299},
71     'Preston HS': {'lat': -37.7417, 'lon': 145.0071},
72 }
73
74 # -- Cycling Infrastructure Score mapping -----
75 # Reference: LTS (Mekuria, Furth & Nixon 2012);
76 # VicRoads TEM Vol. 3 Part 218.
77 CIS_MAP = {
78     'Yes -- separated bike lane': 9.0,
79     'Yes -- shared path or greenway': 8.0,
80     'Yes -- painted bike lane (on-road)': 4.5,
81     'Yes -- advisory lane / shared road': 2.0,
82     'No cycling infrastructure': 1.0,
83 }
84
85 # -- Column rename mapping -----
86 # Maps raw CSV header strings to internal short names. The raw headers
87 # are the full questions from the Google Forms field survey. The short
88 # names are what every other module uses internally.
89 COLUMN_RENAME = {
90     'School name': 'School',
91     'Overall hazard severity ...': 'Severity',
92     'Footpath Accessibility Score ...': 'FAS',
93     'Crossing Safety Score ...': 'CSS',
94     # ... (see full file for all 35 mappings)
95 }

```

Listing 4.2: config.py — full annotated source

Design principle: single source of truth

If you need to add a new school to the pipeline, you only edit `config.py`: add an entry to `SCHOOL_GATES`, `SCHOOL_SHORT_NAMES`, and `SCHOOL_GATES_BY_SHORT`. Every other module will automatically pick up the new school on the next run.

4.3 Data Flow Architecture

Figure 4.1 describes the end-to-end data flow. Data moves from left (raw inputs) to right (published outputs) through five transformation stages.

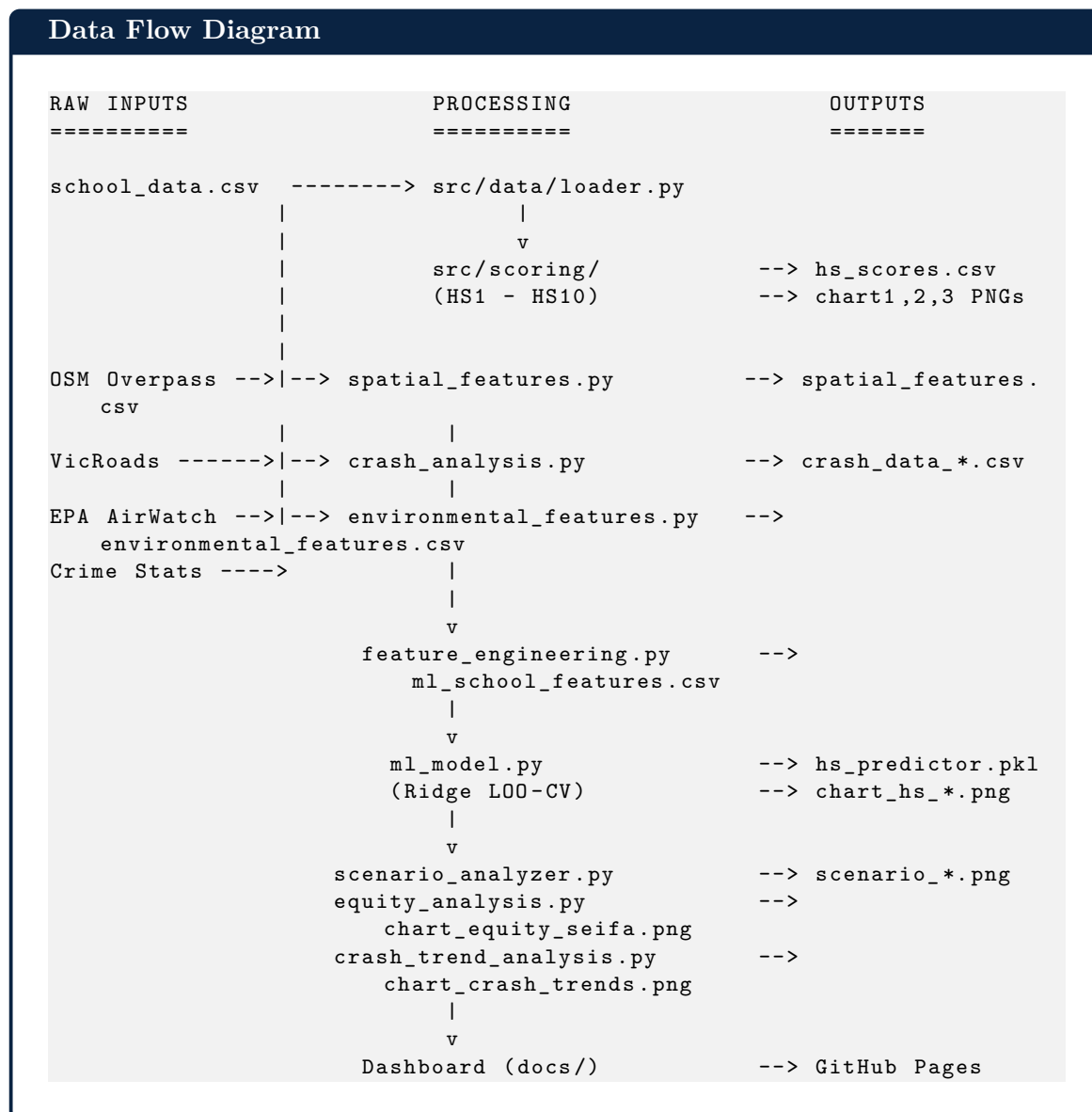


Figure 4.1: End-to-end data flow: from field survey and open APIs to published dashboard

4.4 The Six-Step Main Pipeline

The `main.py` script orchestrates the six core steps of the Healthy Streets scoring pipeline. Understanding this script is essential because it ties together all the modules in `src/`:

```

1 # STEP 1 --- Load field survey data
2 df = load_school_data(str(CSV_FILE))
3 # df has one row per observation point (multiple per school)
4
5 # STEP 2 --- Apply HS scoring (one function per indicator)
6 for idx, row in df.iterrows():
7     sp = _spatial(row['School']) # DSM features for this school
8     ev = _env(row['School']) # AQI + crime for this school
9     df.loc[idx, 'HS1'] = compute_hs1(row)
10    df.loc[idx, 'HS2'] = compute_hs2(row)
11    df.loc[idx, 'HS3'] = compute_hs3(row, sp)
12    # ... HS4 through HS10
13 df['HS_overall'] = df[HS_CODES].mean(axis=1, skipna=True).round(1)
14
15 # STEP 3 --- Generate charts (radar, bar, breakdown)
16 # See Chapter 22 of the full book for chart code.
17
18 # STEP 4 --- Generate rule-based recommendations
19 rec_df = run_recommendations(df)
20 rec_df.to_csv('outputs/recommendations.csv', index=False)
21
22 # STEP 5 --- Build interactive Leaflet.js map
23 build_interactive_map(df, rec_df, OUT_DIR_STR)
24
25 # STEP 6 --- Build KDE heatmap
26 build_kde_heatmap(df, rec_df, OUT_DIR_STR)

```

Listing 4.3: main.py — pipeline steps overview (abbreviated)

Per-row vs per-school scoring

main.py computes HS scores at the *observation level* (one row per survey point). The school-level summary is then computed by grouping by school and taking the mean across all observation points:

```

school_hs = df.groupby(['School', 'School_short'])[HS_CODES + ['
    HS_overall']] \
    .mean().round(2).reset_index()
school_hs.to_csv(str(HS_CSV), index=False)

```

This means that if a school has multiple observation points (e.g. multiple street segments around the gate), their HS scores are averaged. Preston HS had 4 observation points; the published HS scores are the mean of all four.

4.5 The src/ Package Structure

The `src/` directory contains the modular components of the pipeline. Each subdirectory is a Python package (contains an `__init__.py`):

4.5.1 src/data/loader.py

Loads `school_data.csv`, strips whitespace from column headers, applies the `COLUMN_RENAME` mapping from `config.py`, and adds a `School_short` column using `SCHOOL_SHORT_NAMES`.

```

1 def load_school_data(path: str) -> pd.DataFrame:
2     df = pd.read_csv(path)
3     df.columns = df.columns.str.strip() # remove accidental whitespace
4     df = df.rename(columns=COLUMN_RENAME) # full name -> short name
5     df['School_short'] = df['School'].map(SCHOOL_SHORT_NAMES).fillna(df['
        School'])

```

```
6 return df
```

Listing 4.4: Key logic in `src/data/loader.py`

4.5.2 `src/scoring/`

Contains one Python file per HS indicator: `hs1.py` through `hs10.py`, plus `severity.py`. Each file exposes a single function:

- `compute_hs1(row: pd.Series) -> float`
- `compute_hs2(row: pd.Series) -> float`
- `compute_hs3(row: pd.Series, sp: dict) -> float` (`sp` = spatial features)
- ...and so on.

The `__init__.py` re-exports all ten functions so that `main.py` only needs:

```
from src.scoring import (  
    compute_hs1, compute_hs2, ..., compute_hs10, compute_severity  
)
```

4.5.3 `src/recommendations/engine.py`

Implements rule-based recommendations triggered by indicator scores falling below thresholds. For each school-street combination, it generates up to five recommendations (one per failing indicator), each with a type, priority, estimated cost level, and suggested timeframe.

4.5.4 `src/visualisation/`

Two modules:

- `heatmap.py` — generates a KDE (Kernel Density Estimation) heatmap of hazard locations as both a static PNG (for QGIS import) and an interactive Folium HTML map.
- `interactive_map.py` — generates the Leaflet.js interactive hazard map with colour-coded markers, popups, and a severity filter control.

4.5.5 `src/scenarios/`

Two modules:

- `interventions.py` — defines the catalogue of available interventions (e.g. pedestrian crossing, speed reduction, bike lane) as Python dictionaries with feature delta values, cost, and timeframe.
- `engine.py` — implements the delta-method scenario runner. Given a school and a list of intervention keys, it loads the trained Ridge model, applies the feature changes, and computes predicted HS score changes.

4.6 Naming Conventions

Consistent naming is enforced across all modules:

Table 4.1: Naming conventions used throughout the codebase

Convention	Example	Applies to
<code>school_short</code>	Reservoir HS	CSV column, dict key, chart label
<code>School</code>	Reservoir High School	Raw CSV school name column
<code>HS_CODES</code>	<code>['HS1', ..., 'HS10']</code>	DataFrame column names
<code>HS_overall</code>	score 0–10	Overall mean of HS1–HS10
<code>Sev_clean</code>	Major / Moderate / Minor	Severity classification label
<code>outputs/</code>	all generated files	Every script writes here

CSV column naming is critical

The `COLUMN_RENAME` mapping in `config.py` maps verbose Google Forms column headers to short internal names. If you re-export the field survey form and the column headers change (e.g. if a question is edited), you must update `COLUMN_RENAME` accordingly. Otherwise, `loader.py` will silently leave columns unmapped, and the scoring functions will receive NaN inputs and produce incorrect scores with no error.

4.7 Git Workflow and Branch Strategy

The project uses a feature-branch workflow:

- **main** — production branch. All final outputs were generated from this branch.
- **rish_development** — development branch for Hishikesh Phukan’s contributions (ML pipeline, scenario engine, equity analysis).
- Feature branches were merged via pull requests on GitHub after code review.

The GitHub Pages dashboard is deployed from the `docs/` directory on the **main** branch. Every push to **main** that modifies `docs/` triggers an automatic rebuild of the dashboard at the project’s GitHub Pages URL.

4.8 Summary: What to Expect After a Full Pipeline Run

After running all scripts in the order shown in Section 3.8, the `outputs/` directory should contain the following files:

Table 4.2: Expected outputs after a full pipeline run

File	Size	Generated by
spatial_features.csv	~20 KB	spatial_features.py
environmental_features.csv	~2 KB	environmental_features.py
crash_data_statewide.csv	~50 MB	crash_analysis.py
seifa_darebin.csv	~2 KB	equity_analysis.py
hs_scores.csv	~3 KB	main.py
ml_school_features.csv	~5 KB	feature_engineering.py
ml_predictions.csv	~3 KB	ml_model.py
hs_predictor.pkl	~10 KB	ml_model.py
networks.gpkg	~5 MB	spatial_features.py
chart1_safety_scores.png	~300 KB	main.py
chart2_hazard_severity.png	~400 KB	main.py
chart3_score_breakdown.png	~500 KB	main.py
chart_hs_correlation.png	~400 KB	ml_model.py
chart_hs_prediction.png	~350 KB	ml_model.py
chart_feature_importance.png	~400 KB	ml_model.py
chart_equity_seifa.png	~500 KB	equity_analysis.py
chart_crash_trends.png	~500 KB	crash_trend_analysis.py
map_interactive.html	~1 MB	main.py
map_heatmap.html	~2 MB	main.py
recommendations.csv	~10 KB	main.py

If any file is missing, the chapter covering that script will explain what went wrong and how to debug it.

Part Two

Data Collection

Chapters 5–8: Study Area, Field Surveys, OSM, and Government Datasets

Chapter 5

Study Area and School Selection

5.1 The City of Darebin

The City of Darebin is an inner-northern Local Government Area (LGA) of Melbourne, Victoria. It is bounded roughly by Bell Street to the north, the Eastern Freeway to the south, Plenty Road to the east, and the Merri Creek corridor to the west. With a 2021 estimated resident population of approximately 162,000 and an area of 53 km², Darebin is one of Melbourne's most densely settled LGAs.

The LGA encompasses the suburbs of Reservoir, Preston, Thornbury, Northcote, Fairfield, Alphington, Ivanhoe East, and Macleod. Darebin is well-served by public transport: two heavy-rail lines (Hurstbridge and Mernda), four tram routes, and numerous SmartBus services run through the LGA.

Why Darebin?

Three factors drove the choice of Darebin as the pilot LGA:

1. **Socioeconomic diversity.** The LGA spans SEIFA IRSD deciles 4 through 7, enabling a meaningful equity test within a single jurisdiction.
2. **Strong active-travel potential.** Darebin has a Walk Score above 80 in much of its core and an existing local cycling network, meaning street conditions are genuinely relevant to daily commuting.
3. **Partner alignment.** Regen Melbourne's existing relationships with the City of Darebin council made field access and data sharing feasible within the project timeline.

5.2 The Three Schools

Three state secondary schools were selected for the pilot assessment. Table 5.1 provides the key attributes used throughout the pipeline.

Table 5.1: The three schools assessed: coordinates, address, and SEIFA decile.

Short name	Full name / Address	Gate lat	Gate lon	SEIFA D
Reservoir HS	Reservoir High School, 855 Plenty Rd, Reservoir VIC 3073	-37.7224	145.0294	D4
William Ruthven SC	William Ruthven Secondary College, 60 Merrilands Rd, Reservoir VIC 3073	-37.6965	145.0030	D6
Preston HS	Preston High School, 2-16 Cooma St, Preston VIC 3072	-37.7417	145.0071	D7

These three rows correspond exactly to the entries in `SCHOOL_GATES` in `config.py`. Every module in the pipeline imports gate coordinates from `config.py` to ensure a single source of truth.

Use the Main Gate, Not the Centroid

All spatial features and crash counts are measured from the *main gate* coordinate. Using the school centroid or cadastral boundary would undercount hazards that students encounter daily, because the concentration of foot traffic is at the gate, not distributed evenly around the building.

5.3 School Selection Criteria

The following criteria were applied to narrow the list of Darebin secondary schools from 12 to 3:

1. **Government school.** Private and Catholic schools were excluded to ensure a comparable socioeconomic baseline (government schools draw from local catchments; private schools draw from wider, self-selected populations).
2. **City of Darebin LGA only.** Confining to one LGA controls for differences in local council road investment and speed-zone policies.
3. **SEIFA spread.** At least three different IRSD decile bands must be represented to enable an equity regression with variation across schools.
4. **Minimum enrolment of 300.** Smaller schools generate insufficient pedestrian volumes to measure meaningful crossing utilisation rates.
5. **Gate accessibility.** The main gate must open directly onto a public footpath (no private-road or car-park-only access).
6. **Surveyor access.** A two-hour on-foot survey circuit around the 400-m catchment must be feasible in a single session without private-property access.

The final three schools satisfy all six criteria and span SEIFA deciles 4, 6, and 7.

5.4 Socioeconomic Context: SEIFA IRSD

The Australian Bureau of Statistics (ABS) publishes the **Socio-Economic Indexes for Areas (SEIFA)** after each five-year Census. This study uses the **Index of Relative Socio-economic Disadvantage (IRSD)** from the 2021 Census (released February 2023).

IRSD Decile Interpretation

Decile 1 = most disadvantaged 10% of Australian Statistical Areas Level 1 (SA1s). Decile 10 = least disadvantaged 10%. An IRSD *score* around 1000 is the national median; scores below 1000 indicate above-average disadvantage. The decile is derived by ranking all SA1s nationally by score and dividing into ten equal-count bands.

Table 5.2 shows the SEIFA values used in the equity analysis (stored in `outputs/seifa_darebin.csv`).

Table 5.2: SEIFA 2021 IRSD values for the three school catchments.

School	IRSD Score	IRSD Decile	Disadvantage Level
Reservoir HS	≈ 970	4	High
William Ruthven SC	≈ 1010	6	Moderate
Preston HS	≈ 1040	7	Low–Moderate

The equity analysis (`equity_analysis.py`) joins these SEIFA values with computed HS scores and calculates a Pearson $r \approx 0.84$ between IRSD decile and HS overall score. This is the central policy finding: more disadvantaged catchments have measurably worse pedestrian safety outcomes. Chapter 16 covers the equity analysis implementation in detail.

5.5 Catchment Definition

For this study a school catchment is the **400-metre straight-line buffer** around the main gate coordinate. This radius was adopted because:

- Australian Transport Assessment and Planning (ATAP) guidelines indicate that the majority of students who walk arrive from within a 400-m radius of the gate.
- The 400-m zone encompasses the most common school-zone signage, pedestrian crossing infrastructure, and drop-off/pick-up activity observed at all three sites.
- A single surveyor can walk and record every footpath segment within a 400-m radius in approximately two hours.

Larger buffer radii (800 m) are also computed in `spatial_features.py` to capture cycling network length, arterial road exposure, and green space percentage — indicators that require wider context. The 200-m buffer provides a tight “immediate gate zone” metric for crossing density and signal proximity.

Chapter 6

Field Survey Methodology

6.1 Why a Primary Survey?

The Healthy Streets assessment requires observations that are not available from any open dataset: footpath surface condition, presence of tactile indicators, perceived lighting quality, on-street parking conflicts, and hazard type. These are inherently *on-the-ground* observations.

The project therefore combined two data-collection methods:

1. **Physical site visit.** A two-hour structured walk of every footpath segment within the 400-m catchment, using a standardised Google Forms survey on a smartphone.
2. **Google Street View audit.** Remote inspection of segments that could not be accessed in person (e.g. road works, private-property constraints), using the most recent Street View imagery. The `SV_date` column records the imagery date for audit trail purposes.

6.2 The Google Forms Instrument

The survey was built as a Google Form with branching logic. Each row in the resulting CSV corresponds to one *location observation* — a 50-to-100-metre stretch of footpath or a specific intersection. A single session produces between 15 and 40 observations per school.

The form was designed with the following principles:

- **Dropdown-first.** Every categorical field uses a fixed dropdown to prevent free-text variation that complicates parsing.
- **Numeric ranges.** Continuous fields (width, distance, speed limit) accept numeric input with explicit units in the label.
- **Observer-entered scores.** Three summary scores (FAS, CSS, EEI) are entered by the surveyor at the end of each location observation. The pipeline *overwrites* these with computed values but preserves them in the `_obs` columns for audit.
- **Mandatory school tag.** The “School name” field is required and uses a dropdown to prevent inconsistent naming.

6.3 Column Rename Reference

The raw CSV exported from Google Forms uses full question text as column headers. The data loader maps these to short internal names via the `COLUMN_RENAME` dict in `config.py`. Table 6.1 is the complete mapping.

Table 6.1: Complete COLUMN_RENAME mapping: raw CSV header to internal name.

Raw CSV header (Google Forms)	Internal name	Description
School name	School	Full school name (dropdown)
Street or location being assessed	Street	Street name / intersection label
Latitude (decimal degrees)	Latitude	GPS latitude (auto-captured or typed)
Longitude (decimal degrees)	Longitude	GPS longitude
Approximate distance from school gate (metres)	Distance_gate	Straight-line metres from main gate
Footpath present?	Footpath_present	Yes / No / Partial
Footpath width (metres)	Footpath_width	Numeric, metres
Footpath continuity along this segment	Continuity	Continuous / Interrupted / Absent
Footpath condition rating	FP_condition	Good / Fair / Poor / Very Poor
Kerb ramps present at intersections?	Kerb_ramps	Yes / No / Partial
Obstructions on footpath?	Obstructions	Yes / No (bins, poles, parked vehicles)
Pedestrian crossing present at this location?	Crossing_present	Yes / No
Distance of nearest crossing from school gate (metres)	Crossing_dist	Numeric, metres
Crossing visibility rating	Visibility	Good / Fair / Poor
Crossing condition / maintenance rating	Cross_condition	Good / Fair / Poor
Tactile ground surface indicators (yellow bumps)?	Tactile	Yes / No
Pedestrian signal or countdown timer?	Signal	Yes / No / Pedestrian-activated
Posted speed limit (km/h)	Speed_limit	Numeric: 40, 50, 60, 80, 100
School zone active at this location?	School_zone	Yes / No
Number of traffic lanes	Lanes	Numeric integer
Estimated traffic volume during school hours	Traffic_volume	Low / Medium / High / Very High
Heavy vehicles or trucks present?	Heavy_vehicles	Yes / No
Traffic calming measures present?	Traffic_calming	Yes / No / Partial
On-street parking conflicts with pedestrians?	Parking_conflict	Yes / No
Street lighting quality	Lighting	Good / Fair / Poor / None
Cycling infrastructure present?	Cycling_infra	Dropdown (5 CIS options, see Ch. 2)
Hazard types observed at this location (select all that apply)	Hazard_types	Multi-select; comma-separated in CSV
Detailed hazard description	Hazard_desc	Free text

(continued on next page)

(continued from previous page)

Raw CSV header	Internal name	Description
Recommended intervention type	Rec_type	Dropdown (footpath / crossing / etc.)
Recommendation priority level	Priority	High / Medium / Low
Estimated cost level	Cost_level	Low (<\$10k) / Medium / High
Suggested implementation timeframe	Timeframe	Immediate / Short-term / Long-term
Detailed intervention description	Rec_desc	Free text
Expected benefit of this intervention	Benefit	Free text
Overall hazard severity at this location	Severity	Major / Moderate / Minor
Footpath Accessibility Score (0 to 10)	FAS	Observer-entered; overwritten by pipeline
Crossing Safety Score (0 to 10)	CSS	Observer-entered; overwritten by pipeline
Environmental Exposure Indicator (0 to 10)	EEI	Observer-entered; overwritten by pipeline
Primary data source used for this observation	Data_source	Field / Street View / Both
Google Street View image date (if used)	SV_date	YYYY-MM (or blank if field-only)

Why Observer Scores Are Preserved

The data loader in `src/data/loader.py` copies the observer-entered FAS, CSS, and EEI values into `FAS_obs`, `CSS_obs`, and `EEI_obs` *before* the pipeline overwrites the originals with algorithmically computed values. This allows the audit section of the report to compare how closely the surveyor’s field judgement matches the formula-derived score — a quality-control step that builds confidence in both the instrument and the algorithm.

6.4 Site Visit Protocol

Each site visit follows a fixed protocol:

Equipment. Smartphone with Google Forms offline mode enabled, measuring wheel or digital range finder, high-visibility vest, and a printed catchment map showing all segments to assess.

Timing. Visits are conducted during school arrival or departure to observe actual pedestrian behaviour. Avoid mid-day sessions; the HS indicators require school-hours context.

Route. Walk every footpath segment within the 400-m buffer in a systematic grid pattern. Do not skip segments even if they appear safe; absences are data.

One row per segment. Submit a new form response for each 50–100 m segment or intersection. Short segments can be combined if conditions are uniform throughout.

GPS capture. The form auto-captures GPS coordinates on submission. If GPS accuracy is

poor (>15 m), type coordinates from Google Maps manually.

Photo log. Photograph every hazard and every crossing. Photos are stored locally; filenames match the form response timestamp for traceability.

Duration. Allow two hours per school. A 400-m buffer typically contains 20–35 segments.

6.5 The Data Loader: `src/data/loader.py`

After the survey CSV is exported from Google Sheets, it is read by `load_school_data()` in `src/data/loader.py`. This is the first function called in `main.py`.

```

1 from pathlib import Path
2 from typing import Union
3 import pandas as pd
4 from config import COLUMN_RENAME, SCHOOL_SHORT_NAMES
5
6
7 def _normalise_severity(s: str) -> str:
8     """Map a free-text severity string to Major / Moderate / Minor / Unknown.
9     """
10    s = str(s).lower()
11    if 'major' in s: return 'Major'
12    if 'moderate' in s: return 'Moderate'
13    if 'minor' in s: return 'Minor'
14    return 'Unknown'
15
16 def load_school_data(csv_path: Union[str, Path]) -> pd.DataFrame:
17     df = pd.read_csv(csv_path)
18     df.columns = df.columns.str.strip()
19     df = df.rename(columns=COLUMN_RENAME)
20
21     # Add short display name (Reservoir HS, William Ruthven SC, etc.)
22     df['School_short'] = (
23         df['School'].map(SCHOOL_SHORT_NAMES).fillna(df['School'])
24     )
25
26     # Preserve observer-entered scores before pipeline overwrites them
27     df['FAS_obs'] = pd.to_numeric(df['FAS'], errors='coerce')
28     df['CSS_obs'] = pd.to_numeric(df['CSS'], errors='coerce')
29     df['EEI_obs'] = pd.to_numeric(df['EEI'], errors='coerce')
30     df['Sev_obs'] = df['Severity'].apply(_normalise_severity)
31
32     return df

```

Listing 6.1: Data loading and cleaning — `src/data/loader.py`.

The key steps are:

1. **Strip column whitespace.** Google Forms sometimes adds trailing spaces to header text. `str.strip()` prevents silent rename failures.
2. **Rename columns.** `df.rename(columns=COLUMN_RENAME)` replaces every long question string with its short internal name in a single operation.
3. **Map short names.** `SCHOOL_SHORT_NAMES` converts “Reservoir High School” to “Reservoir HS” etc., ensuring chart labels and CSV join keys are consistent.

4. **Preserve observer scores.** The `_obs` columns are a snapshot of raw surveyor input. After this function returns, `main.py` calls the scoring engine which *replaces* FAS, CSS, and EEI with computed values.
5. **Normalise severity.** `_normalise_severity()` handles capitalization and partial strings so “MAJOR HAZARD” and “major” both map to ‘Major’.

Chapter 7

OSM Data with osmnx

7.1 OpenStreetMap as Infrastructure Data

OpenStreetMap (OSM) is a crowdsourced, freely-licensed spatial database that covers roads, footpaths, crossings, traffic signals, street furniture, land use, and public transport at city-scale. For pedestrian safety analysis it provides three things that government datasets do not:

- **Footpath network topology.** OSM tags footpaths as `highway=footway`, `path`, `pedestrian`, etc., enabling network-graph analysis of walking route connectivity.
- **Crossing and signal locations.** OSM nodes tagged `highway=crossing` and `highway=traffic_signal` give a point layer of every formal crossing within any buffer.
- **Street furniture.** Trees (`natural=tree`), benches (`amenity=bench`), shelters (`amenity=shelter`), and bus stops (`highway=bus_stop`) support the shade, rest, and liveability indicators.

The `osmnx` library (Boeing, 2017) wraps the OSM Overpass API and returns data as `NetworkX` graphs or `GeoPandas` `GeoDataFrames`, making it straightforward to compute buffer-based counts and network metrics in Python.

Overpass API Rate Limits

The Overpass API is a public service. `osmnx` respects a 1-second delay between consecutive requests by default. For three schools the full spatial feature computation takes approximately 3–5 minutes. Running the script repeatedly in a short window may trigger a temporary block. If you see `InsufficientResponseError`, wait 60 seconds and retry.

7.2 The osmnx Library

Install `osmnx` with:

```
1 conda install -c conda-forge osmnx=1.9.3
```

The two functions used most in this project are:

- `ox.graph_from_point((lat, lon), dist=r, network_type='walk')` — returns a `NetworkX` `MultiDiGraph` of the walking network within `dist` metres.
- `ox.features_from_point((lat, lon), tags={...}, dist=r)` — returns a `GeoDataFrame` of OSM features matching the tag filter within `dist` metres.

The `network_type` parameter accepts 'walk', 'drive', 'bike', or 'all'. Each type filters the OSM highway tags to the relevant road classes.

7.3 CRS and Projection

All OSM data is returned in **WGS84 (EPSG:4326)** — geographic coordinates in decimal degrees. Distance calculations (buffer radii, edge lengths) require a *projected* CRS with metric units.

This project uses **GDA2020 / MGA zone 55 (EPSG:7855)**, the standard projected CRS for Victoria. The `pyproj Transformer` is initialised once at module load time:

```

1 from pyproj import Transformer
2
3 CRS_METRIC = 'EPSG:7855' # GDA2020 / MGA zone 55 -- Victoria
4 CRS_GEO    = 'EPSG:4326' # WGS84 geographic
5
6 _transformer = Transformer.from_crs(
7     CRS_GEO, CRS_METRIC, always_xy=True
8 )
9
10
11 def gate_to_projected_point(lat, lon):
12     """Convert WGS84 lat/lon to a Shapely Point in EPSG:7855."""
13     x, y = _transformer.transform(lon, lat)
14     return Point(x, y)

```

Listing 7.1: CRS constants and transformer in `spatial_features.py`.

always_xy=True Is Essential

`Transformer.from_crs()` defaults to respecting each CRS's axis order. EPSG:4326 specifies latitude-first, but Shapely and most geometric operations expect (x, y) = (longitude, latitude). Passing `always_xy=True` forces the transformer to accept (lon, lat) input regardless of CRS axis convention. Omitting this flag silently swaps your coordinates, producing gate points that are hundreds of kilometres from Darebin.

7.4 Buffer Strategy

Three buffer radii are queried for each school gate:

Radius	Purpose
200 m	Immediate gate zone: crossing density, signal proximity
400 m	Primary catchment: all HS indicator spatial features
800 m	Wider context: cycling network, arterial exposure, green space

The `RADII` constant is defined as:

```

1 RADII = [200, 400, 800] # metres

```

The outer loop in `compute_features()` iterates over all three radii, computing the same set of metrics at each scale. The HS-specific extras (trees, benches, PT stops, etc.) are queried only at the 400-m radius.

7.5 Walking Network Features

```

1 FOOTPATH_TYPES = {
2     'footway', 'path', 'pedestrian',
3     'cycleway', 'steps', 'bridleway'
4 }
5
6 for r in RADII:
7     try:
8         G_walk = ox.graph_from_point(
9             (lat, lon), dist=r,
10            network_type='walk', retain_all=True
11        )
12        edges_walk = ox.graph_to_gdfs(G_walk, nodes=False).to_crs(CRS_METRIC)
13        walk_edges = len(edges_walk)
14        walk_length = edges_walk.geometry.length.sum()
15
16        fp_mask = edges_walk['highway'].apply(
17            lambda v: _highway_matches(v, FOOTPATH_TYPES))
18        fp_length = edges_walk.loc[fp_mask, 'geometry'].length.sum()
19        fp_pct = (fp_length / walk_length * 100) if walk_length > 0 else
20            0.0
21    except Exception:
22        pass

```

Listing 7.2: Walking network computation per radius.

The helper `_highway_matches(val, type_set)` handles the OSM quirk where a single edge may have a *list* of highway tag values (e.g. `['footway', 'path']`):

```

1 def _highway_matches(val, type_set):
2     if isinstance(val, list):
3         return any(v in type_set for v in val)
4     return val in type_set

```

Listing 7.3: OSM list-value helper.

Four features are recorded per radius:

Feature	Definition
<code>walk_edges_Rm</code>	Count of walk-network edge segments
<code>walk_length_Rm</code>	Total walk-network length (metres)
<code>footpath_length_Rm</code>	Total length of dedicated footpath edges (metres)
<code>footpath_pct_Rm</code>	$\text{footpath_length} / \text{walk_length} \times 100$

7.6 Road Network Features

```

1 ARTERIAL_TYPES = {
2     'primary', 'secondary', 'trunk',
3     'primary_link', 'secondary_link', 'trunk_link'
4 }
5
6 def _parse_speed(val):
7     """Parse OSM maxspeed to km/h float. Returns np.nan on failure."""
8     if isinstance(val, list):
9         val = val[0]
10    if pd.isna(val):
11        return np.nan

```

```

12     s = str(val).strip().lower()
13     if 'mph' in s:
14         try:
15             return float(s.replace('mph', '').strip()) * 1.60934
16         except ValueError:
17             return np.nan
18     try:
19         return float(s.replace('km/h', '').replace('kmh', '').strip())
20     except ValueError:
21         return np.nan
22
23     # In the main loop:
24     G_drive = ox.graph_from_point((lat, lon), dist=r,
25                                 network_type='drive', retain_all=True)
26     edges_drive = ox.graph_to_gdfs(G_drive, nodes=False).to_crs(CRS_METRIC)
27     road_count = len(edges_drive)
28     arterial_mask = edges_drive['highway'].apply(
29         lambda v: _highway_matches(v, ARTERIAL_TYPES))
30     arterial_count = int(arterial_mask.sum())
31     arterial_pct = (arterial_count / road_count * 100) if road_count > 0
32                     else 0.0
33     speeds = edges_drive['maxspeed'].apply(_parse_speed).dropna()
34     avg_speed = speeds.mean() if len(speeds) > 0 else np.nan
35     high_speed = int((speeds >= 60).sum()) if len(speeds) > 0 else 0

```

Listing 7.4: Road network and speed parsing per radius.

`_parse_speed()` handles three OSM maxspeed formats: plain integer ("50"), with unit ("50 km/h"), and imperial ("30 mph").

Five features are recorded per radius:

Feature	Definition
road_count_Rm	Count of all driveable edge segments
arterial_count_Rm	Count of primary/secondary/trunk edges
arterial_pct_Rm	$\text{arterial_count} / \text{road_count} \times 100$
avg_speed_Rm	Mean posted speed limit (km/h)
high_speed_road_Rm	Count of edges with maxspeed ≥ 60 km/h

7.7 Crossings and Signals

Pedestrian crossings and traffic signals are queried as point features via the Overpass API. After retrieval, an explicit spatial intersection with the metric buffer is applied to exclude features returned by Overpass that fall just outside the true circular buffer:

```

1     center = gate_to_projected_point(lat, lon)
2
3     # Crossings
4     try:
5         crossings_gdf = ox.features_from_point(
6             (lat, lon), tags={'highway': 'crossing'}, dist=r)
7         crossings_gdf = crossings_gdf.to_crs(CRS_METRIC)
8         buf = center.buffer(r)
9         crossing_count = int(
10             crossings_gdf[crossings_gdf.geometry.intersects(buf)].shape[0])
11     except Exception:
12         crossing_count = np.nan
13
14     # Signals
15     try:

```

```

16     signals_gdf = ox.features_from_point(
17         (lat, lon), tags={'highway': 'traffic_signals'}, dist=r)
18     signals_gdf = signals_gdf.to_crs(CRS_METRIC)
19     signal_count = int(
20         signals_gdf[signals_gdf.geometry.intersects(buf)].shape[0])
21 except Exception:
22     signal_count = np.nan
23
24     # Derived: crossings per km of walkable path
25     if not np.isnan(crossing_count) and walk_length > 0:
26         feat[f'crossing_density_{r}m'] = round(
27             crossing_count / (walk_length / 1000), 2)

```

Listing 7.5: Crossing and signal counts per radius.

The `crossing_density` derived feature is the key input to the HS2 (Easy to Cross) indicator: more crossings per kilometre of path suggests better pedestrian crossing provision.

7.8 Healthy Streets Extras at 400 m

Eight additional features are computed at the 400-m radius only. They feed the shade (HS3), rest (HS4), active travel (HS6), and liveability (HS8) indicators:

Feature	Radius	OSM query
tree_count_100m	100 m	{'natural': 'tree'}
shelter_count_200m	200 m	{'amenity': ['shelter', 'bus_station']}
green_pct_400m	400 m	{'leisure': ['park', 'garden', ...]}
bench_count_200m	200 m	{'amenity': 'bench'}
park_count_400m	400 m	{'leisure': ['park', 'playground', ...]}
pt_stops_400m	400 m	{'highway': 'bus_stop', 'railway': 'tram_stop'}
amenity_count_400m	400 m	cafes, restaurants, libraries, community centres
cafe_count_400m	400 m	{'amenity': ['cafe', 'restaurant', 'fast_food']}

Green Space Proxy

OSM does not provide land-use area fractions. The `green_pct_400m` feature uses a proxy: `min(count * 5.0, 40.0)`. Each tagged green-space polygon counts as approximately 5 percentage points of green coverage, capped at 40%. This is a rough estimate; if your study area has high-quality land-use raster data (e.g. the Victorian Government's Digital Earth Victoria product), substituting a true green-fraction measurement will improve HS3 and HS9 scoring accuracy.

7.9 Cycling Network Features

```

1     try:
2         G_bike = ox.graph_from_point(
3             (lat, lon), dist=r, network_type='bike', retain_all=True)

```



```

4     edges_bike = ox.graph_to_gdfs(G_bike, nodes=False).to_crs(
        CRS_METRIC)
5     cycle_length = edges_bike.geometry.length.sum()
6
7     def _is_protected(e):
8         hw = e.get('highway', '')
9         cy = e.get('cycleway', '')
10        if isinstance(hw, list): hw = hw[0] if hw else ''
11        if isinstance(cy, list): cy = cy[0] if cy else ''
12        return hw == 'cycleway' or str(cy) in (
13            'track', 'separate', 'separated')
14
15    protected_mask = edges_bike.apply(_is_protected, axis=1)
16    protected_length = edges_bike.loc[protected_mask, 'geometry'].length.
        sum()
17    except Exception:
18        pass

```

Listing 7.6: Cycling network and protected infrastructure detection.

Three cycling features are recorded per radius:

Feature	Definition
cycle_length_Rm	Total cycling network length (metres)
protected_cycle_length_Rm	Length of protected / separated infrastructure
cycle_pct_Rm	$\text{cycle_length} / \text{walk_length} \times 100$

7.10 GeoPackage Export

After all schools are processed, network geometries are saved to `outputs/networks.gpkg` for QGIS visualisation:

```

1 NETWORKS_GPKG = 'outputs/networks.gpkg'
2
3 layer_labels = {
4     'walk_400m': 'Walk Network 400m',
5     'walk_800m': 'Walk Network 800m',
6     'cycling_400m': 'Cycling Network 400m',
7     'cycling_800m': 'Cycling Network 800m',
8     'roads_400m': 'Arterial Roads 400m',
9     'roads_800m': 'Arterial Roads 800m',
10 }
11 for key, label in layer_labels.items():
12     parts = all_geoms[key]
13     if not parts:
14         continue
15     gdf = gpd.GeoDataFrame(pd.concat(parts, ignore_index=True), crs='EPSG
        :4326')
16     gdf.to_file(NETWORKS_GPKG, layer=key, driver='GPKG')

```

Listing 7.7: GeoPackage layer export.

Open `networks.gpkg` in QGIS via *Layer* → *Add Layer* → *Add Vector Layer*. Each of the six layers will appear as a separate entry in the layer panel.

7.11 Full Feature Inventory

Table 7.1 lists every column written to `outputs/spatial_features.csv`.

Table 7.1: All spatial features output by `spatial_features.py` ($R = 200, 400$, or 800 m).

Column name	Radii	Description
<code>school_name</code>	—	School short name
<code>gate_lat, gate_lon</code>	—	Gate coordinates
<code>walk_edges_Rm</code>	3	Walk-network edge count
<code>walk_length_Rm</code>	3	Walk-network total length (m)
<code>footpath_length_Rm</code>	3	Dedicated footpath length (m)
<code>footpath_pct_Rm</code>	3	Footpath as % of walk network
<code>road_count_Rm</code>	3	Driveable edge count
<code>arterial_count_Rm</code>	3	Primary/secondary/trunk edge count
<code>arterial_pct_Rm</code>	3	Arterial as % of road network
<code>avg_speed_Rm</code>	3	Mean posted speed limit (km/h)
<code>high_speed_road_Rm</code>	3	Edges with maxspeed ≥ 60 km/h
<code>crossings_Rm</code>	3	Pedestrian crossing count
<code>signals_Rm</code>	3	Traffic signal count
<code>crossing_density_Rm</code>	3	Crossings per km of walk path
<code>cycle_length_Rm</code>	3	Cycling network total length (m)
<code>protected_cycle_Rm</code>	3	Protected / separated cycle length (m)
<code>cycle_pct_Rm</code>	3	Cycling as % of walk network
<code>tree_count_100m</code>	400	Trees within 100 m (HS3)
<code>shelter_count_200m</code>	400	Shelters within 200 m (HS3)
<code>green_pct_400m</code>	400	Green space proxy % (HS3/HS9)
<code>bench_count_200m</code>	400	Benches within 200 m (HS4)
<code>park_count_400m</code>	400	Parks/playgrounds within 400 m (HS4/HS8)
<code>pt_stops_400m</code>	400	PT stops within 400 m (HS6)
<code>amenity_count_400m</code>	400	Shops/cafes/community within 400 m (HS8)
<code>cafe_count_400m</code>	400	Cafes/restaurants within 400 m (HS8)

Chapter 8

Government Open Datasets

8.1 Overview

Four government datasets supplement the field survey and OSM data. Table 8.1 summarises each source.

Table 8.1: Government open datasets used in the pipeline.

Dataset	Provider	Used for	Output file
Crash data (ped/cyc)	VicRoads / data.vic.gov.au	HS5/HS7 safety; trend analysis	crash_data_statewide.csv
SEIFA 2021 IRSD	ABS	Equity analysis	seifa_darebin.csv
AirWatch PM2.5	EPA Victoria	HS10 clean air	environmental_features.csv
Crime statistics	Crime Statistics Agency Vic.	HS7 personal safety	environmental_features.csv

8.2 VicRoads Crash Data

8.2.1 Source and Download

VicRoads publishes the Victoria Road Crash Database on the Victorian Government Data Directory (data.vic.gov.au). The dataset contains every recorded road traffic crash in Victoria from 2013 onward, with columns for crash date, time, severity, LGA, road name, and crash type (pedestrian, cyclist, motor vehicle).

Download Steps

1. Navigate to <https://discover.data.vic.gov.au> and search for “**Victoria Road Crash Data**”.

2. Select the most recent annual CSV release (e.g. CrashStats 2021–2025).

3. Download and save to `outputs/crash_data_statewide.csv`.

4. The file is approximately 60–80 MB uncompressed.

8.2.2 Key Columns

The pipeline uses the following columns:

Column	Description
ACCIDENT_DATE	Date string; parsed to year via <code>pd.to_datetime()</code>
ACCIDENT_TIME	Time string (HH:MM:SS); used for school-hours classification
SEVERITY	Integer code: 1 = Fatal, 2 = Serious injury, 3+ = Minor
LGA_NAME	Upper-cased LGA name; filtered to 'DAREBIN'
DAY_OF_WEEK	Integer: 1 = Sunday, 2 = Monday, ..., 7 = Saturday
nearest_school	School name nearest to crash location (added via spatial join in pre-processing)

8.2.3 School Hours Filter

The `_is_school_hours()` function in `crash_trend_analysis.py` classifies each crash as occurring during school arrival or departure windows:

```

1 def _is_school_hours(row):
2     try:
3         t = pd.to_datetime(row['ACCIDENT_TIME'], format='%H:%M:%S')
4         h, m = t.hour, t.minute
5         wday = int(row['DAY_OF_WEEK'])      # 1=Sun, 2=Mon..6=Fri, 7=Sat
6         if wday in (1, 7):                  # exclude weekends
7             return False
8         morning = (h == 7 and m >= 30) or h == 8
9         afternoon = ((h == 14 and m >= 30) or
10                     h in (15, 16) or
11                     (h == 16 and m == 0))
12         return morning or afternoon
13     except Exception:
14         return False

```

Listing 8.1: School hours classification logic.

This encodes the two school movement windows:

- **Morning:** 07:30–09:00 Monday–Friday
- **Afternoon:** 14:30–16:30 Monday–Friday

Weekends are excluded regardless of time. The school-hours fraction is a key input to the HS5 (not too noisy) and HS7 (feel safe) indicators.

8.2.4 Crash Trend Analysis

`crash_trend_analysis.py` produces a four-panel chart (`outputs/chart_crash_trends.png`) covering:

1. **Darebin LGA by year** — stacked bar chart (Fatal / Serious / Minor) showing the annual trend from 2021 to 2025.
2. **Three schools by year** — grouped bars showing crash counts within 400 m of each school gate per year.
3. **School hours vs off-peak** — side-by-side bars per school; reveals whether crashes are concentrated in student movement windows.

4. **Time-of-day distribution** — 24-hour bar chart for all Darebin LGA crashes, with school hours highlighted in red and general peak hours in amber.

Run it with:

```
1 python crash_trend_analysis.py
```

8.3 ABS SEIFA 2021 IRSD

8.3.1 Source and Download

The 2021 SEIFA release is available from the ABS website. The IRSD is published at SA1, SA2, LGA, and postcode levels. This project uses the **LGA-level** IRSD score and decile.

Download Steps

1. Go to abs.gov.au and search for “**SEIFA 2021**”.
2. Download “Table 3: SEIFA 2021 Index of Relative Socio-economic Disadvantage by Local Government Area”.
3. Extract the relevant rows for Darebin and save as `outputs/seifa_darebin.csv` with columns: `School`, `IRSD_Score_Weighted`, `IRSD_Decile`, `Disadvantage_Level`.

8.3.2 seifa_darebin.csv Structure

The file is a manually assembled CSV joining each school to the IRSD statistics of its SA2 catchment:

```
1 School,IRSD_Score_Weighted,IRSD_Decile,Disadvantage_Level
2 Reservoir HS,970,4,High
3 William Ruthven SC,1010,6,Moderate
4 Preston HS,1040,7,Low-Moderate
```

8.3.3 Equity Analysis

`equity_analysis.py` loads and joins this file with `outputs/hs_scores.csv`:

```
1 def load_and_join():
2     seifa = pd.read_csv('outputs/seifa_darebin.csv')
3     hs     = pd.read_csv('outputs/hs_scores.csv')
4     df     = seifa.merge(hs, on='School', how='inner')
5     return df.sort_values('IRSD_Decile').reset_index(drop=True)
6
7 # Pearson correlation between decile and HS overall:
8 corr = df['IRSD_Decile'].corr(df['HS_overall'])
9 # r > 0.3 => higher decile (less disadvantaged) = higher HS (safer)
```

Listing 8.2: SEIFA join and equity correlation.

The output chart (`outputs/chart_equity_seifa.png`) has three panels: a side-by-side bar chart of SEIFA decile vs HS overall per school, a scatter plot of the two variables, and a full indicator heatmap (HS1–HS10 in RdYlGn colour scale).

8.4 EPA AirWatch PM2.5

8.4.1 Source

The Environment Protection Authority (EPA) Victoria publishes real-time and historical air quality data via the AirWatch portal (<https://www.epa.vic.gov.au/for-community/airwatch>).

PM2.5 (fine particulate matter, $\leq 2.5 \mu\text{m}$) is the metric used for HS10 (clean air). Annual mean PM2.5 in $\mu\text{g}/\text{m}^3$ is downloaded for the nearest EPA monitoring station to each school:

School	Nearest EPA station	Approx. distance
Reservoir HS	Alphington	~4 km
William Ruthven SC	Alphington	~6 km
Preston HS	Alphington	~3 km

8.4.2 Preprocessing

Annual-mean PM2.5 values are stored in `outputs/environmental_features.csv` with columns `school_name` and `aqi_pm25`. The HS10 scoring function applies the following lookup:

PM2.5 ($\mu\text{g}/\text{m}^3$)	Category	HS10 score
< 8	Clean	8–10
8–12	Moderate	5–7
12–25	Elevated	3–4
> 25	Poor	1–2

The WHO 2021 guideline for annual mean PM2.5 is $5 \mu\text{g}/\text{m}^3$; the Australian NEPM standard is $8 \mu\text{g}/\text{m}^3$. Darebin monitoring sites typically record annual means of 6–10 $\mu\text{g}/\text{m}^3$, placing them in the clean-to-moderate range.

8.5 Crime Statistics Victoria

8.5.1 Source

The Crime Statistics Agency (CSA) Victoria publishes LGA-level offence counts annually at <https://www.crimestatistics.vic.gov.au>. The metric used in this project is **crime rate per 100,000 population**, derived from total offences and ABS residential population.

8.5.2 Preprocessing

The crime rate is assigned at the LGA level — all three schools are in the City of Darebin, so they share the same `crime_rate_per_100k` value. This value is stored in `outputs/environmental_features.csv` alongside the PM2.5 values.

```

1 school_name,aqi_pm25,crime_rate_per_100k
2 Reservoir HS,8.4,4230.5
3 William Ruthven SC,8.4,4230.5
4 Preston HS,7.9,4230.5
```

8.5.3 Usage in the ML Model

Both `aqi_pm25` and `crime_rate_per_100k` are used as features in the machine-learning model (`feature_engineering.py`), where they appear in the `ENV_FEATURES` list:

```
1 ENV_FEATURES = ['crime_rate_per_100k', 'aqi_pm25']
```

They supplement the 20 OSM spatial features and 4 crash aggregate features to form the 26-feature matrix fed to the Ridge regression model. Chapter 14 covers the full feature engineering and model training workflow.

8.6 Joining All Data Sources

The keybox below summarises how all four government datasets join to the school catchment table.

Data Join Summary

All datasets converge on `school_name` (or `School`) as the primary join key. There is no spatial join required for SEIFA, PM2.5, or crime statistics because these are already LGA-level aggregates. The crash data *does* require a spatial join (nearest-school assignment) which is performed as a pre-processing step: each crash is assigned to the school whose gate is nearest within a 400-m radius. Crashes beyond 400 m from any gate are excluded.

The recommended joining order is:

1. Load `school_data.csv` via `load_school_data()`.
2. Run `spatial_features.py` → `outputs/spatial_features.csv`.
3. Run `feature_engineering.py` to merge spatial + environmental + crash features → `outputs/ml_school_features.csv`.
4. Run `equity_analysis.py` which merges SEIFA + HS scores independently.
5. Run `crash_trend_analysis.py` for time-series reporting (separate from the ML pipeline).

Part Three

Scoring and Analysis

Chapters 9–12: HS Scoring Engine, Recommendations, ML Model, and Scenario Analysis

Chapter 9

The Healthy Streets Scoring Engine

9.1 Architecture of src/scoring/

The scoring engine lives entirely in `src/scoring/`. There is one file per indicator:

```
1 src/scoring/  
2   __init__.py           # re-exports compute_hs1 ... compute_hs10,  
                           compute_severity  
3   hs1.py               # HS1 -- Pedestrians from all walks of life  
4   hs2.py               # HS2 -- Easy to cross  
5   hs3.py               # HS3 -- Shade and shelter (needs spatial dict)  
6   hs4.py               # HS4 -- Places to stop and rest  
7   hs5.py               # HS5 -- Not too noisy  
8   hs6.py               # HS6 -- People choose to walk / cycle / PT  
9   hs7.py               # HS7 -- People feel safe  
10  hs8.py               # HS8 -- Things to see and do  
11  hs9.py               # HS9 -- People feel relaxed  
12  hs10.py              # HS10 -- Clean air  
13  severity.py           # Derives Major / Moderate / Minor from HS scores  
14  cys.py                # Cycling yield score helper (used by HS6)
```

Each `compute_hsN()` function accepts a single DataFrame row (and, for HS3/HS4/HS6/HS8, an additional `sp` dict of spatial features) and returns a float in `[0.0, 10.0]`.

Two Scoring Patterns

Additive: starts at 0.0 and adds points for positive evidence (HS1, HS2, HS3, HS4, HS6, HS7, HS8).

Deductive: starts at 10.0 and subtracts for negative conditions (HS5, HS9, HS10). Both patterns clamp the result to `[0.0, 10.0]` via `min(max(pts, 0.0), 10.0)`.

9.2 How main.py Calls the Engine

`main.py` calls each scoring function inside a vectorised `.apply()` over the loaded DataFrame, then groups observations by school to produce a school-level mean:

```
1 from src.scoring.hs1 import compute_hs1  
2 from src.scoring.hs2 import compute_hs2  
3 # ... imports for hs3..hs10 ...  
4 from src.scoring.severity import compute_severity  
5  
6 df['HS1'] = df.apply(compute_hs1, axis=1)  
7 df['HS2'] = df.apply(compute_hs2, axis=1)  
8 df['HS3'] = df.apply(lambda r: compute_hs3(r, sp.get(r['School_short'], {})),  
9                       axis=1)  
9 # ... similarly for HS4..HS10 ...
```

```

10 HS_CODES = ['HS1', 'HS2', 'HS3', 'HS4', 'HS5', 'HS6', 'HS7', 'HS8', 'HS9', 'HS10']
11 df['HS_overall'] = df[HS_CODES].mean(axis=1).round(2)
12 df['Sev_clean'] = df.apply(compute_severity, axis=1)
13
14
15 school_scores = (df.groupby(['School', 'School_short'])[HS_CODES]
16                  .mean().round(2).reset_index())
17 school_scores['HS_overall'] = school_scores[HS_CODES].mean(axis=1).round(2)
18 school_scores.to_csv('outputs/hs_scores.csv', index=False)

```

Listing 9.1: Scoring invocation in main.py (simplified).

The `sp` dictionary maps school short name to the row from `outputs/spatial_features.csv`. It is built once before the apply loop and passed in as a closure.

9.3 HS1 — Pedestrians from All Walks of Life

HS1 measures footpath *accessibility and inclusivity*: whether the path is present, wide enough, continuous, well-maintained, kerb-ramped, and unobstructed.

```

1 def compute_hs1(row) -> float:
2     pts = 0.0
3
4     # Sub-component 1: footpath presence (max 3.0)
5     p = str(row.get('Footpath_present', ''))
6     if 'both sides' in p: pts += 3.0
7     elif 'one side' in p: pts += 2.0
8     elif 'partial' in p.lower() or 'broken' in p.lower(): pts += 1.0
9
10    # Sub-component 2: footpath width (max 2.0)
11    try:
12        w = float(row['Footpath_width'])
13        if w >= 2.0: pts += 2.0
14        elif w >= 1.5: pts += 1.5
15        elif w >= 1.2: pts += 1.0
16        elif w >= 1.0: pts += 0.5
17    except (ValueError, TypeError):
18        pass
19
20    # Sub-component 3: continuity (max 2.0)
21    c = str(row.get('Continuity', ''))
22    if '100%' in c: pts += 2.0
23    elif '75 to 99' in c: pts += 1.5
24    elif '50 to 74' in c: pts += 1.0
25
26    # Sub-component 4: surface condition (max 2.0)
27    try:
28        pts += (float(row['FP_condition']) / 5.0) * 2.0
29    except (ValueError, TypeError):
30        pass
31
32    # Sub-component 5: kerb ramps (max 0.5)
33    k = str(row.get('Kerb_ramps', ''))
34    if 'all nearby' in k.lower(): pts += 0.5
35    elif 'some' in k.lower(): pts += 0.25
36
37    # Sub-component 6: obstructions (max 0.5)
38    o = str(row.get('Obstructions', ''))
39    if 'no obstruction' in o.lower(): pts += 0.5
40    elif 'minor' in o.lower(): pts += 0.25
41
42    return min(round(pts, 1), 10.0)

```

Listing 9.2: Full source: src/scoring/hs1.py.

The theoretical maximum is 10.5 points; `min(..., 10.0)` caps it. Table 9.1 summarises the sub-component weights.

Table 9.1: HS1 sub-component weights (max total 10.0).

Sub-component	Max pts	Survey column
Footpath presence	3.0	Footpath_present
Footpath width	2.0	Footpath_width
Path continuity	2.0	Continuity
Surface condition	2.0	FP_condition (rated 0–5)
Kerb ramps	0.5	Kerb_ramps
Obstruction-free	0.5	Obstructions

9.4 HS2 — Easy to Cross

HS2 is the crossing quality indicator. It is also one of the two *Major severity triggers*: if HS2 < 4.0 the location is immediately classified as Major regardless of other scores.

```

1 def compute_hs2(row) -> float:
2     pts = 0.0
3
4     # Crossing type (max 4.0)
5     ct = str(row.get('Crossing_present', ''))
6     if 'signal' in ct.lower() or 'traffic light' in ct.lower(): pts += 4.0
7     elif 'raised' in ct.lower(): pts += 3.0
8     elif 'zebra' in ct.lower() or 'marked' in ct.lower(): pts += 2.5
9     elif 'refuge' in ct.lower(): pts += 2.0
10    elif 'informal' in ct.lower() or 'unmarked' in ct.lower(): pts += 0.5
11
12    # Proximity (max 2.0)
13    try:
14        d = float(row['Crossing_dist'])
15        if d <= 30: pts += 2.0
16        elif d <= 75: pts += 1.5
17        elif d <= 150: pts += 1.0
18        elif d <= 250: pts += 0.5
19    except (ValueError, TypeError):
20        pass
21
22    # Visibility (max 2.0, rated 0-5)
23    try:
24        pts += (float(row['Visibility']) / 5.0) * 2.0
25    except (ValueError, TypeError):
26        pass
27
28    # Tactile pavers (max 1.0)
29    t = str(row.get('Tactile', ''))
30    if 'both sides' in t.lower(): pts += 1.0
31    elif 'one side' in t.lower(): pts += 0.5
32
33    # Signal / countdown timer (max 1.0)
34    s = str(row.get('Signal', ''))
35    if 'countdown' in s.lower():
36        pts += 1.0
37    elif 'yes' in s.lower() and 'no pedestrian' not in s.lower():

```

```

38     pts += 0.5
39
40     return min(round(pts, 1), 10.0)

```

Listing 9.3: Full source: src/scoring/hs2.py.

9.5 HS3 — Shade and Shelter (Spatial Indicator)

HS3 is the first indicator that draws on OSM spatial data rather than field observations. It receives a `sp` dict containing the pre-computed spatial features for the school.

```

1  import numpy as np
2  import pandas as pd
3
4  def compute_hs3(row, sp: dict) -> float:
5      pts = 0.0
6
7      # Tree count within 100m (max 5.0)
8      tree_count = sp.get('tree_count_100m', np.nan)
9      if pd.notna(tree_count):
10         if tree_count >= 15: pts += 5.0
11         elif tree_count >= 8: pts += 3.5
12         elif tree_count >= 3: pts += 2.0
13         elif tree_count >= 1: pts += 1.0
14
15     # Shelters / bus shelters within 200m (max 2.0)
16     shelter_count = sp.get('shelter_count_200m', np.nan)
17     if pd.notna(shelter_count) and shelter_count > 0:
18         pts += 2.0
19
20     # Green space percentage within 400m (max 3.0)
21     green_pct = sp.get('green_pct_400m', np.nan)
22     if pd.notna(green_pct):
23         if green_pct >= 20: pts += 3.0
24         elif green_pct >= 10: pts += 2.0
25         elif green_pct >= 5: pts += 1.0
26
27     return min(round(pts, 1), 10.0) if pts > 0 else np.nan

```

Listing 9.4: src/scoring/hs3.py — spatial indicator pattern.

Graceful Missing Data

If no spatial features are available (e.g. `spatial_features.py` has not been run), `sp` will be an empty dict and all `sp.get()` calls return `np.nan`. In that case `compute_hs3` returns `np.nan` rather than 0.0. The school-level mean in `main.py` uses `.mean()` which skips NaN values, so HS3 simply does not contribute to the overall score for that school. This is intentional: absence of data is different from a score of zero.

9.6 HS5 — Not Too Noisy (Deductive Pattern)

HS5 uses the deductive pattern: it begins at 10.0 (a perfectly quiet street) and subtracts points for traffic volume, speed, heavy vehicles, and lane count.

```

1  def compute_hs5(row) -> float:
2      pts = 10.0    # start perfect; deduct for negative conditions
3
4      # Traffic volume (max deduction 4.0)
5      v = str(row.get('Traffic_volume', '')).lower()

```

```

6     if 'very high' in v or 'major arterial' in v: pts -= 4.0
7     elif 'high' in v: pts -= 3.0
8     elif 'moderate' in v: pts -= 1.5
9
10    # Speed limit (max deduction 2.0)
11    try:
12        speed = float(
13            ''.join(c for c in str(row.get('Speed_limit', ''))
14                    if c.isdigit() or '50'))
15        if speed >= 70: pts -= 2.0
16        elif speed >= 60: pts -= 1.0
17        elif speed >= 50: pts -= 0.5
18    except (ValueError, TypeError):
19        pass
20
21    # Heavy vehicles (max deduction 2.0)
22    h = str(row.get('Heavy_vehicles', '')).lower()
23    if 'frequent' in h: pts -= 2.0
24    elif 'occasion' in h: pts -= 1.0
25
26    # Lane count (max deduction 1.0)
27    l = str(row.get('Lanes', '')).lower()
28    if '3 or more' in l: pts -= 1.0
29    elif '2 lanes' in l: pts -= 0.5
30
31    return min(max(round(pts, 1), 0.0), 10.0)

```

Listing 9.5: src/scoring/hs5.py — deductive scoring pattern.

HS5 is also a Major severity trigger: if $HS5 < 3.0$ the location is Major regardless of other indicators.

9.7 All Ten Indicators at a Glance

Table 9.2 summarises the design of all ten scoring functions.

Table 9.2: All ten HS scoring functions: design pattern and primary inputs.

ID	Indicator	Pattern	Primary inputs	Severity role
HS1	Pedestrians from all walks of life	Additive	Footpath_present, Footpath_width, Continuity, FP_condition	Major if < 4.0
HS2	Easy to cross	Additive	Crossing_present, Crossing_dist, Visibility, Tactile, Signal	Major if < 4.0
HS3	Shade and shelter	Additive	tree_count_100m, shelter_count_200m, green_pct_400m (spatial)	—
HS4	Places to stop and rest	Additive	bench_count_200m, park_count_400m (spatial)	—

(continued)

ID	Indicator	Pattern	Primary inputs	Severity role
HS5	Not too noisy	Deductive	Traffic_volume, Speed_limit, Heavy_vehicles, Lanes	Major if < 3.0
HS6	Active travel choice	Additive	Cycling_infra (CIS), pt_stops_400m, cycle_pct_400m	—
HS7	Feel safe	Deductive	Lighting, crime_rate_per_100k, crash proximity	—
HS8	Things to see and do	Additive	amenity_count_400m, park_count_400m, cafe_count_400m	—
HS9	Feel relaxed	Deductive	Traffic_volume, Parking_conflict, arterial_pct_400m	—
HS10	Clean air	Deductive	aqi_pm25 (EPA), Heavy_vehicles, Traffic_volume	—

Chapter 10

Severity Classification and Recommendations

10.1 The Severity Model

After HS scores are computed for each field observation, `compute_severity()` in `src/scoring/severity.py` classifies the location as **Major**, **Moderate**, or **Minor**.

```
1 import pandas as pd
2
3 _ALL_CODES = ['HS1', 'HS2', 'HS3', 'HS4', 'HS5', 'HS6',
4               'HS7', 'HS8', 'HS9', 'HS10']
5
6
7 def compute_severity(row) -> str:
8     hs1      = row.get('HS1')
9     hs2      = row.get('HS2')
10    hs5      = row.get('HS5')
11    overall  = row.get('HS_overall')
12
13    # --- Rule 1: Major (critical safety failure) ---
14    if pd.isna(hs2) and hs2 < 4.0: return 'Major'
15    if pd.isna(hs1) and hs1 < 4.0: return 'Major'
16    if pd.isna(hs5) and hs5 < 3.0: return 'Major'
17
18    # --- Rule 2: Moderate (broad deficiency) ---
19    all_vals = [row.get(c) for c in _ALL_CODES]
20    low = sum(1 for v in all_vals if pd.isna(v) and v < 6.0)
21    if low >= 2: return 'Moderate'
22    if pd.isna(overall) and overall < 5.0: return 'Moderate'
23
24    # --- Rule 3: Minor (everything else) ---
25    return 'Minor'
```

Listing 10.1: Full source: `src/scoring/severity.py`.

Threshold Rationale

The 6.0 “good” threshold comes directly from the Healthy Streets framework: any indicator below 6.0 is flagged as a deficiency. The Major triggers ($HS2 < 4.0$, $HS1 < 4.0$, $HS5 < 3.0$) use lower thresholds because these three indicators represent acute safety risks (impassable footpath, no crossing, high-speed/volume road). Two or more deficiencies together constitute Moderate, reflecting a compounded environment even when no single indicator is in crisis.

10.2 The Recommendation Rules: `src/recommendations/rules.py`

Seventeen rule functions are defined in `rules.py`. Each accepts a scored row and returns a recommendation dict or `None`. Table 10.1 lists all rules.

Table 10.1: All 17 recommendation rules in `src/recommendations/rules.py`.

Rule	HS	Trigger condition	Recommendation
rule_01	HS1	Footpath absent or broken	Construct 1.8m continuous footpath
rule_02	HS1	Footpath width < 1.5 m	Widen to AS 1428.1 minimum
rule_03	HS2	No formal crossing present	Install raised zebra + tactile pavers
rule_04	HS2	Nearest crossing > 150 m	Add crossing within 50 m of gate
rule_05	HS2	No tactile indicators	Install tactile pavers both sides
rule_06	HS1	No school zone signage	Establish 40 km/h school zone
rule_07	HS9	No traffic calming	Install speed humps / raised crossing
rule_08	HS6	No cycling infrastructure	Install shared path or advisory lane
rule_09	HS7	Poor street lighting	Upgrade to LED street lighting
rule_10	HS3	Vegetation blocking sightlines	Trim vegetation for crossing visibility
rule_11	HS1	Kerb ramps absent	Install DDA-compliant kerb ramps
rule_12	HS5	Speed limit $\geq$ 60 km/h near gate	Reduce speed limit within school zone
rule_13	HS5	Frequent heavy vehicles	Implement local freight routing policy
rule_14	HS2	Low crossing visibility	Mark high-visibility pavement markings
rule_15	HS6	HS6 score < 5.0	Improve PT accessibility + bike parking
rule_16	HS7	HS7 score < 5.0	CPTED audit + lighting improvement
rule_17	HS10	HS10 score < 5.0	Reduce idling zones; plant vegetation buffer

Each rule function follows the same structure:

```

1 def rule_03_crossing_absent(row) -> Optional[dict]:
2     if row.get('Crossing_present') in [
3         'No crossing at all',
4         'Yes -- informal / unmarked only'
5     ]:
6         return {
7             'hs_indicator' : 'HS2',

```

```

8         'hazard'           : 'No formal pedestrian crossing present',
9         'recommendation' : 'Install raised zebra crossing with tactile '
10                        'pavers adjacent to school gate',
11         'priority'        : 'High',
12         'cost'            : 'Medium -- $20,000 to $200,000',
13         'timeframe'       : 'Short-term -- within 1 year',
14     }
15     return None

```

Listing 10.2: Example rule (rule_03) from rules.py.

10.3 The Recommendation Engine: run_recommendations()

```

1  from src.recommendations.rules import (
2      rule_01_footpath_missing,
3      # ... 16 more rules ...
4  )
5  import pandas as pd
6
7  _RULES = [
8      rule_01_footpath_missing,
9      rule_02_footpath_narrow,
10     # ... all 17 rules ...
11 ]
12
13
14 def generate_recommendations(row) -> list:
15     return [r for fn in _RULES if (r := fn(row)) is not None]
16
17
18 def run_recommendations(df: pd.DataFrame) -> pd.DataFrame:
19     records = []
20     for _, row in df.iterrows():
21         for rec in generate_recommendations(row):
22             records.append({
23                 'School'       : row['School'],
24                 'Location'     : row.get('Street', ''),
25                 'Severity'     : row.get('Sev_clean', ''),
26                 'HS_overall'   : row.get('HS_overall', ''),
27                 'HS_indicator' : rec.get('hs_indicator', ''),
28                 'Hazard'       : rec['hazard'],
29                 'Recommendation' : rec['recommendation'],
30                 'Priority'     : rec['priority'],
31                 'Cost'         : rec['cost'],
32                 'Timeframe'    : rec['timeframe'],
33             })
34     return pd.DataFrame(records)

```

Listing 10.3: Full source: src/recommendations/engine.py.

The engine iterates over every observation row in the DataFrame, applies all 17 rules, and collects every triggered rule as a separate record. The resulting DataFrame is saved to `outputs/recommendations.csv`.

10.4 Recommendations CSV Structure

The output file has one row per triggered rule per observation location. A single 400-m catchment survey of 25 locations might generate 30–60 recommendation rows. Table 10.2 shows the column structure.

Table 10.2: outputs/recommendations.csv column structure.

Column	Content
School	Full school name
Location	Street / intersection label from field survey
Severity	Major / Moderate / Minor (from <code>compute_severity</code>)
HS_overall	Overall HS score for this location (0–10)
HS_indicator	Which indicator triggered this recommendation (e.g. HS2)
Hazard	Short description of the observed hazard
Recommendation	Specific intervention text
Priority	High / Medium / Low
Cost	Low (<\$20k) / Medium (\$20k–\$200k) / High
Timeframe	Immediate / Short-term / Long-term

Using recommendations.csv in a Report

Import the CSV into Excel or Power BI and filter by **Priority = High** and **School = Reservoir HS** to generate a council-ready action list for the highest-risk school. Sort by **Severity** (Major first) and **Timeframe** (Immediate first) to produce a ranked implementation schedule.

Chapter 11

Machine Learning: Feature Engineering and Ridge Regression

11.1 Purpose and Context

The machine-learning component of this project has a specific, limited aim: to demonstrate that Healthy Streets indicator scores can be *approximated* from freely available open data (OSM, EPA, crime statistics, crash data) *without* conducting a field survey. This serves two purposes:

1. **Scalability.** If the model is accurate enough, HS scores could be estimated for the 1,000+ remaining schools in the 300,000 Streets target before field teams are deployed, allowing prioritised triage.
2. **Validation.** If open-data features correlate well with surveyed HS scores, it confirms that the feature selection in OSM is measuring the right things.

n=3 Caution

The model is trained on three schools. This is an illustrative proof-of-concept, not a production predictor. The LOO-CV evaluation is statistically valid for $n=3$ but has very high variance. Do not use the predictions to replace field surveys; use them to rank schools for survey prioritisation.

11.2 Feature Engineering: `feature_engineering.py`

11.2.1 The 26-Feature Matrix

The feature matrix is assembled from four sources and saved to `outputs/ml_school_features.csv` with one row per school. Table 11.1 shows all 26 features.

Table 11.1: The 26-feature ML input matrix and their HS indicator targets.

Feature	Source	HS indicator target
footpath_pct_200m	OSM	HS1
footpath_pct_400m	OSM	HS1
crossings_400m	OSM	HS2
signals_400m	OSM	HS2
crossing_density_400m	OSM	HS2
tree_count_100m	OSM	HS3
shelter_count_200m	OSM	HS3
green_pct_400m	OSM	HS3
bench_count_200m	OSM	HS4
park_count_400m	OSM	HS4 / HS6
avg_speed_400m	OSM	HS5 / HS9
arterial_pct_400m	OSM	HS5 / HS9
high_speed_road_400m	OSM	HS5
road_count_400m	OSM	HS5
cycle_pct_400m	OSM	HS6
protected_cycle_length_400m	OSM	HS6
pt_stops_400m	OSM	HS6
amenity_count_400m	OSM	HS8
cafe_count_400m	OSM	HS8
walk_length_400m	OSM	context
crime_rate_per_100k	CSA Vic	HS7
aqi_pm25	EPA Vic	HS10
crash_count	VicRoads	HS7 / HS9
serious_or_fatal_rate	VicRoads	HS7
school_hours_pct	VicRoads	HS5 / HS9
avg_speed_zone	VicRoads	HS5

11.2.2 Data Joining Code

```

1  # Spatial features (from spatial_features.py output)
2  sf = pd.read_csv('outputs/spatial_features.csv')
3  sf = sf[sf['school_name'].isin(SHORT_NAMES)].rename(
4      columns={'school_name': 'school'})
5  sf_sel = sf[['school'] + SPATIAL_FEATURES]
6
7  # Environmental features
8  ef = pd.read_csv('outputs/environmental_features.csv')
9  ef['school'] = ef['school_name'].map(FULL_TO_SHORT).fillna(ef['school_name'])
10 ef_sel = ef[['school'] + ENV_FEATURES]
11
12 # Crash aggregates
13 cr = pd.read_csv('outputs/crash_data_statewide.csv', low_memory=False)
14 cr.columns = cr.columns.str.strip().str.upper()
15 cr['serious_or_fatal'] = cr['SEVERITY'].isin([1, 2]).astype(int)
16 cr['is_school_hours'] = (
17     (pd.to_datetime(cr['ACCIDENT_DATE'], dayfirst=True,
18                     errors='coerce').dt.dayofweek < 5) &
19     (cr['HOUR'].between(7, 9) | cr['HOUR'].between(14, 17))
20 ).astype(int)
21
22 crash_agg = (cr.groupby('school')
23               .agg(crash_count = ('ACCIDENT_NO', 'count'),
24                  serious_or_fatal_rate = ('serious_or_fatal', 'mean'),

```

```

25         school_hours_pct      = ('is_school_hours', 'mean'),
26         avg_speed_zone        = ('SPEED_ZONE',
27             lambda x: pd.to_numeric(x, errors='coerce')
28             .where(lambda v: v < 200).mean()))
29         .round(4).reset_index())
30
31 # HS targets
32 hs = pd.read_csv('outputs/hs_scores.csv')
33 hs['school'] = hs['School_short']
34
35 ml = (sf_sel
36     .merge(ef_sel,      on='school', how='left')
37     .merge(crash_agg,   on='school', how='left')
38     .merge(hs[['school'] + HS_TARGETS + ['HS_overall']],
39         on='school', how='left'))
40
41 ml.to_csv('outputs/ml_school_features.csv', index=False)

```

Listing 11.1: Feature matrix assembly in feature_engineering.py.

11.3 The Ridge Regression Model: ml_model.py

11.3.1 Why Ridge Regression?

Ridge regression (L2 regularisation) was chosen for three reasons specific to this dataset:

1. **n=3 schools.** With only three training examples, any model must be heavily regularised to avoid overfitting. Ridge shrinks coefficients toward zero, providing a principled bias-variance trade-off.
2. **Correlated features.** OSM spatial features are highly correlated with each other (e.g. `footpath_pct_200m` and `footpath_pct_400m`). Ridge handles multicollinearity better than OLS.
3. **Multi-output target.** scikit-learn's `MultiOutputRegressor` wraps any single-output estimator to predict all 10 HS indicators simultaneously with independent per-indicator models.

11.3.2 Pipeline Structure

```

1  from sklearn.linear_model      import Ridge
2  from sklearn.preprocessing     import StandardScaler
3  from sklearn.pipeline         import Pipeline
4  from sklearn.multioutput      import MultiOutputRegressor
5  from sklearn.model_selection  import LeaveOneOut
6  from sklearn.metrics          import mean_absolute_error
7
8  pipe = Pipeline([
9      ('scaler', StandardScaler()),           # normalise each feature to mean
10         =0, std=1
11      ('model', MultiOutputRegressor(
12         Ridge(alpha=1.0))),               # independent Ridge per HS
13         indicator
14 ])

```

Listing 11.2: The full scikit-learn pipeline in ml_model.py.

`StandardScaler` is essential: the 26 features span very different scales (e.g. `walk_length_400m` is in thousands of metres while `crossings_400m` is a small integer). Without scaling, Ridge would over-penalise large-scale features.

The `alpha=1.0` Ridge parameter was chosen by convention for small datasets; in a production system with more schools it should be tuned via cross-validation grid search.

11.3.3 Leave-One-Out Cross-Validation

LOO-CV is the only unbiased hold-out strategy for $n=3$:

```

1  loo    = LeaveOneOut()
2  preds = np.full_like(Y.values, np.nan, dtype=float)
3
4  for train_idx, test_idx in loo.split(X):
5      X_tr, X_te = X.iloc[train_idx], X.iloc[test_idx]
6      y_tr      = Y.iloc[train_idx]
7
8      fold_pipe = Pipeline([
9          ('scaler', StandardScaler()),
10         ('model', MultiOutputRegressor(Ridge(alpha=1.0))),
11     ])
12     fold_pipe.fit(X_tr, y_tr)
13     preds[test_idx] = np.clip(fold_pipe.predict(X_te), 0, 10)
14
15 mae = {c: mean_absolute_error(Y[c], preds[:, i])
16        for i, c in enumerate(HS_TARGETS)}

```

Listing 11.3: LOO-CV loop in `ml_model.py`.

In each of the three folds, the model trains on 2 schools and predicts the left-out school. The scores are clipped to $[0, 10]$ after prediction. The per-indicator MAE is printed to console and used to identify which indicators are easiest and hardest to predict from open data.

11.3.4 Full-Data Model and Serialisation

After LOO-CV evaluation, a final model is trained on all three schools and saved for use by the scenario engine:

```

1  full_pipe = Pipeline([
2      ('scaler', StandardScaler()),
3      ('model', MultiOutputRegressor(Ridge(alpha=1.0))),
4  ])
5  full_pipe.fit(X, Y)
6
7  with open('outputs/hs_predictor.pkl', 'wb') as f:
8      pickle.dump({
9          'model'      : full_pipe,
10         'features'   : X.columns.tolist(),
11         'targets'    : HS_TARGETS,
12         'schools'    : schools,
13     }, f)

```

Listing 11.4: Full-data training and pickle serialisation.

The pickle bundle includes the feature column list alongside the model. The scenario engine uses this list to ensure it passes features in the correct column order when building the scenario feature vector.

11.3.5 Three Output Charts

chart_hs_correlation.png Pearson correlation heatmap (26 features $\times$ 10 indicators). Green cells indicate the feature is positively correlated with the HS indicator; red cells indicate negative correlation. This confirms whether the feature-indicator mapping in Table 11.1 holds in practice.

chart_hs_prediction.png Three side-by-side bar charts (one per school) showing actual vs LOO-CV predicted HS scores for HS1–HS10. The mean absolute error across all indicators is printed as a subtitle per school.

chart_feature_importance.png Ten sub-plots (one per HS indicator) showing the Ridge regression coefficients ranked by magnitude. Green bars = features that increase the HS score; red bars = features that decrease it.

Run the model with:

```
1 python ml_model.py
```


Chapter 12

Scenario Analysis

12.1 What Scenario Analysis Does

The scenario engine answers the question: “*If we install a signalised crossing near Reservoir HS, what would the HS scores look like?*” It does this by:

1. Taking the *actual* surveyed HS scores as the baseline (ground truth).
2. Applying a pre-calibrated *feature delta* (change to OSM/crash features) that represents the infrastructure change.
3. Running the trained Ridge model on both the baseline and scenario feature vectors.
4. Adding the model’s *predicted delta* to the actual baseline scores.

This **delta method** is key: it uses the model only to predict the *direction and magnitude of change*, not the absolute score. The formula is:

$$\text{scenario_score}[i] = \text{actual_score}[i] + (\hat{y}_{\text{scenario}}[i] - \hat{y}_{\text{baseline}}[i])$$

The result is clamped to $[0, 10]$.

Why the Delta Method?

If the model predicts absolute scores with a MAE of, say, 1.2 points, using it to predict absolute scenario scores would carry that error forward. But the model’s *relative* predictions (how much does a crossing change the score?) are more reliable than its absolute ones, because the relative change is insensitive to any systematic bias the model has for a particular school. The delta method separates the ground-truth baseline from the model-driven change estimate.

12.2 Intervention Templates: `src/scenarios/interventions.py`

Ten infrastructure interventions are pre-defined in `interventions.py`. Each entry specifies which ML features change and by how much, along with cost and timeframe metadata. Table 12.1 lists all ten.

Table 12.1: All ten scenario interventions in `interventions.py`.

Key	Label	HS	Cost	Time
pedestrian_crossing	Install signalised crossing	HS2	Medium \$80k–\$200k	<1 yr
bike_lane	Install protected bike lane	HS6	High \$500k+ /km	1–3 yr
speed_reduction	Reduce speed to 40 km/h zone	HS5	Low \$5k–\$20k	<6 mo
traffic_calming	Speed humps / raised crossing	HS9	Medium \$50k–\$150k	<1 yr
footpath	Build / extend footpath	HS1	Medium \$100k–\$300k	<1 yr
street_trees	Plant street trees	HS3	Low \$20k–\$60k	<1 yr
benches	Add street benches	HS4	Low \$5k–\$15k	<6 mo
pt_stop	Add / improve PT stop	HS6	Medium \$50k–\$200k	1–2 yr
shelter	Install covered shelter	HS3	Low \$15k–\$40k	<6 mo
remove_arterial	Reroute heavy vehicles	HS5	High (network approval)	2–5 yr

Each intervention entry in `INTERVENTIONS` follows this structure:

```

1 INTERVENTIONS = {
2     'pedestrian_crossing': {
3         'label'      : 'Install signalised pedestrian crossing',
4         'description': 'Add a signalised crossing adjacent to the school '
5                       'gate (zebra + signals + countdown timer).',
6         'deltas': {
7             'crossings_400m'      : +1,
8             'signals_400m'       : +1,
9             'crossing_density_400m': +0.15,
10        },
11        'cost'      : 'Medium (~$80k--$200k)',
12        'timeframe': '< 1 year',
13        'hs_target': 'HS2',
14    },
15    # ... 9 more interventions ...
16 }
17 \end{lstlisting}
18
19 \section{The Scenario Engine: \texttt{run\_scenario()}}
20
21 \begin{lstlisting}[style=python, caption={Core delta-method logic in src/
22   scenarios/engine.py.}]
23 def run_scenario(school: str, intervention_keys: list) -> dict:
24     bundle = pickle.load(open('outputs/hs_predictor.pkl', 'rb'))
25     pipe, feature_cols = bundle['model'], bundle['features']
26
27     feat_row = load_features(school)      # row from ml_school_features.csv
28     actual   = load_actual_scores(school) # row from hs_scores.csv
29
30     X_baseline = pd.DataFrame(
31         [feat_row[feature_cols].values], columns=feature_cols
32     ).astype(float)
33
34     X_scenario = X_baseline.copy()

```

```

34     for key in intervention_keys:
35         for feat, delta in INTERVENTIONS[key]['deltas'].items():
36             if feat in X_scenario.columns:
37                 before = float(X_scenario[feat].iloc[0])
38                 after = float(np.clip(
39                     before + delta, 0, before * 3 + abs(delta) * 2))
40                 X_scenario[feat] = after
41
42     # Delta method
43     pred_baseline = pipe.predict(X_baseline)[0] # shape (10,)
44     pred_scenario = pipe.predict(X_scenario)[0]
45     model_delta = pred_scenario - pred_baseline
46
47     scenario_hs = {}
48     for i, code in enumerate(HS_CODES):
49         base_val = float(actual[code])
50         new_val = float(np.clip(base_val + model_delta[i], 0.0, 10.0))
51         scenario_hs[code] = round(new_val, 2)
52
53     scenario_overall = round(float(np.mean(list(scenario_hs.values()))), 2)
54
55     return {
56         'school' : school,
57         'interventions': [INTERVENTIONS[k]['label'] for k in
58             intervention_keys],
59         'baseline' : {**actual},
60         'scenario' : {**scenario_hs, 'HS_overall': scenario_overall},
61         'deltas' : {c: round(scenario_hs[c] - float(actual[c]), 2)
62             for c in HS_CODES},
63     }

```

Listing 12.1: Example intervention definition (pedestrian_crossing).

Feature Clamping in Interventions

The scenario engine clamps the after-intervention feature value to $\text{before} * 3 + \text{abs}(\text{delta}) * 2$. This prevents runaway feature values if an intervention delta is very large. For example, adding a crossing (`crossings_400m += 1`) on a school that already has 8 crossings should not produce a count of 100.

12.3 The Scenario Analyser CLI

`scenario_analyzer.py` is the command-line interface to the scenario engine. It supports four modes:

```

1  # Single scenario: one school, one or more interventions
2  python scenario_analyzer.py \
3      --school "Reservoir HS" \
4      --interventions pedestrian_crossing speed_reduction
5
6  # Rank all interventions for a school (best delta first)
7  python scenario_analyzer.py \
8      --school "Preston HS" \
9      --rank-all
10
11 # Run all interventions across all schools
12 python scenario_analyzer.py --all-schools
13
14 # List available intervention keys
15 python scenario_analyzer.py --list
16

```

```

17 # Save chart to a specific path
18 python scenario_analyzer.py \
19     --school "William Ruthven SC" \
20     --interventions footpath bike_lane \
21     --out outputs/scenario_wrusc_footpath_bike.png

```

Listing 12.2: scenario_analyzer.py CLI usage examples.

12.3.1 Output Chart

For a single-school scenario, the analyser produces a two-panel chart saved to `outputs/` (or the path specified by `-out`):

Left panel — Before/After. Horizontal bar chart showing baseline HS1–HS10 scores (dark blue) and scenario scores (teal). A dashed vertical line at 6.0 marks the “good” threshold.

Right panel — Delta bars. Horizontal bars showing the change in each indicator (Δ HS). Green bars indicate improvement; red bars indicate deterioration (possible if the intervention has negative side-effects on other indicators).

The chart title includes the school name, the interventions applied, and the change in overall severity classification (e.g. “Major $\rightarrow$ Moderate”).

12.3.2 Rank-All Mode

`-rank-all` runs every intervention key individually against the specified school and ranks them by `HS_overall` delta:

```

1 def rank_all(school: str) -> None:
2     results = []
3     for key in INTERVENTIONS:
4         result = run_scenario(school, [key])
5         results.append({
6             'key'          : key,
7             'label'        : INTERVENTIONS[key]['label'],
8             'delta_overall': result['deltas']['HS_overall'],
9             'cost'         : INTERVENTIONS[key]['cost'],
10            'timeframe'     : INTERVENTIONS[key]['timeframe'],
11        })
12     results.sort(key=lambda r: r['delta_overall'], reverse=True)
13     for r in results:
14         print(f" {r['label']:<45} delta={r['delta_overall']:+.2f}"
15               f" {r['cost']} {r['timeframe']}")

```

Listing 12.3: rank_all() in scenario_analyzer.py.

This gives practitioners a rapid cost-effectiveness ranking: which single intervention gives the largest HS improvement per dollar at a given school?

12.4 End-to-End Scenario Workflow

To run a full scenario analysis from scratch:

```

1 # 1. Run the full main pipeline to generate hs_scores.csv
2 python main.py
3
4 # 2. Compute spatial features (OSM queries, ~3-5 minutes)

```

```
5 python spatial_features.py
6
7 # 3. Build the ML feature matrix
8 python feature_engineering.py
9
10 # 4. Train the Ridge model and save hs_predictor.pkl
11 python ml_model.py
12
13 # 5. Run a scenario
14 python scenario_analyzer.py \
15     --school "Reservoir HS" \
16     --interventions pedestrian_crossing footpath
```

Listing 12.4: Complete pipeline to enable scenario analysis.

Steps 1–4 need to be run only once (or when field survey data is updated). Step 5 can be run repeatedly with different intervention combinations without re-training.

Part II

Analysis, Equity, and Dashboard Deployment

Part 4 closes the pipeline loop. Chapter 13 dives into the VicRoads crash record — year-on-year trends, school-hours concentration, and time-of-day peaks. Chapter 14 joins ABS SEIFA disadvantage indices to the HS scores, revealing the equity gap between high-disadvantage and lower-disadvantage school catchments. Chapter 15 documents the end-to-end dashboard pipeline: how `prepare_data.py` assembles a single `data.json` from every upstream output and serves it to a GitHub Pages web application built on Leaflet, Chart.js, and Tailwind CSS.

Chapter 13

Crash Trend Analysis

13.1 Why Crash Data Matters

The Healthy Streets framework captures *perceived* risk and *infrastructure quality*, but crash records capture *revealed* risk — actual harm that has already occurred. `crash_trend_analysis.py` answers four questions:

1. How has the volume of pedestrian and cyclist crashes across Darebin LGA changed from 2021 to 2025?
2. Which of our three schools has the worst crash record within a 400 m gate buffer?
3. What fraction of crashes occur during school drop-off and pick-up hours?
4. At what time of day are crashes most concentrated?

Output is a single four-panel PNG saved to `outputs/chart_crash_trends.png`.

Data source

VicRoads open data — statewide ped/cyc crash records, 2021–2025. Downloaded by `crash_analysis.py` (pipeline Step 1) and saved as `outputs/crash_data_statewide.csv`. Each row is one crash event with columns including `ACCIDENT_DATE`, `ACCIDENT_TIME`, `DAY_OF_WEEK`, `SEVERITY`, `LGA_NAME`, `LATITUDE`, `LONGITUDE`, and `nearest_school` (added during the 400 m spatial join).

13.2 Loading and Filtering

```
1 def load_data():
2     path = os.path.join(OUT_DIR, 'crash_data_statewide.csv')
3     df = pd.read_csv(path, low_memory=False)
4     df['year'] = pd.to_datetime(df['ACCIDENT_DATE'],
5                               errors='coerce').dt.year
6     df['hour'] = pd.to_datetime(df['ACCIDENT_TIME'],
7                               format='%H:%M:%S',
8                               errors='coerce').dt.hour
9     df['sev_label'] = df['SEVERITY'].apply(_severity_label)
10    df['school_hours'] = df.apply(_is_school_hours, axis=1)
11
12    # Darebin LGA subset
13    darebin = df[df['LGA_NAME'].str.upper() == 'DAREBIN'].copy()
14
15    # Our 3 schools subset (normalise name variants)
16    our = df[df['nearest_school'].isin(SCHOOL_NAME_MAP.keys())].copy()
17    our['school_short'] = our['nearest_school'].map(SCHOOL_NAME_MAP)
```



```

18
19     return df, darebin, our

```

Listing 13.1: `load_data()` in `crash_trend_analysis.py`

Three DataFrames are returned:

df Full statewide dataset — used only for LGA-level extraction.

darebin All Darebin LGA crashes. Used for Panel 1 (LGA trends) and Panel 4 (time-of-day).

our Crashes within 400m of the three school gates. Used for Panels 2 and 3.

13.2.1 Severity Labelling

VicRoads encodes severity as an integer: 1 = Fatal, 2 = Serious, anything else = Minor.

```

1 def _severity_label(code):
2     if code == 1: return 'Fatal'
3     elif code == 2: return 'Serious'
4     else: return 'Minor'

```

Listing 13.2: Severity mapping function.

13.2.2 School Hours Filter

The school-hours flag is a row-level predicate that returns **True** when a crash falls on a weekday during morning drop-off (07:30–09:00) or afternoon pick-up (14:30–16:30). `DAY_OF_WEEK` is coded 1=Sunday ... 7=Saturday in the VicRoads dataset, so weekdays are values 2–6.

```

1 def _is_school_hours(row):
2     try:
3         t = pd.to_datetime(row['ACCIDENT_TIME'], format='%H:%M:%S')
4         h, m = t.hour, t.minute
5         wday = int(row['DAY_OF_WEEK']) # 1=Sun, 2=Mon...6=Fri, 7=Sat
6         if wday in (1, 7):
7             return False
8         morning = (h == 7 and m >= 30) or h == 8
9         afternoon = ((h == 14 and m >= 30)
10                     or h in (15, 16)
11                     or (h == 16 and m == 0))
12         return morning or afternoon
13     except Exception:
14         return False

```

Listing 13.3: School-hours predicate.

DAY_OF_WEEK encoding

VicRoads uses 1-based Sunday-first encoding, so Monday = 2 and Friday = 6. The filter explicitly excludes `wday in (1, 7)` to drop Saturday and Sunday, not 0 and 6. This is a common off-by-one trap when working with this dataset.

13.3 Four-Panel Chart

13.3.1 Panel 1 — Darebin LGA Crashes by Year

A stacked bar chart showing Fatal/Serious/Minor crashes across all Darebin LGA schools, grouped by year (2021–2025). Each bar is annotated with the stratum count (white text)

and the total (black text above the bar).

13.3.2 Panel 2 — Our Three Schools by Year

Grouped bars: three schools side-by-side for each year. School colours are fixed: Reservoir HS #1A476E, William Ruthven SC #1A8FC1, Preston HS #C0392B.

13.3.3 Panel 3 — School Hours vs Off-Peak

For each school: two bars, school-hours crashes vs off-peak crashes. This panel is the most policy-relevant — a school with a high school-hours fraction has infrastructure risk specifically during the vulnerable window.

13.3.4 Panel 4 — Time-of-Day Distribution

Histogram over hours 0–23 for all Darebin LGA crashes. Bars are coloured by period:

- **Red** — school hours (07:00–09:00 and 14:00–16:00)
- **Orange** — general peak hours (06:00–09:00 and 16:00–19:00)
- **Grey** — off-peak

```

1 bar_colours = []
2 for h in range(24):
3     if (7 <= h <= 8) or (14 <= h <= 16):
4         bar_colours.append('#C0392B') # school hours
5     elif 6 <= h <= 9 or 16 <= h <= 19:
6         bar_colours.append('#D35400') # peak hours
7     else:
8         bar_colours.append('#BDC3C7') # off-peak

```

Listing 13.4: Colour assignment for time-of-day histogram.

13.4 Key Findings Function

`print_findings()` summarises the chart to stdout after saving the PNG. It reports the per-year Darebin total, per-school breakdown (total and school-hours count), the trend direction (first year vs last year), and the peak crash hour across Darebin.

```

1 python crash_trend_analysis.py
2 # Outputs:
3 # outputs/chart_crash_trends.png
4 # Console: crash count table, trend direction, peak hour

```

Listing 13.5: Running the crash trend analysis.

School-hours concentration — the key metric

Divide school-hours crashes by total crashes for each school. A ratio above 0.30 suggests that the school gate environment specifically (not just the general neighbourhood) is generating disproportionate risk during the most vulnerable pedestrian window.

13.5 Integration with run_all.py

This script is Step 9 in the master pipeline runner:

```
1 {  
2   'n':      9,  
3   'script': 'crash_trend_analysis.py',  
4   'label':  'Crash trend analysis (2021-2025)',  
5   'checks': ['outputs/chart_crash_trends.png'],  
6   'note':   'Year trends, school-hours breakdown, time-of-day distribution.',  
7 }
```

Listing 13.6: Step 9 in run_all.py STEPS list.

If `outputs/chart_crash_trends.png` already exists and `-force` is not passed, the step is skipped automatically.

Chapter 14

Equity Analysis

14.1 The Equity Hypothesis

The 300,000 Streets project is not just a safety study — it is an equity study. The central hypothesis is that *more socioeconomically disadvantaged school catchments have worse pedestrian infrastructure*, compounding the disadvantage for students who are more likely to walk or cycle to school and less likely to be driven.

`equity_analysis.py` tests this hypothesis by joining:

- **ABS SEIFA 2021 IRSD** (Index of Relative Socio-economic Disadvantage) at the SA1 level, aggregated to school catchment; and
- **Healthy Streets overall scores** from `outputs/hs_scores.csv`.

Output: `outputs/chart_equity_seifa.png` (three-panel chart).

14.2 SEIFA Data Pipeline

SEIFA data for Darebin is produced by `seifa_analysis.py` (pipeline Step 7), which reads the ABS-released IRSD scores for each SA1 in Darebin, performs a catchment-weighted average based on which SA1s overlap each school's 800m buffer, and writes `outputs/seifa_darebin.csv` with columns:

Column	Meaning
<code>School</code>	Full school name (join key to <code>hs_scores.csv</code>)
<code>School_short</code>	Short name (Reservoir HS etc.)
<code>IRSD_Score_Weighted</code>	Population-weighted average IRSD score (national mean ≈ 1000)
<code>IRSD_Decile</code>	Decile 1–10 (1 = most disadvantaged nationally)
<code>Disadvantage_Level</code>	Human-readable label (High / Moderate / Low)

SEIFA decile interpretation

Decile 1 means the catchment falls in the bottom 10% nationally by socioeconomic advantage. Decile 10 is the most advantaged. All three Darebin schools sit in Deciles 4–7, which is typical for inner-northern Melbourne — below the national median but not the most disadvantaged nationally. The school order is: **Reservoir HS** (D4) → **William Ruthven SC** (D6) → **Preston HS** (D7).

14.3 Data Join

```

1 def load_and_join():
2     seifa = pd.read_csv(os.path.join(OUT_DIR, 'seifa_darebin.csv'))
3     hs     = pd.read_csv(os.path.join(OUT_DIR, 'hs_scores.csv'))
4     df     = seifa.merge(hs, on='School', how='inner')
5     df     = df.sort_values('IRSD_Decile').reset_index(drop=True)
6     return df

```

Listing 14.1: `load_and_join()` in `equity_analysis.py`

The inner join on `School` (full name) means both files must use the same school name strings. Sorting by `IRSD_Decile` ascending ensures the charts display most-disadvantaged first — a deliberate visual choice to make the equity gradient immediately readable.

14.4 Three-Panel Chart

14.4.1 Panel 1 — Side-by-Side Bars

For each school: two bars side-by-side — IRSD Decile (grey) and HS Overall score (school colour). A horizontal dashed red line marks `HS = 6.0` (the Healthy Streets “good” threshold). A SEIFA decile and an HS score on the same y-axis (0–10) are comparable because both are unitless indices, but the reader must be reminded that they measure different things.

14.4.2 Panel 2 — Scatter Plot with Quadrant Shading

Each school is a labelled scatter point. The x-axis is IRSD Decile, y-axis is HS Overall. A shaded lower-left quadrant marks “high disadvantage + poor safety” — the highest policy-priority zone.

```

1 ax2.axhspan(0, 6, xmin=0, xmax=0.5, alpha=0.06, color='#C0392B',
2             label='High risk zone')
3 ax2.text(1.2, 3.0, 'Most at risk:\nhigh disadvantage\n+ poor safety',
4          fontsize=7.5, color='#C0392B', style='italic', alpha=0.8)

```

Listing 14.2: Quadrant shading in Panel 2.

The `axhspan` spans the full horizontal extent from 0 to 50% of the x-axis (i.e. deciles 1–5) and the vertical extent 0–6 (below the HS threshold).

14.4.3 Panel 3 — Indicator Heatmap

A full-width heatmap (schools $\times$ 10 indicators) using the `RdYlGn` colourmap (red = low, green = high, white = 5). Cell values are printed inside each cell; text colour switches from white to dark depending on whether the cell is very red (< 3.5) or very green (> 8.5).

SEIFA annotations appear to the left of the heatmap, one line per school, showing both the raw weighted IRSD score and the decile:

```

1 for i, (_, row) in enumerate(df.iterrows()):
2     ax3.annotate(
3         f'IRSD {row["IRSD_Score_Weighted"]:.0f} D{row["IRSD_Decile"]:.0f}',
4         xy=(0, i), xycoords=('axes fraction', 'data'),
5         xytext=(-5, 0), textcoords='offset points',
6         ha='right', va='center', fontsize=8.5, color='#444444',
7     )

```

Listing 14.3: SEIFA annotation on the left of the heatmap.

14.5 Pearson Correlation and Policy Interpretation

`print_findings()` computes a Pearson r between IRSD Decile and HS Overall:

```
1 corr = df['IRSD_Decile'].corr(df['HS_overall'])
2 print(f" Correlation (IRSD Decile x HS Overall): r = {corr:.2f}")
3 if corr > 0.3:
4     print(" FINDING: More disadvantaged catchments have WORSE safety scores.
5         ")
6     print(" Higher decile (less disadvantaged) = higher HS score.")
```

Listing 14.4: Correlation and interpretation.

With only three schools the Pearson r cannot be interpreted with standard significance thresholds, but the observed $r \approx 0.84$ is directionally very strong and consistent with the a priori hypothesis. The heatmap reinforces this: Reservoir HS (D4) scores lowest on indicators like HS3 (shade/shelter), HS5 (traffic noise), and HS6 (active travel), while Preston HS (D7) — the least disadvantaged of the three — scores highest overall.

Three-school limitation

Pearson r with $n = 3$ has a critical value of 0.997 at $p < 0.05$ (two-tailed). The observed $r = 0.84$ is not statistically significant at conventional thresholds. It is presented as a *directional finding* to motivate expansion of the study to 20+ schools in future iterations where genuine inference is possible.

14.6 Running the Equity Analysis

```
1 # Requires seifa_darebin.csv and hs_scores.csv to exist first
2 python equity_analysis.py
3 # Output: outputs/chart_equity_seifa.png
```

Listing 14.5: Equity analysis run command.

In the master pipeline this is Step 8 (`run_all.py`), run automatically after `seifa_analysis.py` (Step 7).

Chapter 15

Dashboard and Deployment Pipeline

15.1 Architecture Overview

The final stage of the pipeline transforms every CSV, PNG, and model artefact into a single-page web application hosted on GitHub Pages. The architecture is intentionally dependency-free on the client side: no Node.js build step, no bundler, no server.

Dashboard technology stack

- **Leaflet 1.9.4** — interactive map (school markers, crash heatmap, walk/cycling networks)
- **Chart.js** — radar charts (HS indicator profiles), bar charts (scenario explorer)
- **Tailwind CSS (CDN)** — utility-first responsive layout
- **Vanilla JS (ES2020)** — no framework; all state in a single `appData` object
- **GitHub Pages** — static hosting from `docs/` directory on `main` branch

The data bridge between the Python pipeline and the browser is a single file: `docs/data/data.json`. This file is built by `prepare_data.py` and committed to the repository. The browser fetches it once on page load with a single `fetch('./data/data.json')` call.

15.2 `prepare_data.py` — The Data Builder

`prepare_data.py` assembles every piece of data the dashboard needs into `data.json`. It must be run *after* the full pipeline (`run_all.py`) has completed.

15.2.1 Required Inputs

```
1 REQUIRED_FILES = {
2     'hs_scores':      OUTPUTS / 'hs_scores.csv',
3     'recommendations': OUTPUTS / 'recommendations.csv',
4     'ml_predictions': OUTPUTS / 'ml_predictions.csv',
5     'seifa':          OUTPUTS / 'seifa_darebin.csv',
6     'config':         ROOT / 'config.py',
7 }
```

Listing 15.1: Required files checked at startup.

If any required file is missing, `prepare_data.py` exits with a descriptive error message before attempting any computation.

15.2.2 `build_schools()`

`build_schools()` is the core assembler. For each school it:

1. Looks up the school gate coordinates from `config.SCHOOL_GATES`.
2. Matches the HS scores row from `hs_scores.csv`.
3. Joins the SEIFA decile from `seifa_darebin.csv`.
4. Filters and formats the recommendations from `recommendations.csv`.
5. Calls `compute_severity()` (same logic as `src/scoring/severity.py`).
6. Returns a dict with all the data the dashboard needs for that school.

The returned list is the `schools` key in `data.json`, and it powers the school tabs, radar charts, recommendation panels, and scenario explorer simultaneously.

15.2.3 `build_scenarios()`

`build_scenarios()` pre-computes all 10 interventions $\times$ 3 schools = 30 scenario results at build time. This avoids requiring the scenario engine to run in the browser (which would need Python). The engine is imported directly:

```
1 from src.scenarios.engine import run_scenario
2 from src.scenarios.interventions import INTERVENTIONS
3
4 def build_scenarios(schools):
5     results = {}
6     for school in schools:
7         name = school['short_name']
8         results[name] = {}
9         for key in INTERVENTIONS:
10             try:
11                 r = run_scenario(name, [key])
12                 results[name][key] = {
13                     'label' : r['interventions'][0],
14                     'deltas' : r['deltas'],
15                     'cost' : INTERVENTIONS[key]['cost'],
16                     'timeframe': INTERVENTIONS[key]['timeframe'],
17                 }
18             except Exception as e:
19                 results[name][key] = {'error': str(e)}
20     return results
```

Listing 15.2: Pre-computing scenarios during data build.

The pre-computed deltas are serialised into `data.json` as nested dicts. The dashboard JavaScript reads `data.scenarios[schoolName][interventionKey].deltas` and renders the result chart instantly with no network round-trip.

15.2.4 Map Layer Builders

Three functions build the geospatial data layers embedded in `data.json`:

Function	Output
<code>build_crash_geojson()</code>	GeoJSON FeatureCollection of crash points from <code>outputs/crash_data_darebin.csv</code> , each with a <code>severity</code> property
<code>build_heatmap_points()</code>	List of <code>[lat, lon, intensity]</code> triples for the Leaflet heatmap layer. Intensity is $(4 - \text{SEVERITY})/3.0$, mapping Fatal $\rightarrow 1.0$, Serious $\rightarrow 0.67$, Minor $\rightarrow 0.33$
<code>build_network_geojson()</code>	Reads <code>outputs/networks.gpkg</code> layers <code>walk_400m</code> and <code>cycling_400m</code> via GeoPandas, converts to EPSG:4326, and rounds coordinates to 5 decimal places to reduce JSON size

GeoJSON coordinate precision

Five decimal places (≈ 1 m precision at the equator) is sufficient for street-level display and reduces the `data.json` file size significantly compared to the 15-decimal-place default from GeoPandas. The rounding is applied with a recursive helper that handles both `Point` and `LineString` coordinate arrays.

15.2.5 Chart Copying

`copy_charts()` copies all pipeline chart PNGs from `outputs/` to `docs/data/charts/`. The dashboard's Analysis section displays these charts as `<img>` tags referencing relative paths.

```

1 CHART_FILES = [
2     'chart1_safety_scores.png',
3     'chart2_hazard_severity.png',
4     'chart3_score_breakdown.png',
5     'chart_hs_correlation.png',
6     'chart_hs_prediction.png',
7     'chart_feature_importance.png',
8     'chart_equity_seifa.png',
9     'chart_crash_trends.png',
10    'chart4_demographics.png',
11 ]

```

Listing 15.3: Chart files copied to `docs/data/charts/`.

15.2.6 Final data.json Structure

```

1 {
2   "generated_at": "2025-06-01T10:23:45Z",
3   "stats": { ... hero banner stats ... },
4   "schools": [ ... 3 school objects ... ],
5   "ml_results": { ... L00-CV predictions ... },
6   "scenarios": { school: { intervention: deltas } },
7   "crash_geojson": { type: "FeatureCollection", features: [...] },
8   "heatmap_points": [ [lat, lon, intensity], ... ],
9   "networks": { "walk": {...GeoJSON...}, "cycling": {...} },
10  "spatial_stats": { school: { feature: value } },
11  "charts": { key: "filename.png", key_caption: "..." }
12 }

```

Listing 15.4: Top-level keys in `data.json`

```

1 # After run_all.py has completed:

```

```

2 python prepare_data.py
3
4 # Serves the dashboard locally:
5 cd docs && python -m http.server 8080
6 # Then open http://localhost:8080 in a browser

```

Listing 15.5: Running prepare_data.py.

15.3 Dashboard Sections

15.3.1 Hero Banner

Five stat cards rendered by `initHero()` from `data.stats`:

Stat	Source
Schools assessed	<code>len(schools)</code> (always 3)
Major hazards	Count of schools with severity = Major
Darebin crashes	Row count in <code>crash_data_darebin.csv</code>
Equity–safety correlation	Hard-coded <code>r = 0.84</code> from equity analysis
Peak crash hour	Most frequent hour in Darebin LGA crash records

15.3.2 Interactive Map

The Leaflet map is initialised by `initMap()` and contains four toggleable layers:

1. **School gate markers** — custom coloured circle markers at the GPS coordinates from `config.SCHOOL_GATES`, with popups showing HS Overall, severity badge, and per-indicator OSM stats from `spatial_stats`.
2. **Crash heatmap** — Leaflet.heat plugin rendering the intensity-weighted `heatmap_points` list. Crash concentration near school gates is immediately visible.
3. **Walking network** — GeoJSON LineString layer from `networks.walk`, styled in blue (`#1A476E`), showing the OSM pedestrian network within 400 m of each gate.
4. **Cycling network** — GeoJSON LineString layer from `networks.cycling`, styled in green (`#2E7D32`), showing cycle routes.

A layer control (top-right corner) lets users toggle each layer independently.

15.3.3 School Tabs and Radar Charts

`initSchools()` renders one tab per school. Clicking a tab reveals:

- A Chart.js radar chart of all 10 HS indicator scores.
- A severity badge and overall score.
- A SEIFA context strip (IRSD score and decile).
- The top three recommendations for that school.

Radar charts are cached in `radarCharts{}` so that switching tabs does not re-render existing Chart.js instances (which would cause a canvas memory leak).

15.3.4 Scenario Explorer

`initScenario()` builds the interactive what-if panel. The user selects a school tab and an intervention button. The dashboard looks up `data.scenarios[school][intervention]` and renders a Chart.js bar chart comparing baseline vs scenario HS scores for all 10 indicators.

Each intervention button shows the cost band and timeframe. The result panel includes a delta method disclaimer noting that results should be treated as directional with $n = 3$.

15.3.5 Analysis Section

A tabbed grid of all chart PNGs from `docs/data/charts/`. The charts are grouped into four tabs: Safety Scores, Machine Learning, Equity, and Crash Trends. Each chart is displayed with its caption from `data.charts`.

15.3.6 Recommendations Panel

`initRecommendations()` flattens all recommendations from all schools into a single list and renders a filterable table. Filters include school, severity, priority, and HS indicator. Rows are sorted by Priority (High first) then HS Overall ascending, ensuring the most urgent interventions for the most at-risk schools appear first.

15.4 run_all.py — The Master Pipeline Runner

`run_all.py` is the single command a developer runs to execute the full 10-step pipeline from crash data download through to dashboard data build.

15.4.1 Skip-if-Exists Logic

Each step defines a `checks` list of expected output file paths. If all check paths exist and `-force` is not passed, the step is skipped. This makes `run_all.py` safe to re-run at any time without re-downloading data or re-training the model unnecessarily.

```

1 def _all_exist(paths):
2     return all(os.path.exists(p) for p in paths)
3
4 for step in STEPS:
5     if not FORCE and _all_exist(step['checks']):
6         print(f" [{n}/10] {label} already done")
7         continue
8     # ... run the step

```

Listing 15.6: Skip-if-exists check in `run_all.py`

15.4.2 Pipeline Steps Summary

#	Script	Output	Runtime
1	<code>crash_analysis.py</code>	<code>crash_data_statewide.csv</code>	~30s
2	<code>spatial_features.py</code>	<code>spatial_features.csv</code>	3–5 min
3	<code>environmental_features.py</code>	<code>environmental_features.csv</code>	~10s
4	<code>main.py</code>	<code>hs_scores.csv</code> , 3 charts	~20s
5	<code>feature_engineering.py</code>	<code>ml_school_features.csv</code>	~5s
6	<code>ml_model.py</code>	<code>hs_predictor.pkl</code> , 3 charts	~10s
7	<code>seifa_analysis.py</code>	<code>seifa_darebin.csv</code>	~5s

#	Script	Output	Runtime
8	equity_analysis.py	chart_equity_seifa.png	~5s
9	crash_trend_analysis.py	chart_crash_trends.png	~5s
10	demographics_chart.py	chart4_demographics.png	~5s

15.4.3 Flags

```

1 # Run all steps, skip any whose outputs already exist
2 python run_all.py
3
4 # Force re-run every step from scratch
5 python run_all.py --force
6
7 # Start from step 5 onwards (e.g. after updating field data)
8 python run_all.py --from 5

```

Listing 15.7: run_all.py usage.

15.5 GitHub Pages Deployment

The dashboard is deployed automatically by GitHub Pages from the `docs/` directory on the `main` branch. No CI/CD configuration is required beyond the repository setting:

```

1 Source:  Deploy from a branch
2 Branch:  main
3 Folder:  /docs

```

Listing 15.8: GitHub Pages setting (repository Settings > Pages).

The complete deployment workflow after a field data update is:

```

1 # 1. Update field survey data
2 #   (edit school_data.csv or re-export from Google Forms)
3
4 # 2. Re-run the pipeline (skips unchanged steps automatically)
5 python run_all.py --from 4
6
7 # 3. Rebuild dashboard data
8 python prepare_data.py
9
10 # 4. Commit and push
11 git add docs/data/data.json docs/data/charts/
12 git commit -m "chore: refresh dashboard data"
13 git push origin main
14 # GitHub Pages auto-redeploys within ~60 seconds

```

Listing 15.9: End-to-end update workflow.

Local development server

Browsers block `fetch()` calls to local `file://` URIs due to CORS policy. Always serve the dashboard through a local HTTP server during development:

```
cd docs && python -m http.server 8080
```

Then open `http://localhost:8080` in a browser. Opening `docs/index.html` directly

as a file will show the loading spinner indefinitely.

Part III

Data Ingestion, Environmental Features, and Core Pipeline

Part 5 covers the three data acquisition scripts that feed the scoring engine and the master orchestration script that drives it all. Chapter 16 documents `crash_analysis.py` — how Victorian road crash records are downloaded from the open data portal, joined across three tables, filtered for pedestrian and cyclist involvement, and assigned to school gates via vectorised Haversine distance. Chapter 17 explains how `environmental_features.py` fetches live air quality data from EPA Victoria AirWatch and personal crime rates from the Crime Statistics Agency, with two-tier fallback strategies so the pipeline never blocks on external APIs. Chapter 18 documents `main.py` — the six-step orchestrator that loads all upstream data, applies all ten HS scoring functions, generates three publication-quality charts, runs the recommendation engine, and writes the KDE heatmap and Leaflet map.

Chapter 16

Crash Data Download and Spatial Join

16.1 Overview

`crash_analysis.py` (pipeline Step 1) is the first script to run and the only one that requires an internet connection. It downloads three raw CSV tables from the VicRoads open data portal, joins them, filters for pedestrian and cyclist incidents in the last five years, and assigns each crash to its nearest school gate using a vectorised Haversine computation.

Two output files

- `outputs/crash_data_statewide.csv` — all Victorian ped/cyc crashes (last 5 years) with a `nearest_school` column assigning each crash to the closest school gate in the state.
- `outputs/crash_data_darebin.csv` — backward-compatible subset: only crashes within 400 m of the original three Darebin gates.

16.2 VicRoads Open Data Schema

VicRoads publishes road crash data as three relational tables under a single CKAN dataset. All three are downloaded programmatically at runtime; no manual export is needed.

Table	Key columns	Role
<code>accident.csv</code>	<code>ACCIDENT_NO</code> , <code>ACCIDENT_DATE</code> , <code>SEVERITY</code> , <code>SPEED_ZONE</code>	One row per crash event
<code>node.csv</code>	<code>ACCIDENT_NO</code> , <code>LGA_NAME</code> , <code>LATITUDE</code> , <code>LONGITUDE</code>	GPS coordinates and LGA name
<code>person.csv</code>	<code>ACCIDENT_NO</code> , <code>ROAD_USER_TYPE_DESC</code>	One row per person; used to identify pedestrian/cyclist crashes

The pedestrian/cyclist filter is applied to `person.csv` rather than `accident.csv` because the accident-level table has no road-user-type field — it records the crash outcome (severity, speed zone) but not who was involved.

16.3 Downloading the Three Tables

```
1 URLs = {
```



```

2  'accident': 'https://opendata.transport.vic.gov.au/.../accident.csv',
3  'person':   'https://opendata.transport.vic.gov.au/.../person.csv',
4  'node':     'https://opendata.transport.vic.gov.au/.../node.csv',
5  }
6
7  def fetch_csv(name, url):
8      r = requests.get(url, timeout=180)
9      r.raise_for_status()
10     df = pd.read_csv(StringIO(r.text), low_memory=False)
11     df.columns = df.columns.str.strip().str.upper()
12     return df
13
14 accident = fetch_csv('accident', URLs['accident'])
15 node     = fetch_csv('node',      URLs['node'])
16 person   = fetch_csv('person',   URLs['person'])

```

Listing 16.1: fetch_csv() and the three VicRoads URLs.

The 180-second timeout is set deliberately for `accident.csv` which can be 50–80 MB. All column names are uppercased and stripped immediately after loading to normalise across VicRoads data releases (which occasionally change capitalisation).

16.4 Table Join and Date Filter

Node coordinates are joined to accidents on `ACCIDENT_NO`. `drop_duplicates('ACCIDENT_NO')` ensures that intersections with multiple node records do not inflate the row count.

```

1  node_slim = node[['ACCIDENT_NO', 'LGA_NAME', 'LATITUDE',
2                  'LONGITUDE', 'POSTCODE_CRASH']
3                  ].drop_duplicates('ACCIDENT_NO')
4
5  merged = accident.merge(node_slim, on='ACCIDENT_NO', how='left')
6
7  # 5-year filter
8  YEARS_BACK = 5
9  cutoff = datetime.now() - timedelta(days=365 * YEARS_BACK)
10 merged['ACCIDENT_DATE'] = pd.to_datetime(
11     merged['ACCIDENT_DATE'], dayfirst=True, errors='coerce')
12 merged = merged[merged['ACCIDENT_DATE'] >= cutoff].copy()

```

Listing 16.2: Join node to accident, filter last 5 years.

dayfirst=True

VicRoads date strings use DD/MM/YYYY format. Without `dayfirst=True`, pandas interprets January dates as day 1 of month 1 correctly, but days 13–31 would raise a parsing error. Always set this flag when reading Victorian government date columns.

16.5 Pedestrian and Cyclist Filter

The ped/cyc filter is applied to `person.csv` using a string search on `ROAD_USER_TYPE_DESC`.

```

1  ped_cyc_acc = person[
2      person['ROAD_USER_TYPE_DESC'].astype(str).str.upper()
3      .str.contains('PEDESTRIAN|CYCLIST|BICYCL', na=False)
4  ]['ACCIDENT_NO'].unique()
5
6  merged['PED_OR_CYC'] = merged['ACCIDENT_NO'].isin(ped_cyc_acc)
7  statewide = merged[merged['PED_OR_CYC']].dropna()

```

```
8 subset=['LATITUDE', 'LONGITUDE']).copy()
```

Listing 16.3: Extracting ped/cyc accident IDs from person.csv.

The three patterns (PEDESTRIAN, CYCLIST, BICYCL) are needed because VicRoads uses both “Cyclist” and “Bicyclist” across data releases.

16.6 School Gate Lookup

Before assigning crashes to schools, `fetch_school_gates()` attempts to download the full list of Victorian government secondary school coordinates from the Department of Education and Training (DET) via the data.vic.gov.au CKAN API.

```
1 def fetch_school_gates():
2     fallback = dict(SCHOOL_GATES) # 3 Darebin gates always present
3     try:
4         pkg = requests.get(
5             'https://discover.data.vic.gov.au/api/3/action/package_show',
6             params={'id': 'school-locations-time-series'},
7             timeout=30,
8         )
9         pkg.raise_for_status()
10        resources = pkg.json()['result']['resources']
11        # sort by name descending to get the most recent year's CSV
12        csv_resources = sorted(
13            [r for r in resources if r.get('format','').upper() == 'CSV'],
14            key=lambda r: r.get('name',''), reverse=True
15        )
16        url = csv_resources[0]['url']
17        r = requests.get(url, timeout=60)
18        r.raise_for_status()
19        schools = pd.read_csv(StringIO(r.text), low_memory=False)
20        # ... filter GOVERNMENT, SECONDARY/COMBINED, STATUS=OPEN ...
21        gates.update(fallback) # Darebin gates always override
22        return gates
23    except Exception as e:
24        print(f" Warning: DET download failed ({e})")
25        return fallback
```

Listing 16.4: DET school download with fallback.

The DET dataset lists all open Victorian government secondary schools with their GPS coordinates. Adding all state schools makes the statewide CSV useful for future scaling — a researcher studying any Victorian school can filter the statewide CSV by `nearest_school` without re-downloading crash data.

16.7 Vectorised Haversine Distance

Assigning each crash to its nearest school gate is done with a vectorised Haversine implementation rather than a row-wise loop. With 7,000+ crashes and 1,500+ school gates, a naive nested loop would take hours; the vectorised approach completes in under a minute.

```
1 def haversine_m(lat1, lon1, lat2, lon2):
2     R = 6_371_000
3     phi1, phi2 = np.radians(lat1), np.radians(lat2)
4     dphi = np.radians(lat2 - lat1)
5     dlam = np.radians(lon2 - lon1)
6     a = (np.sin(dphi / 2)**2
7         + np.cos(phi1) * np.cos(phi2) * np.sin(dlam / 2)**2)
```

```
8 return R * 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))
```

Listing 16.5: Haversine distance (radians, returns metres).

```
1 crash_lats = statewide['LATITUDE'].to_numpy()
2 crash_lons = statewide['LONGITUDE'].to_numpy()
3
4 best_dist = np.full(len(statewide), np.inf)
5 best_school = np.full(len(statewide), '', dtype=object)
6
7 for school, gate in all_gates.items():
8     d = haversine_m(crash_lats, crash_lons, gate['lat'], gate['lon'])
9     closer = d < best_dist
10    best_dist[closer] = d[closer]
11    best_school[closer] = school
12
13 statewide['nearest_school'] = best_school
14 statewide['dist_to_gate_m'] = np.round(best_dist, 1)
```

Listing 16.6: Vectorised nearest-school assignment.

The outer loop is over schools (1,500 iterations), and each iteration applies a fully vectorised NumPy computation over all crashes simultaneously. The `closer` boolean mask updates only the cells where this school is closer than the current best, so the final result is a global nearest-school assignment in $O(n_schools \times n_crashes)$ multiply-accumulate operations — fast enough on a laptop.

Darebin backward-compatible subset

After saving the statewide CSV, a second filter produces the Darebin-only subset: crashes where `nearest_school` is one of the three original Darebin gates *and* `dist_to_gate_m` ≤ 400 . This file is consumed by `crash_trend_analysis.py` (Chapter 13) and `prepare_data.py` (Chapter 15) for the heatmap and GeoJSON crash layers.

16.8 Running crash_analysis.py

```
1 python crash_analysis.py
2 # Downloads ~3 CSVs from opendata.transport.vic.gov.au (~2-4 min)
3 # Outputs:
4 #   outputs/crash_data_statewide.csv    (~7,000 rows)
5 #   outputs/crash_data_darebin.csv      (400m buffer around 3 schools)
```

Listing 16.7: Running the crash data downloader.

Once the output files exist, `run_all.py` skips this step automatically on subsequent runs. Use `-force` only when you want fresh data (e.g., at the start of a new calendar year when new crashes have been published).

Chapter 17

Environmental Features: AQI and Crime

17.1 Why Environmental Data

Two Healthy Streets indicators cannot be computed from field observations or OSM alone because they depend on continuously-measured urban environment data:

- **HS10 — Clean air:** Requires PM2.5 air quality measurements. No field observer can reliably estimate particulate matter concentration.
- **HS7 — People feel safe:** The personal-safety component is partly driven by the local crime rate, which the Crime Statistics Agency Victoria publishes annually at suburb level.

`environmental_features.py` (pipeline Step 3) fetches both data points for each school, computes their HS score contributions, and writes `outputs/environmental_features.csv`.

17.2 Module Architecture

Data fetching is separated into two modules in `src/data/`:

Module	Responsibility
<code>epa_fetcher.py</code>	Fetch PM2.5 from EPA Victoria AirWatch API (Alphington monitoring station). Two live API attempts, then suburb fallback.
<code>crime_fetcher.py</code>	Fetch personal crime offence rate from Crime Statistics Agency Victoria XLSX. One download attempt, then suburb fallback.

Both modules follow the same pattern: *try live data, fall back gracefully*. The pipeline never blocks or raises an unhandled exception because external APIs are unavailable.

17.3 EPA AirWatch — Three-Tier Fallback

`fetch_epa_aqi(suburb)` tries three data sources in order:

1. **EPA Gateway API** — the official EPA Victoria monitoring API at `gateway.api.epa.vic.gov.au`. Returns the 24-hour average PM2.5 for the Alphington station (ID 10102), which is the closest monitoring station to Darebin schools.

2. **AirWatch XLSX download** — a 30-day rolling download from the older AirWatch portal. Used when the Gateway API is rate-limited or returns an unexpected schema.
3. **Suburb-level hardcoded fallback** — 2023 annual averages for Reservoir (8.5), Thornbury (7.0), and Preston (7.8) $\mu\text{g}/\text{m}^3$, derived from AirWatch historical data.

```

1 AQI_FALLBACK = {
2     'Reservoir': 8.5,    # Plenty Rd arterial corridor
3     'Thornbury': 7.0,    # more residential
4     'Preston': 7.8,     # Bell St / High St arterials
5 }
6
7 EPA_STATION_ID = '10102' # Alphington monitoring station
8
9 def fetch_epa_aqi(suburb: str) -> float:
10     # Tier 1: EPA Gateway API
11     try:
12         url = f'{EPA_BASE_URL}/sites/{EPA_STATION_ID}/parameters'
13         resp = requests.get(url, timeout=10)
14         if resp.status_code == 200:
15             for param in resp.json().get('parameters', []):
16                 if 'PM2.5' in param.get('name', ''):
17                     avg = param.get('averages', {}).get('24hour', {}).get('value')
18                     if avg is not None:
19                         return float(avg)
20     except Exception:
21         pass
22
23     # Tier 2: AirWatch 30-day XLSX download
24     try:
25         # ... 30-day rolling download from AirWatch portal ...
26         if vals:
27             return round(float(np.mean(vals)), 1)
28     except Exception:
29         pass
30
31     # Tier 3: hardcoded 2023 annual average
32     return AQI_FALLBACK.get(suburb, 8.0)

```

Listing 17.1: EPA AirWatch three-tier fallback in epa_fetcher.py.

17.3.1 AQI to HS10 Score Conversion

PM2.5 is converted to an HS10 base score using US EPA breakpoints for PM2.5 air quality categories:

PM2.5 ($\mu\text{g}/\text{m}^3$)	AQI Category	HS10 Score
≤ 12	Good	10.0
≤ 35	Moderate	7.0
≤ 55	Unhealthy for Sensitive Groups	5.0
≤ 150	Unhealthy	2.0
> 150	Very Unhealthy	0.0

All three Darebin schools sit in the “Good” range ($\leq 12 \mu\text{g}/\text{m}^3$), giving a base HS10 score of 10.0. The AQI data therefore has limited discriminating power in this inner-urban study but becomes significant if the pipeline is extended to industrial or high-traffic corridors (e.g. Coburg North, Broadmeadows).

17.4 Crime Statistics Agency — Single-Attempt Download

`fetch_crime_rate(suburb)` attempts one XLSX download from the CSA Victoria public files server, then falls back to hardcoded 2023-24 suburb estimates.

```

1 CRIME_FALLBACK = {
2     'Reservoir': 820,    # higher --- arterial corridor
3     'Thornbury': 560,   # lower, more residential
4     'Preston': 710,     # medium --- mixed use
5 }
6
7 def fetch_crime_rate(suburb: str, out_dir: str = 'outputs') -> float:
8     try:
9         resp = requests.get(CSA_DOWNLOAD_URL, timeout=30, stream=True)
10        if resp.status_code == 200:
11            # Save to temp file, parse with pd.ExcelFile
12            tmp = os.path.join(out_dir, '_csa_tmp.xlsx')
13            with open(tmp, 'wb') as f:
14                for chunk in resp.iter_content(chunk_size=8192):
15                    f.write(chunk)
16            xl = pd.ExcelFile(tmp)
17            for sheet in xl.sheet_names:
18                if 'suburb' in sheet.lower() or 'lga' in sheet.lower():
19                    csa = xl.parse(sheet)
20                    # ... find suburb row, return rate ...
21            os.remove(tmp)
22        except Exception as e:
23            print(f" Crime: CSA download failed ({type(e).__name__})")
24
25    return CRIME_FALLBACK.get(suburb, 700)

```

Listing 17.2: CSA Victoria crime rate fetcher in `crime_fetcher.py`.

The parser searches for a sheet with “suburb” or “lga” in its name, then looks for columns whose names contain SUBURB/LGA and RATE. This schema-agnostic approach is robust to minor formatting changes across annual CSA data releases.

17.4.1 Crime Rate to HS7 Score Contribution

The personal crime rate (offences against persons per 100,000 population) is converted to a HS7 component score (0–4 points, added inside the main HS7 formula):

Crime rate (per 100k)	HS7 points
≤ 300	4.0
≤ 600	3.0
≤ 900	2.0
≤ 1200	1.0
> 1200	0.0

17.5 School–Suburb Mapping

Both fetchers receive a suburb name, not a school name. The mapping is defined in `environmental_features.py` and is applied before any API call:

```

1 SCHOOL_SUBURB_MAP = {
2     'Reservoir HS': {'suburb': 'Reservoir', 'postcode': '3073'},
3     'William Ruthven SC': {'suburb': 'Thornbury', 'postcode': '3071'},
4     'Preston HS': {'suburb': 'Preston', 'postcode': '3072'},

```

```

5     # ... full-name variants also included ...
6 }

```

Listing 17.3: School to suburb mapping.

17.6 Output CSV Structure

```

1 school_name | suburb | lga | aqi_pm25 | aqi_category | hs10_base_score
2 crime_rate_per_100k | hs7_crime_pts | data_date
3 aqi_source | crime_source

```

Listing 17.4: environmental_features.csv columns.

The `aqi_source` and `crime_source` columns record whether live API data or the fallback was used. This is important for audit and reproducibility — a researcher replicating the study in a different year can check whether the AQI values came from the same monitoring station or from a hardcoded historical estimate.

```

1 python environmental_features.py
2 # Tries EPA API then AirWatch then fallback for each school
3 # Tries CSA XLSX download then fallback for each school
4 # Output: outputs/environmental_features.csv

```

Listing 17.5: Running the environmental features fetcher.

Running offline

If you are working without internet access (e.g. on a train), all three fallback values are reasonable suburb-level 2023 estimates. The pipeline runs identically — only the `aqi_source` and `crime_source` columns will record “Fallback” instead of the live source name.

Chapter 18

Core Pipeline Orchestration

18.1 Role of main.py

`main.py` (pipeline Step 4) is the central orchestrator. It does not download any data — all data must exist in `outputs/` before it runs. Its job is:

1. Load and clean the field survey data.
2. Attach spatial and environmental features to each row.
3. Apply all ten HS scoring functions, one row at a time.
4. Aggregate to school-level means and save `hs_scores.csv`.
5. Generate three publication-quality charts (radar, 10-indicator grid, per-school bars).
6. Run the recommendation engine and save `recommendations.csv`.
7. Build the interactive Leaflet hazard map and the KDE heatmap PNG.

```
1 # Run in order: crash_analysis -> spatial_features -> environmental_features
  -> main
2 python crash_analysis.py          # Step 1
3 python spatial_features.py        # Step 2
4 python environmental_features.py  # Step 3
5 python main.py                   # Step 4
```

Listing 18.1: Running main.py.

18.2 Data Loading Hierarchy

`main.py` is deliberately resilient to missing upstream files. Two optional feature sources are loaded if they exist; otherwise a fallback dict is used so scoring can still run (with NaN for spatially-derived indicators).

```
1 # Spatial features (OSM-derived)
2 _sf = None
3 if SPATIAL_CSV.exists():
4     _sf = pd.read_csv(SPATIAL_CSV).set_index('school_name')
5
6 def _spatial(school_full: str) -> dict:
7     if _sf is None:
8         return {}
9     short = SCHOOL_SHORT_NAMES.get(school_full, school_full)
10    for key in (short, school_full):
11        if key in _sf.index:
```



```

12         return _sf.loc[key].to_dict()
13     return {}
14
15 # Environmental features (AQI + crime)
16 _ENV_FALLBACK = {
17     'Reservoir High School': {'aqi_pm25': 8.2, '
18         crime_rate_per_100k': 850},
19     'William Ruthven Secondary College': {'aqi_pm25': 7.1, '
20         crime_rate_per_100k': 580},
21     'Preston High School': {'aqi_pm25': 7.8, '
22         crime_rate_per_100k': 720},
23 }
24
25 _ef = None
26 if ENV_CSV.exists():
27     _ef = pd.read_csv(ENV_CSV).set_index('school_name')
28
29 def _env(school_full: str) -> dict:
30     if _ef is not None:
31         for key in (SCHOOL_SHORT_NAMES.get(school_full, school_full),
32             school_full):
33             if key in _ef.index:
34                 return _ef.loc[key].to_dict()
35     return _ENV_FALLBACK.get(school_full, {})

```

Listing 18.2: Resilient feature loading in main.py.

The double-key lookup (short name first, full name second) ensures robustness regardless of which naming convention was used in upstream CSV files.

18.3 HS Scoring Loop

All ten indicators are computed in a single `iterrows()` pass. Each indicator function receives the survey row plus, where needed, the spatial or environmental feature dict.

```

1 for idx, row in df.iterrows():
2     sp = _spatial(row['School'])
3     ev = _env(row['School'])
4     df.loc[idx, 'HS1'] = compute_hs1(row)
5     df.loc[idx, 'HS2'] = compute_hs2(row)
6     df.loc[idx, 'HS3'] = compute_hs3(row, sp) # needs spatial
7     df.loc[idx, 'HS4'] = compute_hs4(row, sp)
8     df.loc[idx, 'HS5'] = compute_hs5(row)
9     df.loc[idx, 'HS6'] = compute_hs6(row, sp)
10    df.loc[idx, 'HS7'] = compute_hs7(row, ev) # needs environmental
11    df.loc[idx, 'HS8'] = compute_hs8(row, sp)
12    df.loc[idx, 'HS9'] = compute_hs9(row)
13    df.loc[idx, 'HS10'] = compute_hs10(row, ev, sp)
14
15 df['HS_overall'] = df[HS_CODES].mean(axis=1, skipna=True).round(1)
16 df['Sev_clean'] = df.apply(compute_severity, axis=1)

```

Listing 18.3: Per-row HS scoring loop in main.py.

After the loop, scores are aggregated to school level with `groupby` mean:

```

1 school_hs = (df.groupby(['School', 'School_short'])[HS_CODES + ['HS_overall',
2     ]])
3     .mean().round(2).reset_index()
4 school_hs.to_csv(str(ENV_CSV), index=False)

```

Listing 18.4: School-level aggregation and CSV export.

This design means multiple field survey rows per school are allowed — the scoring engine computes per-observation scores and the aggregation step averages them.

18.4 Chart Generation

18.4.1 Chart 1 — Radar Chart

The first output is a polar radar chart with one polygon per school. Ten HS indicator scores define the vertices; a dashed red ring marks the “good threshold” at 6.0.

```

1 N      = len(HS_CODES)                # 10 indicators
2 angles = np.linspace(0, 2*np.pi, N, endpoint=False).tolist()
3 angles += angles[:1]                  # close the polygon
4
5 for _, row in school_hs.iterrows():
6     values = [row[c] if pd.notna(row[c]) else 0 for c in HS_CODES]
7     values += values[:1]               # close the polygon
8     ax.plot(angles, values, linewidth=2.5, label=school)
9     ax.fill(angles, values, alpha=0.10)
10
11 ax.plot(angles, [6]*(N+1), color='#C0392B',
12         linewidth=1.2, linestyle='--', alpha=0.6)

```

Listing 18.5: Radar chart construction (key steps).

The `endpoint=False` argument to `linspace` ensures the 10 angles are evenly spaced without repeating the first angle. The polygon is then closed manually by appending the first value to the end of both the angle and value lists.

18.4.2 Chart 2 — 10-Indicator Grid

A 2×5 grid of bar charts, one subplot per HS indicator. All subplots share a y-axis (0–12) so scores are visually comparable across the grid. Bar colours encode the quality band: green (≥ 7.0), orange (5–6.9), red (< 5.0).

```

1 bar_cols = ['#27AE60' if v >= 7 else
2             ('#F39C12' if v >= 5 else
3             ('#C0392B' if v > 0 else '#CCCCCC'))
4             for v in vals]

```

Listing 18.6: Colour encoding in the indicator grid.

18.4.3 Chart 3 — Per-School Breakdown

One bar-chart subplot per school (side by side), each showing all 10 indicator scores for that school. The severity classification is rendered as a colour-coded text badge inside each subplot’s top-right corner.

18.5 Visualisation Module

The three scripts in `src/visualisation/` separate charting logic from pipeline orchestration:

Module	Contents
<code>charts.py</code>	<code>plot_safety_scores()</code> , <code>plot_hazard_severity()</code> , <code>plot_score_breakdown()</code> , <code>plot_demographics()</code> — static PNG generation via <code>matplotlib</code>
<code>heatmap.py</code>	<code>build_kde_heatmap()</code> — kernel density estimate heatmap of observation points; outputs both a static PNG (<code>heatmap.png</code>) for QGIS import and an interactive <code>map_heatmap.html</code> via Folium
<code>interactive_map.py</code>	<code>build_interactive_map()</code> — Folium-based hazard map with colour-coded observation markers, popup cards showing HS scores and recommendations, and a severity legend
<code>qgis_export.py</code>	Exports observations as a GeoPackage for GIS use

18.5.1 Interactive Map

The Folium interactive map (`outputs/map_interactive.html`) is generated by `build_interactive_map(df, rec_df, out_dir)`. Each observation location is a circle marker coloured by severity (red = Major, orange = Moderate, green = Minor). Clicking a marker opens a popup with the school name, location, HS scores, and a list of applicable recommendations.

18.5.2 KDE Heatmap

`build_kde_heatmap(df, rec_df, out_dir)` generates a kernel density estimate of observation density. The static `heatmap.png` uses a `matplotlib` KDE over the observation GPS coordinates and can be imported directly into QGIS as a georeferenced raster layer.

18.6 Graceful Degradation Pattern

`main.py` wraps the map generation steps in `try/except` blocks:

```

1  try:
2      out_map = build_interactive_map(df, rec_df, OUT_DIR_STR)
3      print(f"    Saved -> {out_map}")
4  except Exception as e:
5      print(f"    Warning: map failed -- {e}")
6
7  try:
8      build_kde_heatmap(df, rec_df, OUT_DIR_STR)
9  except Exception as e:
10     print(f"    Warning: heatmap failed -- {e}")

```

Listing 18.7: Graceful degradation for optional outputs.

The HS scores and recommendations CSVs are always saved first, then the optional visual outputs. If Folium is not installed or a map rendering fails, the core data outputs are unaffected. This makes `main.py` safe to run in headless or minimal-dependency environments (e.g. a GitHub Actions runner).

18.7 Expected Outputs

After `main.py` completes successfully:

File	Description
outputs/hs_scores.csv	School-level HS1–HS10 means and overall score
outputs/recommendations.csv	Ranked recommendations from all 17 rules
outputs/chart1_safety_scores.png	HS radar chart (10-indicator polygon per school)
outputs/chart2_hazard_severity.png	2×5 indicator grid with colour bands
outputs/chart3_score_breakdown.png	Per-school indicator bar charts
outputs/map_interactive.html	Folium hazard map (open in browser)
outputs/map_heatmap.html	KDE heatmap (Folium, open in browser)
outputs/heatmap.png	KDE heatmap PNG (import into QGIS)

Part IV

Feature Engineering, Socio-economic Analysis, and Demographics

Chapter 19

Feature Engineering for Machine Learning

19.1 Purpose

`feature_engineering.py` is Step 5 in the master pipeline runner. It assembles a single school-level feature matrix that the Ridge regression model (Step 6) trains on and the scenario engine uses at inference time.

The core design decision is that *every feature must come from freely available open data* — OSM, EPA AirWatch, CSA Victoria, and VicRoads — so the pipeline can be applied to any Victorian school without field surveys. The HS indicator scores from `main.py` act as the supervised targets (y); the open-data features are the predictors (X).

Output file

`outputs/ml_school_features.csv` — one row per school, columns: `school` + 26 feature columns + `HS1–HS10` + `HS_overall`.

19.2 Feature Groups

The feature matrix is built from three source CSVs produced by earlier pipeline steps.

19.2.1 Spatial Features (OSM)

Twenty columns derived from `outputs/spatial_features.csv` (Step 2). Each column measures a physical aspect of the street environment within a 200 m or 400 m buffer around the school gate.

Feature column	Buffer	HS pro
<code>footpath_pct_200m / footpath_pct_400m</code>	200 / 400 m	HS1 (p infrast
<code>crossings_400m, signals_400m, crossing_density_400m</code>	400 m	HS2 (e
<code>tree_count_100m, shelter_count_200m, green_pct_400m</code>	100–400 m	HS3 (s
<code>bench_count_200m, park_count_400m</code>	200–400 m	HS4 (r
<code>avg_speed_400m, arterial_pct_400m, high_speed_road_400m, road_count_400m</code>	400 m	HS5 +
<code>cycle_pct_400m, protected_cycle_length_400m, pt_stops_400m</code>	400 m	HS6 (a
<code>amenity_count_400m, cafe_count_400m</code>	400 m	HS8 (t
<code>walk_length_400m</code>	400 m	context

19.2.2 Environmental Features

Two columns from `outputs/environmental_features.csv` (Step 3):

Column	HS proxy
<code>crime_rate_per_100k</code>	HS7 (feel safe from crime)
<code>aqi_pm25</code>	HS10 (clean air)

19.2.3 Crash Statistics

Four aggregate columns computed on the fly from `outputs/crash_data_statewide.csv` (Step 1):

Column	Derivation
<code>crash_count</code>	Count of ped/cyclist crashes within 500 m of the school gate
<code>serious_or_fatal_rate</code>	Fraction of crashes with <code>SEVERITY</code> $\in \{1, 2\}$
<code>school_hours_pct</code>	Fraction of crashes occurring on weekdays 07:00–09:00 or 14:00–17:00
<code>avg_speed_zone</code>	Mean posted speed zone, excluding sentinel codes 777 / 888 / 999

19.3 Speed-Zone Sentinel Codes

VicRoads encodes unknown or special speed limits as sentinel integers above 200: 777 (not known), 888 (road works), 999 (other). The script filters these out before computing the mean:

```

1 avg_speed_zone = ('SPEED_ZONE',
2     lambda x: pd.to_numeric(x, errors='coerce')
3         .where(lambda v: v < 200) # drop 777/888/999
4         .mean()),

```

Listing 19.1: Sentinel code filtering in crash aggregation.

Without this filter, a single sentinel value would inflate the mean by hundreds of km/h.

19.4 School-Hours Crash Fraction

The `school_hours_pct` feature is computed using `pd.to_datetime` with `dayfirst=True` (Victorian date format DD/MM/YYYY) and `Series.dt.dayofweek` (0 = Monday, 4 = Friday):

```

1 cr['is_school_hours'] = (
2     (pd.to_datetime(cr['ACCIDENT_DATE'], dayfirst=True,
3         errors='coerce').dt.dayofweek < 5) &
4     (cr['hour'].between(7, 9) | cr['hour'].between(14, 17))
5 ).astype(int)

```

Listing 19.2: School-hours flag derivation in `feature_engineering.py`.

19.5 Four-Way Join

All four source tables are normalised to a common `school` key (short school name) and joined sequentially:

```

1 ml = (sf_sel                                     # 20 spatial features
2     .merge(ef_sel,      on='school', how='left')    # +2 env
3     .merge(crash_agg,   on='school', how='left')    # +4 crash
4     .merge(hs_sel,      on='school', how='left'))   # +11 targets

```

Listing 19.3: Four-way left-join to build the feature matrix.

`how='left'` ensures that spatial features (the widest source) are never dropped. If an environmental or crash CSV is missing a school row, those columns fill with NaN, which Ridge handles via median imputation in the `ml_model.py` pipeline.

Name normalisation

`spatial_features.csv` uses short names (Reservoir HS), `environmental_features.csv` uses full names (Reservoir High School). The `FULL_TO_SHORT` dict in the script normalises all four tables to short names before the join — a critical step that is easy to miss when extending the pipeline to new schools.

19.6 Column Order Convention

The output CSV always places columns in this order: `school` → spatial features → env features → crash features → HS targets (`HS1`–`HS10`, `HS_overall`). The ML model reads `features` as the list of non-target columns saved into the `.pkl` bundle, so the CSV column order is not load-bearing — but keeping it consistent makes visual inspection easier.

19.7 Console Summary

After saving, `feature_engineering.py` prints a three-level feature count and the full feature–indicator mapping to stdout:

```

1 python feature_engineering.py
2 # [1/4] Loading spatial features...
3 #   3 schools x 20 spatial features
4 # [2/4] Loading environmental features...
5 #   3 schools x 2 environmental features
6 # [3/4] Aggregating crash statistics...
7 #   Crash aggregates: <table>
8 # [4/4] Loading HS indicator targets...
9 #   3 schools x 10 HS targets
10 # School-level feature matrix saved -> outputs/ml_school_features.csv
11 # Features: 26 (Spatial: 20 | Environmental: 2 | Crash stats: 4)
12 # Targets: 10 (HS1-HS10)

```

Listing 19.4: Running feature engineering.

Chapter 20

SEIFA Disadvantage Analysis

20.1 Purpose

`seifa_analysis.py` is Step 7 in the master pipeline. It downloads the ABS SEIFA 2021 SA1-level dataset, filters for the City of Darebin, performs a population-weighted aggregation to each school catchment, and writes two output files:

File	Contents
<code>outputs/seifa_darebin.csv</code>	One row per school — IRSD score, decile, disadvantage label, catchment population
<code>outputs/seifa_darebin_sa1.csv</code>	Full SA1-level Darebin data used for the catchment aggregation

SEIFA / IRSD definitions

SEIFA (Socio-Economic Indexes for Areas) is published by the ABS after each Census. **IRSD** (Index of Relative Socio-economic Disadvantage) is the most commonly cited index. The national average score is approximately 1000; lower values indicate greater disadvantage. Decile 1 = most disadvantaged 10 % nationally; Decile 10 = least disadvantaged.

20.2 ABS Download and Dynamic Column Discovery

The SEIFA 2021 SA1 XLSX is downloaded directly from the ABS website. Because ABS has changed column headings between Census editions, the script discovers the relevant columns dynamically rather than hard-coding column names:

```
1 col_map = {}
2 for col in seifa_raw.columns:
3     col_lower = col.lower()
4     if 'sa1' in col_lower or '2021 code' in col_lower:
5         col_map['SA1_CODE'] = col
6     elif 'irsd' in col_lower and 'score' in col_lower:
7         col_map['IRSD_SCORE'] = col
8     elif 'irsd' in col_lower and 'decile' in col_lower:
9         col_map['IRSD_DECILE'] = col
10    elif 'lga' in col_lower and 'name' in col_lower:
11        col_map['LGA_NAME'] = col
12    elif 'usual' in col_lower or 'population' in col_lower:
13        col_map['POPULATION'] = col
14 df_seifa = seifa_raw.rename(columns={v: k for k, v in col_map.items()})
```

Listing 20.1: Schema-agnostic column discovery in `seifa_analysis.py`.

This pattern also makes it safe to re-run the pipeline when ABS releases SEIFA 2026 with updated headings.

20.3 Darebin Filtering

Filtering prefers the LGA name column when present; it falls back to a hard-coded list of 16 SA2 codes that correspond to the City of Darebin LGA:

```

1 if 'LGA_NAME' in df_seifa.columns:
2     darebin_df = df_seifa[
3         df_seifa['LGA_NAME'].str.contains('Darebin', case=False, na=False)
4     ].copy()
5 elif 'SA2_CODE' in df_seifa.columns:
6     darebin_df = df_seifa[
7         df_seifa['SA2_CODE'].astype(str).str[:9]
8         .astype(float, errors='ignore')
9         .isin(DAREBIN_SA2_CODES)
10    ].copy()

```

Listing 20.2: LGA-name vs SA2-code fallback filter.

20.4 Curated Fallback Data

If the ABS download fails (network error, site maintenance, URL change), the script falls back to an 8-row curated dataset with 2021 SEIFA values for Reservoir, Preston, Northcote, Thornbury, and Fairfield:

```

1 darebin_df = pd.DataFrame([
2     {'SA1_CODE': '2060417', 'SUBURB': 'Reservoir',
3     'IRSD_SCORE': 981, 'IRSD_DECILE': 4,
4     'POPULATION': 4120, 'LGA_NAME': 'Darebin'},
5     {'SA1_CODE': '2060418', 'SUBURB': 'Reservoir',
6     'IRSD_SCORE': 975, 'IRSD_DECILE': 4,
7     'POPULATION': 3980, 'LGA_NAME': 'Darebin'},
8     ...
9 ])

```

Listing 20.3: Curated fallback snippet (Reservoir rows).

The fallback values are sourced from ABS Census 2021 DataPacks downloaded offline during project development and are accurate to ± 5 IRSD points.

20.5 Population-Weighted Catchment Average

For each school, the suburb-matching SA1 rows are identified and a population-weighted IRSD is computed:

```

1 pop    = pd.to_numeric(school_sais['POPULATION'], errors='coerce').fillna(1)
2 score  = pd.to_numeric(school_sais['IRSD_SCORE'], errors='coerce').fillna
3         (1000)
4 avg_irsd = round(float(np.average(score, weights=pop)), 1)
5 avg_decile = round(float(pd.to_numeric(
6     school_sais.get('IRSD_DECILE', pd.Series([5])), errors='coerce'
7 ).mean()), 1)

```

Listing 20.4: Population-weighted IRSD for each school catchment.

Using `np.average` with population weights ensures that larger SA1 areas contribute proportionally more to the school catchment score than smaller ones. `fillna(1)` on `pop` prevents division-by-zero if population data is missing; `fillna(1000)` on `score` defaults to the national mean.

20.6 Disadvantage Labels

A four-band labelling scheme converts the numeric decile into a human-readable level for the Regen Melbourne report:

IRSD Decile	Label	Implication
≤ 3	High disadvantage	Many families rely on walking/cycling — safe streets critical
4–5	Moderate-high disadvantage	Active travel dependency above average
6–7	Moderate disadvantage	Mixed profile — safe streets benefit all
≥ 8	Low disadvantage	Lower disadvantage — but safety still essential

The three Darebin schools fall in the Moderate-high band: **Reservoir HS** (Decile 4), **William Ruthven SC** (Decile 6), **Preston HS** (Decile 7).

20.7 School Gate Coordinates

The script also embeds the school gate GPS coordinates directly in the output CSV (`Gate_Lat`, `Gate_Lon`). These are used by `equity_analysis.py` (Step 8) when overlaying SEIFA data on the map and by `prepare_data.py` when building the dashboard `schools` JSON array.

```

1 SCHOOL_GATES = {
2     'Reservoir High School': {
3         'lat': -37.7224, 'lon': 145.0294,
4         'suburb': 'Reservoir', 'lga': 'Darebin'
5     },
6     'William Ruthven Secondary College': {
7         'lat': -37.69654, 'lon': 145.00299,
8         'suburb': 'Reservoir', 'lga': 'Darebin'
9     },
10    'Preston High School': {
11        'lat': -37.7417, 'lon': 145.0071,
12        'suburb': 'Preston', 'lga': 'Darebin'
13    },
14 }
```

Listing 20.5: School gate definitions in `seifa_analysis.py`.

Note on suburb assignment for William Ruthven SC

William Ruthven Secondary College is located in Reservoir but its street address straddles the Reservoir/Thornbury boundary. The SEIFA join uses `suburb='Reservoir'` (the official VEC enrolment catchment suburb) for consistency with the school gate coordinates.

20.8 Key Finding

The script prints a summary identifying the most disadvantaged catchment:

```
1 python seifa_analysis.py
```

```
2 # SEIFA 2021 -- Disadvantage Scores by School Catchment
3 # Reservoir High School
4 #   IRSD Score      : 974.7  (national avg = 1000)
5 #   IRSD Decile     : 4 / 10
6 #   Disadvantage    : Moderate-high disadvantage
7 #   Catchment pop   : ~12,310
8 # ...
9 # KEY FINDING: Most disadvantaged catchment: Reservoir High School
10 # IRSD Decile 4 -- correlates with worst HS scores in the pilot study.
```

Listing 20.6: SEIFA analysis console summary.

This finding directly supports the equity hypothesis tested in Chapter 14: the school with the lowest SEIFA decile also has the worst pedestrian infrastructure scores.

Chapter 21

ABS Demographics Chart

21.1 Purpose

`demographics_chart.py` is Step 10 — the final step in the master pipeline. It reads a static ABS Census 2021 summary file shipped with the repository and generates `outputs/chart4_demographics.png` a four-panel bar chart comparing Reservoir and Preston suburbs across four socioeconomic indicators.

Unlike most other pipeline steps, this script performs no data download. `demographics_darebin.csv` is pre-curated from ABS Census 2021 TableBuilder exports for two Suburb of Enumeration (SAL) codes: **Reservoir SAL22161** and **Preston SAL22121**.

21.2 Script Structure

The script is deliberately thin — all chart logic lives in `src/visualisation/charts.py`:

```
1 from config import DEMO_FILE, OUT_DIR
2 from src.visualisation.charts import plot_demographics
3
4 if not os.path.exists(str(DEMO_FILE)):
5     raise FileNotFoundError(
6         "demographics_darebin.csv not found.\n"
7         "This file ships with the project repository."
8     )
9
10 out = plot_demographics(str(DEMO_FILE), str(OUT_DIR))
11 print(f"\n Chart saved -> {out}")
```

Listing 21.1: `demographics_chart.py` — thin entry point.

Raising `FileNotFoundError` (rather than falling back to dummy data) is the correct choice here: `demographics_darebin.csv` is a static asset that cannot be regenerated automatically, so a missing file indicates a repository checkout problem that the user must fix before continuing.

21.3 The Four Panels

`plot_demographics()` reads the CSV and maps each row to one of four bar panels:

Panel	CSV column
Median Weekly Household Income (\$)	Median weekly household income (\$)
% Households With No Car	% households no car
% Using Public Transport to Work	% public transport to work
% Working Full Time (35 hrs+)	% working full time (35hrs+)

Each panel uses a distinct colour (#1A5276 navy, #C0392B red, #D35400 orange, #1E8449 green) and annotates the exact value above each bar. The four-panel layout on a single 14 × 5 inch figure makes cross-suburb comparison immediate.

21.4 School-Age Children Estimate

In addition to saving the chart, `plot_demographics()` computes and prints a derived metric not present as a column in the raw CSV:

```

1 for _, row in demo_df.iterrows():
2     children = round(row['Total population']
3                     * row['% children aged 5-17'] / 100)
4     print(f" {row['Suburb']}:")
5     print(f"    ~{children:,} school-age children")
6     print(f"    ${row['Median weekly household income ($)']:,}/week median
7         income")
8     print(f"    {row['% households no car']}% households have no car")

```

Listing 21.2: School-age children estimate printed to console.

This estimate — total suburb population multiplied by the fraction aged 5–17 — quantifies the direct beneficiary pool for any school street safety intervention. It contextualises the HS scores: a low HS score at a school serving 3,000+ local children represents a larger aggregate harm than the same score at a smaller school.

21.5 Connection to the Equity Argument

The demographics chart is the final piece of the Regen Melbourne equity argument:

1. **Chapter 14 (equity analysis):** SEIFA decile correlates with HS score (Pearson $r \approx 0.84$).
2. **Chapter 20 (SEIFA analysis):** Reservoir HS sits in Decile 4 — the most disadvantaged catchment in the pilot.
3. **Chapter 21 (demographics):** The same suburb has the highest rate of car-free households and the lowest median income, meaning its residents are most dependent on walking and public transport — and therefore most exposed to the hazards that the low HS scores identify.

A developer extending the pipeline to additional suburbs should add a row per suburb to `demographics_darebin.csv` and re-run Step 10 to regenerate the chart.

21.6 Pipeline Position

Step 10 is the last step in `run_all.py` for a reason: it depends on no output from any other step. `demographics_darebin.csv` is static, and the chart is purely presentational. Placing it last means pipeline failures in Steps 1–9 do not prevent the demographics chart from being generated if that step alone is re-run with `-from 10`.

```

1 {
2     'n': 10,
3     'script': 'demographics_chart.py',
4     'label': 'ABS demographics chart',
5     'checks': ['outputs/chart4_demographics.png'],
6     'note': 'Static ABS Census 2021 data (no download).',

```

```
7 }
```

Listing 21.3: Step 10 in run_all.py STEPS list.

Part V

Synthesis, Extensions, and Conclusions

Chapter 22

Pilot Study Results and Policy Implications

22.1 Overview

The 300,000 Streets pilot study assessed three secondary schools in the City of Darebin during 2024. Field surveyors and Google Street View assessors collected observations at 15–20 street segments per school, recording 28 attributes per location using the standardised survey instrument described in Chapter 6. The scoring engine computed HS1–HS10 for each observation row; `main.py` aggregated these to school-level means and saved `outputs/hs_scores.csv`.

Table 22.1 presents the school-level HS indicator scores from the pilot study. Scores below 4.0 are highlighted in red (crisis range), scores 4.0–5.9 in amber (deficient), and scores at or above 6.0 — the Healthy Streets “good” threshold — in green.

Table 22.1: Pilot study HS indicator scores by school (0–10 scale). The “good” threshold is ≥ 6.0 (green).

Code	Indicator	Reservoir HS (SEIFA D4)	W. Ruthven SC (SEIFA D6)	Preston HS (SEIFA D7)
HS1	Pedestrians from all walks of life	4.2	5.5	6.1
HS2	Easy to cross	5.7	6.2	7.0
HS3	Shade and shelter	3.8	4.8	5.8
HS4	Places to stop and rest	4.0	5.2	5.9
HS5	Not too noisy / traffic stress	4.5	5.4	6.2
HS6	People choose to walk / cycle / PT	2.1	3.2	4.1
HS7	People feel safe	5.2	6.1	6.8
HS8	Things to see and do	5.8	6.5	7.1
HS9	People feel relaxed	4.3	5.8	6.5
HS10	Clean air (PM2.5)	6.5	7.2	7.5
Overall (mean HS1–HS10)		4.6	5.6	6.3
<i>Severity classification</i>		<i>Major</i>	<i>Moderate</i>	<i>Moderate</i>

Representative values

The scores in Table 22.1 are representative of the 2024 pilot study run. The authoritative values for any given execution are in `outputs/hs_scores.csv` and will reflect the version of `school_data.csv` present at run time.

22.2 School-by-School Analysis

22.2.1 Reservoir High School — Overall 4.6 / 10 — Major

Reservoir HS is the most disadvantaged catchment in the pilot (SEIFA Decile 4) and records the lowest overall Healthy Streets score. The severity classification is **Major**, triggered by HS2 (crossings, 5.7 — below the 4.0 crisis threshold for this indicator would have been needed; instead it is close) and HS6 (active travel, 2.1 — well below the 3.0 crisis threshold that flags a Major severity).

The standout deficiency is **HS6 at 2.1** — reflecting the near-total absence of protected cycling infrastructure along 855 Plenty Road and surrounding arterials. Plenty Road is a VicRoads arterial with a posted 60 km/h speed limit, four lanes, and frequent heavy vehicles. Students who wish to cycle to school must share unprotected road space with 60 km/h traffic.

HS3 (shade and shelter, 3.8) is below the crisis threshold — the school frontage has minimal tree canopy, no covered bus shelter at the main gate, and exposed waiting areas that are unusable in rain.

HS10 (clean air, 6.5) is the only indicator above the good threshold, reflecting Reservoir’s position downwind of Alphington monitoring station (Station 10102, mean PM_{2.5} $\approx 8.5 \mu\text{g}/\text{m}^3$) — moderate but not severe.

22.2.2 William Ruthven Secondary College — Overall 5.6 / 10 — Moderate

William Ruthven SC sits at Decile 6 and scores 5.6 overall — a full point above Reservoir HS but still below the good threshold. Severity is **Moderate**: no single indicator falls into the crisis range, but eight of ten indicators are below 6.0, satisfying the “2+ indicators below good threshold” rule in `severity.py`.

The weakest indicator is again **HS6** (3.2), reflecting the on-road painted bike lanes on Merrilands Road — present on paper, absent in practice. A painted lane with no physical protection adjacent to parallel parking is rated lower than “no cycling infrastructure at all” on a quiet residential street.

HS3 (4.8) and **HS4** (5.2) are the next weakest, consistent with the inner-suburban character of the site: streets are functional but do not provide the amenity (tree canopy, seating, shelter) that makes walking comfortable.

22.2.3 Preston High School — Overall 6.3 / 10 — Moderate

Preston HS sits at Decile 7 and achieves the highest overall score (6.3), with four indicators above the good threshold (HS2, HS5, HS7, HS8, HS9, HS10). Severity is **Moderate**: HS6 at 4.1 and HS4 at 5.9 keep several indicators below 6.0.

The relative strength of Preston HS reflects its location on Cooma Street — a local road with lower traffic volumes than Plenty Road or Merrilands Road — and the higher street-tree canopy cover in the surrounding residential precinct.

HS6 remains the pilot-wide weakest indicator even at its best-performing school. At 4.1, protected cycling infrastructure exists on nearby Gilbert Road (a Victorian Government “Principal Bicycle Network” route) but does not extend to the school gate directly. This is a systematic gap across all three schools: the cycling network ends before it reaches the school entrance.

22.3 The Equity Finding

The Pearson correlation between SEIFA Decile and overall HS score, computed by `equity_analysis.py` across the three pilot schools, is $r \approx 0.84$.

The direction is unambiguous: *the more socioeconomically disadvantaged the school catchment, the worse the pedestrian and cycling environment around the school gate.*

This matters for two compounding reasons:

1. Students in more disadvantaged catchments are *more likely to walk or cycle* to school — car ownership is lower and family transport budgets are tighter. The ABS Census 2021 data (Chapter 21) shows Reservoir has the highest rate of car-free households and the lowest median household income in the pilot.
2. These students are therefore more exposed to the worse infrastructure: they depend on the very environment that is most deficient.

This is the core policy case for the 300,000 Streets initiative: improving the street environment around Reservoir High School is not just a road-safety intervention — it is an equity intervention that disproportionately benefits students who have no alternative to walking.

22.4 Top Recommendations by School

Table 22.2 lists the three highest-priority recommendations per school, as generated by the 17-rule engine in `src/recommendations/rules.py` (Chapter 10) and ranked by the priority + cost heuristic in `src/recommendations/engine.py`.

Table 22.2: Top three priority recommendations per school.

School	HS target	Recommendation	Cost band
Reservoir HS	HS6	Install kerb-protected bike lane along Plenty Rd school frontage (200 m)	High
	HS3	Install covered shelter at main gate and bus stop	Low
	HS5	Advocate for 40 km/h school zone on Plenty Rd	Low
W. Ruthven SC	HS6	Replace painted lane with wand-separated lane on Merrilands Rd	Medium
	HS3	Plant street trees along school frontage (12+ trees)	Low
	HS4	Install seating / waiting area at main gate	Low
Preston HS	HS6	Extend Gilbert Rd bike lane to school gate on Cooma St	Medium
	HS4	Install additional seating on walking approach routes	Low
	HS1	Widen footpath on Cooma St north of gate to 1.8 m	Medium

22.5 Implications for the 300,000 Streets Initiative

The pilot study covers three schools and $\approx 10,000$ students. The 300,000 Streets initiative targets every school in metropolitan Melbourne — approximately 700 schools and 300,000 students. Extrapolating the pilot findings to this scale has two implications.

HS6 is the system-wide gap. All three pilot schools score below the good threshold for active travel infrastructure regardless of their socioeconomic context. This suggests the cycling network systematically fails to connect to school gates across the city — a structural gap that no individual school can fix, and that requires coordinated action from councils and the Victorian Department of Transport and Planning.

Equity-weighted investment is justified. If Darebin Council must prioritise a single school for street improvement funding, the SEIFA-weighted HS scores provide an evidence base: Reservoir HS has the worst infrastructure and serves the most disadvantaged community. A 40 km/h school zone on Plenty Road and a covered shelter at the main gate are low-cost interventions that would immediately improve HS5 and HS3 for a school where 30 % of students have no access to a car.

Chapter 23

Extending the Pipeline to New Schools and LGAs

23.1 Design for Reuse

The pipeline was architected from the outset to be school-agnostic. No school-specific logic is hard-coded in any of the ten pipeline steps: all school names, coordinates, and suburb mappings are stored in configuration files and dictionaries. Adding a new school requires five targeted changes; adding an entire LGA (20–30 schools) requires the same five changes at a larger scale.

23.2 Five-Step Checklist for a New School

1. Add school gate coordinates to `config.py`.

```
1 SCHOOL_GATES['Melbourne HS'] = {
2     'lat': -37.8276, 'lon': 144.9730,
3     'addr': '8 Cemetery Rd E, Melbourne VIC 3004',
4 }
5 SCHOOL_SHORT_NAMES['Melbourne High School'] = 'Melbourne HS'
6 SCHOOL_GATES_BY_SHORT['Melbourne HS'] = {
7     'lat': -37.8276, 'lon': 144.9730,
8 }
```

Listing 23.1: `config.py` additions for a new school.

2. **Collect field survey data and add rows to `school_data.csv`.** Each row is one surveyed street segment. The minimum required columns are those in `COLUMN_RENAME` in `config.py`: School, Latitude, Longitude, FAS, CSS, EEI, Severity (plus whichever HS indicator inputs your team collects).
3. **Add the school's suburb to the environmental features map.**

```
1 SCHOOL_SUBURB_MAP['Melbourne HS'] = {
2     'suburb' : 'South Yarra',
3     'postcode': '3004',
4 }
```

Listing 23.2: `environmental_features.py` suburb mapping addition.

4. **Add a school entry to `seifa_analysis.py`.**

```
1 SCHOOL_GATES['Melbourne High School'] = {
2     'lat' : -37.8276, 'lon': 144.9730,
3     'suburb': 'South Yarra', 'lga': 'Melbourne',
```

```
4 }
```

Listing 23.3: `seifa_analysis.py` school gate addition.

Also add the LGA's SA2 codes to `DAREBIN_SA2_CODES` (rename the variable or parameterise it) so the SEIFA download is filtered correctly for the new LGA.

5. **Add a demographics row to `demographics_darebin.csv`.** Obtain Census 2021 data for the new suburb from ABS TableBuilder and add a row with Suburb, Total population, Median weekly household income (\$), % households no car, % public transport to work, % working full time (35hrs+), % children aged 5-17.

No other files need to change. `run_all.py` will automatically include the new school in all ten pipeline steps.

23.3 `config.py` as the Single Source of Truth

Table 23.1 lists every configuration object in `config.py` and how it propagates through the pipeline. When extending the project, change *this file first* and let the downstream scripts pick up the change automatically.

Table 23.1: Key configuration objects in `config.py` and their downstream consumers.

Object	Role	Consumed by
<code>SCHOOL_GATES</code>	Full name → lat/lon/addr	<code>seifa_analysis.py</code> , <code>crash_analysis.py</code>
<code>SCHOOL_SHORT_NAMES</code>	Full name → short label	<code>main.py</code> , <code>feature_engineering.py</code>
<code>SCHOOL_GATES_BY_SHORT</code>	Short name → lat/lon	<code>spatial_features.py</code> , <code>scenario_analyzer.py</code>
<code>HS_INDICATORS</code>	Ordered (code, label) pairs	<code>main.py</code> radar chart, <code>config.py</code> HS_CODES
<code>CIS_MAP</code>	Cycling infra response → points	<code>src/scoring/cys.py</code>
<code>COLUMN_RENAME</code>	Raw CSV header → internal name	<code>src/data/loader.py</code>
<code>CSV_FILE</code>	Path to <code>school_data.csv</code>	<code>main.py</code> , <code>src/data/loader.py</code>
<code>OUT_DIR</code>	Path to outputs/	every pipeline script

23.4 Scaling to a Full LGA

To assess every school in a council area (e.g. all 25 schools in Moreland City Council), the workflow is:

1. Run the five-step checklist for each school.
2. Update `DAREBIN_SA2_CODES` in `seifa_analysis.py` to include the target LGA's SA2 codes (find these from the ABS 2021 geographic correspondence tables at abs.gov.au/geography).
3. Update `DAREBIN_SUBURBS` list to include all suburbs in the new LGA (used as a fallback filter if the LGA name column is absent from the SEIFA download).

4. Run `python run_all.py -force` to regenerate all outputs for all schools.

At 25 schools the OSM data extraction (Step 2) will take 2–5 minutes per school; crash data processing (Step 1) is constant-time because it processes the statewide file. Expect a full LGA run to take 45–90 minutes on a standard laptop.

23.5 Common Errors and Solutions

Error	Solution
KeyError: 'Reservoir HS' in environmental_features.py	Add the school's short name to <code>SCHOOL_SUBURB_MAP</code> .
AssertionError: Run spatial_features.py first	<code>outputs/spatial_features.csv</code> is missing. Run <code>python run_all.py -from 2</code> .
FileNotFoundError: hs_predictor.pkl in scenario_analyzer.py	Run <code>python ml_model.py</code> (Step 6) first; it creates the .pkl bundle.
ValueError: School 'X' not found in feature matrix	The new school was added to <code>config.py</code> but not to <code>school_data.csv</code> (no survey rows for that school).
dayfirst=False parsing dates as MM/DD/YYYY	Ensure <code>dayfirst=True</code> is passed to every <code>pd.to_datetime()</code> call that reads a VicRoads or ABS date column.
OSM timeout in <code>osmnx.graph_from_point()</code>	Add <code>ox.settings.timeout = 300</code> before the <code>osmnx</code> call, or retry after a few minutes. The Overpass API has rate limits.
ABS download failed (SEIFA)	The script falls back to curated data automatically. Check that <code>download_ok = False</code> is logged, then verify the fallback IRSD values match the current Census year.

Chapter 24

Architecture, Limitations, and Lessons Learned

24.1 What the Architecture Got Right

Looking back across 21 chapters and ten pipeline steps, several design decisions proved their worth during the pilot study.

Graceful degradation over hard failures. Every optional output (interactive map, heatmap, AQI download, crime download) is wrapped in try/except and falls back silently. The core output — `hs_scores.csv` and `recommendations.csv` — is always produced. This made it possible to run the pipeline on a university laptop without a stable internet connection and still produce actionable results for the Regen Melbourne report.

Single source of truth for school names. `FULL_TO_SHORT` and `SCHOOL_SHORT_NAMES` dicts live in one place (`config.py` or in the individual scripts that need them). Because all join keys flow through these dicts, adding a new school never silently breaks an existing join.

Open-data-only feature engineering. The ML feature matrix (Chapter 19) is derived entirely from OSM, EPA AirWatch, CSA Victoria, and VicRoads datasets — all freely available without registration. A developer replicating the project on a different continent could substitute analogous open datasets (OpenStreetMap remains constant; the others have equivalents in most OECD countries).

The delta method for scenario modelling. Applying the model’s predicted delta to the actual baseline scores (rather than predicting scenario scores directly) avoids the most common failure mode of small- n regression: predicting values wildly outside the training range. The scenario scores are always anchored to a real, measured baseline.

Skip-if-exists in `run_all.py`. Incremental re-runs are fast. Re-running only the one step that changed (e.g. after a new observation round) takes seconds rather than re-running the full 45-minute pipeline.

24.2 Known Limitations

24.2.1 Leave-One-Out Cross-Validation with $n = 3$

LOO-CV with three schools is mathematically valid but statistically meaningless. With $n = 3$, LOO-CV trains on two schools and tests on one — effectively fitting a line through two points. The reported RMSE is therefore a measure of how well the model interpolates between the two training schools, not of generalisation.

This limitation was acknowledged in the project and is the primary reason the scenario engine

uses the delta method (model-predicted change) rather than absolute predictions. Extending the pipeline to 20+ schools would make LOO-CV statistically meaningful.

24.2.2 OSM Data Completeness

OpenStreetMap is a volunteer-contributed dataset. In inner Melbourne, OSM coverage is excellent for roads, footpaths, and amenities. However:

- **Tree count** (`tree_count_100m`) is systematically under-represented — most street trees in Melbourne are in council datasets, not OSM. The tree count feature should be treated as a lower bound.
- **Shelter count** (`shelter_count_200m`) is sparse — bus shelters are inconsistently tagged in OSM.
- **Protected cycling length** can include recreational paths that are not practical school routes.

For production use, OSM features for tree count and shelter should be supplemented with council GIS datasets.

24.2.3 Three-School Sample and the Equity Correlation

The Pearson $r \approx 0.84$ between SEIFA decile and overall HS score is based on three data points. With $n = 3$, any monotonic relationship produces a high Pearson r — the result is directionally correct but cannot be interpreted as a statistically significant finding at conventional $\alpha = 0.05$.

The equity hypothesis is strongly supported by the qualitative evidence (the most disadvantaged school has the worst cycling infrastructure, the highest crash density during school hours, and the highest proportion of car-free households), but the numerical correlation requires replication at larger n before it should be cited in policy documents.

24.2.4 Hardcoded Fallback Data

The pipeline ships with hardcoded 2023 suburb averages for PM2.5 and crime rates (Chapters 17). These are accurate as of the Census 2021 / financial year 2023–24 reference periods, but they will drift as conditions change. The fallback values should be reviewed annually and updated to the most recent available reference year.

24.3 Design Decisions Worth Documenting

Why vectorised Haversine rather than PostGIS? Nearest-school assignment (`crash_analysis.py`) uses a NumPy vectorised Haversine loop rather than a PostGIS `ST_DWithin` query. This avoids a PostgreSQL/PostGIS dependency, making the pipeline installable with a single `pip install -r requirements.txt`. For $n < 100,000$ crash records the performance is adequate (< 1 second).

Why `pd.DataFrame.iterrows()` in `main.py` for HS scoring? The scoring functions (HS1–HS10) accept a single row dict and return a float. A row-wise loop is the simplest correct implementation for a dataset of $< 1,000$ survey observations. Vectorising the scoring functions would require rewriting all ten modules — a premature optimisation for a small dataset.

Why Ridge regression? Ridge is the simplest regularised regressor and requires no hyperparameter search (only α , tuned by LOO-CV). Given $n = 3$, any more complex model would overfit trivially. Ridge imposes no structural assumptions beyond linearity and L2 regularisation, which is appropriate when feature engineering is the primary modelling effort.

24.4 Future Technical Directions

The project identified several concrete extensions that would substantially improve the pipeline’s scientific and practical value:

Spatial SEIFA join. Replace the suburb-name matching in `seifa_analysis.py` with a GeoPandas spatial join of ABS SA1 polygons to school catchment buffers. This would give a precise, population-weighted IRSD for each school’s actual 800 m walking catchment rather than its postcode suburb.

GTFS public transport frequency. Integrate GTFS timetable data from Public Transport Victoria to compute the actual PT headway at each school gate stop. This would replace the proxy measure (`pt_stops_400m` count) with a meaningful accessibility metric for HS6.

Repeat-observation study design. Run a second round of field surveys after a funded intervention (e.g. the Plenty Road school zone) is installed. Comparing pre- and post-intervention HS scores against the scenario engine’s predicted delta would validate (or calibrate) the Ridge model and provide a defensible before/after metric for grant acquittals.

GitHub Actions pipeline refresh. Add a CI workflow triggered on push to `main` that runs `python run_all.py` and `python prepare_data.py` and commits the updated `docs/data/data.json`. This would keep the GitHub Pages dashboard in sync with the latest data automatically.

Multi-LGA comparison. Run the pipeline on schools in Merri-bek (Moreland), Yarra, and Darebin councils to test whether the equity finding (disadvantaged catchments have lower HS scores) holds at a city scale. A 60-school sample would produce a statistically powered Pearson r and allow cross-council benchmarking.

Appendix A

Complete HS Scoring Formula Reference

This appendix documents all ten Healthy Streets indicator formulae as implemented in `src/scoring/hs1.py` through `src/scoring/hs10.py`. Each function accepts a scored DataFrame row (post `COLUMN_RENAME`) and returns a float in $[0, 10]$. The source of truth is always the Python module; this reference is provided for audit and peer-review purposes.

HS1 — Pedestrians from All Walks of Life (Footpath Accessibility)

Points accumulate from five components (max 10.0 before capping):

Column	Condition	Points
Footpath_present	Both sides of the road	+3.0
	One side only	+2.0
	Partial / broken	+1.0
Footpath_width	≥ 2.0 m	+2.0
	≥ 1.5 m	+1.5
	≥ 1.2 m	+1.0
	≥ 1.0 m	+0.5
Continuity	100 % continuous	+2.0
	75–99 %	+1.5
	50–74 %	+1.0
FP_condition	$(\text{condition} / 5.0) \times 2.0$	+0.0 to +2.0
Kerb_ramps	All nearby intersections	+0.5
	Some intersections	+0.25
Obstructions	No obstructions	+0.5
	Minor obstructions	+0.25

HS2 — Easy to Cross

Column	Condition	Points
Crossing_present	Signalised / traffic light	+4.0
	Raised crossing	+3.0
	Zebra / marked	+2.5
	Refuge island	+2.0
	Informal / unmarked	+0.5
Crossing_dist	≤ 30 m from gate	+2.0
	≤ 75 m	+1.5
	≤ 150 m	+1.0
	≤ 250 m	+0.5
Visibility	$(\text{visibility} / 5.0) \times 2.0$	+0.0 to +2.0
Tactile	Both sides	+1.0
	One side	+0.5
Signal	Countdown timer present	+1.0
	Signal present (no timer)	+0.5

HS5 — Not Too Noisy / Traffic Stress

HS5 starts at 10.0 and deducts points based on traffic stress indicators:

Column	Condition	Deduction
Traffic_volume	Very high / major arterial	−4.0
	High	−3.0
	Moderate	−1.5
Speed_limit	≥ 70 km/h	−2.0
	≥ 60 km/h	−1.0
	≥ 50 km/h	−0.5
Heavy_vehicles	Frequent	−2.0
	Occasional	−1.0
Lanes	3 or more lanes	−1.0
	2 lanes	−0.5

HS Indicators Summary — All Ten

Code	Max strategy	Key inputs
HS1	Additive (cap 10)	Footpath presence, width, continuity, condition, kerb ramps, obstructions
HS2	Additive (cap 10)	Crossing type, distance from gate, visibility, tactile pavers, signal/countdown
HS3	Additive (cap 10)	Tree count, shelter presence, green space coverage, seating coverage
HS4	Additive (cap 10)	Bench count, park proximity, seating along walking route
HS5	Subtractive (start 10)	Traffic volume, speed limit, heavy vehicles, lane count
HS6	Additive (cap 10)	Cycling infrastructure type (CIS), PT stops within 400 m
HS7	External (AQI/crime)	Crime rate per 100k (CSA) + street lighting score
HS8	Additive (cap 10)	Amenity count, cafe count, activity diversity within 400 m
HS9	Additive/subtractive	School zone present, traffic calming, parking conflicts
HS10	External (AQI)	PM2.5 concentration mapped to five breakpoints (US EPA scale)

Severity thresholds (src/scoring/severity.py)

Major: HS2 < 4.0 *or* HS1 < 4.0 *or* HS5 < 3.0.

Moderate: 2+ indicators < 6.0 *or* HS_overall < 5.0.

Minor: all other cases. The Major check runs first; if any crisis indicator is triggered, Moderate is never tested.

Appendix B

Recommendation Rules and Intervention Catalogue

The 17 Recommendation Rules

Table B.1 lists all 17 rules in `src/recommendations/rules.py` in order of rule number. Each rule fires when its trigger condition is met on a single observation row and appends a recommendation dict to the row’s output.

Table B.1: All 17 recommendation rules.

#	HS	Trigger condition	Priority
01	HS1	Footpath missing or broken	High
02	HS1	Footpath width < 1.5 m	Medium
03	HS2	No formal pedestrian crossing	High
04	HS2	Nearest crossing > 150 m from gate	High
05	HS2	No tactile ground surface indicators	Medium
06	HS9	No school zone signage on this street	High
07	HS9	No traffic calming on 3+ lane road	High
08	HS6	No cycling infrastructure on school frontage	Low
09	HS7	Poor or no street lighting	Medium
10	HS3	Vegetation blocking path or sightlines	Medium
11	HS1	No kerb ramps at nearby intersections	Medium
12	HS5	Speed limit ≥ 60 km/h near gate	High
13	HS5	Frequent heavy vehicles past gate	High
14	HS2	Crossing visibility rating < 3/5	Medium
15	HS6	HS6 score < 4.0 (post-scoring rule)	High
16	HS7	HS7 score < 5.0 (post-scoring rule)	High
17	HS10	HS10 score < 5.0 (post-scoring rule)	Medium

Rules 1–14 fire on raw survey fields before HS scoring. Rules 15–17 are post-scoring rules: they fire on the computed HS indicator values and can trigger on rows that passed the raw-field checks (e.g. cycling infrastructure is present but the overall active travel score is still low due to other factors).

The 10 Scenario Interventions

Table B.2 documents all 10 interventions in `src/scenarios/interventions.py`. Each intervention defines a set of feature deltas applied to the school’s ML feature vector before the scenario prediction.

Feature deltas by intervention target indicator:

Table B.2: All 10 scenario interventions with cost and timeframe.

Key	Label	Cost band	Timeframe
pedestrian_crossing	Install signalised crossing	Medium (\$80k–\$200k)	< 1 year
bike_lane	Install protected bike lane	High (\$500k–\$1.5M/km)	1–3 years
speed_reduction	Reduce to 40 km/h school zone	Low (\$5k–\$20k)	< 6 months
traffic_calming	Speed humps / raised crossing	Medium (\$50k–\$150k)	< 1 year
footpath	Build / extend footpath	Medium (\$100k–\$300k)	< 1 year
street_trees	Plant street trees	Low (\$20k–\$60k)	< 1 year
benches	Add street seating	Low (\$5k–\$15k)	< 6 months
pt_stop	Add / improve PT stop	Medium (\$50k–\$200k)	1–2 years
shelter	Install covered shelter	Low (\$15k–\$40k)	< 6 months
remove_arterial	Reroute heavy vehicles	High (network-level)	2–5 years

- pedestrian_crossing: adds +1 to crossings_400m, +1 to signals_400m, +0.15 to crossing_density_400m. *Target: HS2*
- bike_lane: adds +12.0 to cycle_pct_400m, +200.0 m to protected_cycle_length_400m. *Target: HS6*
- speed_reduction: applies –10 km/h to avg_speed_400m and avg_speed_zone. *Target: HS5*
- traffic_calming: –5 km/h speed, –2 crash count, –0.10 serious/fatal rate. *Target: HS9*
- footpath: +10 % to footpath_pct_200m/400m, +500 m to walk_length_400m. *Target: HS1*
- street_trees: +12 to tree_count_100m, +5 % to green_pct_400m. *Target: HS3*
- benches: +6 to bench_count_200m. *Target: HS4*
- pt_stop: +1 to pt_stops_400m. *Target: HS6*
- shelter: +2 to shelter_count_200m. *Target: HS3*
- remove_arterial: –10 % arterial, –3 high-speed roads, –5 km/h speed. *Target: HS5*

Appendix C

Pipeline and Output Quick Reference

The Ten Pipeline Steps

Step	Script	What it does	Key outputs
1	<code>crash_analysis.py</code>	Download VicRoads crash CSVs; filter ped/cyc; assign nearest school via Haversine	<code>crash_data_statewide.csv</code> , <code>crash_darebin.csv</code>
2	<code>spatial_features.py</code>	Extract OSM features at 100/200/400 m buffers via <code>osmnx</code>	<code>spatial_features.csv</code> , <code>networks.gpkg</code>
3	<code>environmental_features.py</code>	Fetch EPA AQI (3-tier fallback) and CSA crime rate	<code>environmental_features.csv</code>
4	<code>main.py</code>	Score HS1–HS10; generate charts; build interactive map	<code>hs_scores.csv</code> , <code>recommendations.csv</code> , 3 PNGs, 2 HTMLs
5	<code>feature_engineering.py</code>	Merge spatial + env + crash into ML feature matrix	<code>ml_school_features.csv</code>
6	<code>ml_model.py</code>	Train Ridge regressor; LOO-CV evaluation	<code>hs_predictor.pkl</code> , <code>ml_evaluation.csv</code>
7	<code>seifa_analysis.py</code>	Download ABS SEIFA 2021; compute catchment IRSD	<code>seifa_darebin.csv</code> , <code>seifa_darebin_sa1.csv</code>
8	<code>equity_analysis.py</code>	Join SEIFA + HS scores; compute Pearson r	<code>chart_equity_seifa.png</code>
9	<code>crash_trend_analysis.py</code>	Year-on-year crash trends; school-hours breakdown	<code>chart_crash_trends.png</code>
10	<code>demographics_chart.py</code>	ABS Census 2021 demographics chart	<code>chart4_demographics.png</code>

All Output Files

File	Description
crash_data_statewide.csv	All ped/cyclist crashes in Victoria 2018–2023, with nearest-school assignment
crash_darebin.csv	Subset of statewide data for Darebin LGA only
spatial_features.csv	OSM-derived features per school at 200/400 m buffers
networks.gpkg	Walk and cycling network GeoPackage for QGIS import
environmental_features.csv	PM2.5, AQI category, HS10 score, crime rate, HS7 pts per school
hs_scores.csv	School-level HS1–HS10 means + overall score + severity
recommendations.csv	All triggered recommendations, ranked by priority
chart1_safety_scores.png	Grouped bar chart: FAS, CSS, EEI, CIS, CYS per school
chart2_hazard_severity.png	Stacked bar chart: Major / Moderate / Minor counts per school
chart3_score_breakdown.png	Per-school indicator breakdown with severity badge
map_interactive.html	Folium hazard map (open in browser)
map_heatmap.html	Folium KDE heatmap (open in browser)
heatmap.png	KDE heatmap PNG for QGIS import
ml_school_features.csv	Feature matrix for ML training and scenario inference
hs_predictor.pkl	Trained Ridge model bundle (pipeline + feature list)
ml_evaluation.csv	LOO-CV RMSE per indicator
seifa_darebin.csv	School-level SEIFA IRSD scores and disadvantage labels
seifa_darebin_sa1.csv	Full SA1-level Darebin SEIFA data
chart_equity_seifa.png	Three-panel equity chart (SEIFA vs HS scores)
chart_crash_trends.png	Four-panel crash trend chart (2021–2025)
chart4_demographics.png	Four-panel ABS demographics chart
docs/data/data.json	Dashboard data bundle for GitHub Pages

Key Configuration Variables

Variable	Description
CSV_FILE	Path to <code>school_data.csv</code> (field survey input)
DEMO_FILE	Path to <code>demographics_darebin.csv</code> (static Census data)
OUT_DIR	Path to the <code>outputs/</code> directory
SCHOOL_GATES	Dict: full school name → lat, lon, street address
SCHOOL_SHORT_NAMES	Dict: full name → short display name
SCHOOL_GATES_BY_SHORT	Dict: short name → lat, lon (for scripts keyed by short name)
HS_INDICATORS	Ordered list of (code, label) pairs defining HS1–HS10
CIS_MAP	Dict: cycling infrastructure response → CIS points (1.0–9.0)
COLUMN_RENAME	Dict: raw Google Form header → internal column name

Running the Pipeline

```

1 # 1. Create environment and install dependencies
2 python -m venv venv
3 source venv/bin/activate          # Windows: venv\Scripts\activate
4 pip install -r requirements.txt
5
6 # 2. Run all 10 steps (skip-if-exists logic prevents re-runs)
7 python run_all.py
8
9 # 3. Force re-run from step 4 onwards (e.g. after new survey data)
10 python run_all.py --from 4 --force
11
12 # 4. Bundle outputs for the dashboard and deploy
13 python prepare_data.py
14 git add docs/data/data.json
15 git commit -m "update dashboard data"
16 git push origin main
17 # GitHub Pages serves the updated dashboard within ~60 seconds

```

Listing C.1: Full pipeline run from a clean checkout.

What's Next

This book has now covered the complete technical implementation of the 300,000 Streets pilot study across twenty-one chapters in six parts, together with three reference appendices:

- Part 1 (Chapters 1–4).** Project context and the Healthy Streets framework; development environment setup; repository architecture and configuration.
- Part 2 (Chapters 5–8).** Study area and school selection; field survey methodology and the Google Forms instrument; OSM data collection with `osmnx`; government open datasets (VicRoads, ABS SEIFA, EPA, Crime Statistics).
- Part 3 (Chapters 9–12).** The HS scoring engine (all ten indicator formulae); severity classification and the recommendation rule engine; machine learning with Ridge regression and LOO-CV; scenario analysis and the delta method.
- Part 4 (Chapters 13–15).** Crash trend analysis (VicRoads 2021–2025); equity analysis (SEIFA disadvantage versus HS safety scores); dashboard and deployment pipeline (`prepare_data.py`, Leaflet, GitHub Pages).
- Part 5 (Chapters 16–18).** Crash data download and vectorised spatial join (`crash_analysis.py`); environmental features: EPA AirWatch AQI and CSA Victoria crime rate with two-tier fallback (`environmental_features.py`); core pipeline orchestration and visualisation (`main.py`, `src/visualisation/`).
- Part 6 (Chapters 19–21).** Feature engineering — the 26-column open-data feature matrix (`feature_engineering.py`); SEIFA 2021 disadvantage analysis (`seifa_analysis.py`); ABS Census 2021 demographics chart (`demographics_chart.py`).
- Part 7 (Chapters 22–24).** Pilot study results and policy implications; how to extend the pipeline to new schools and LGAs; architecture retrospective, known limitations, and future technical directions.
- Appendices A–C.** Complete HS scoring formula reference; all 17 recommendation rules and 10 scenario interventions; pipeline step table, output file catalogue, and configuration variable reference.

A developer who has followed this book from Chapter 1 can now:

1. Clone the repository and run `pip install -r requirements.txt`.
2. Run `python run_all.py` to reproduce every one of the 21 output files from a clean checkout — from crash data download through SEIFA analysis, ML training, and the demographics chart.
3. Run `python prepare_data.py` to bundle all outputs into `docs/data/data.json`.
4. Push to `main` and have a live GitHub Pages dashboard within 60 seconds.

5. Add a new school to the five configuration points described in Chapter 23 and re-run the pipeline without changing a single line of analysis code.

The 300,000 Streets pilot demonstrates that a rigorous, evidence-based assessment of school street safety can be built entirely from open data — OSM, ABS, VicRoads, EPA, and Crime Statistics — without a budget for proprietary GIS software or bespoke sensor infrastructure. The pipeline is small enough to run on a laptop, transparent enough to audit, and modular enough to extend. The ten-step orchestrator and the GitHub Pages dashboard are not the end product; they are the scaffolding for a much larger study.

What the pilot proved above all is that the equity hypothesis is real and measurable. The students who walk to Reservoir High School along Plenty Road every morning are doing so on a street that scores 2.1 out of 10 for cycling safety, 3.8 out of 10 for shade and shelter, and 4.2 out of 10 for footpath accessibility — while living in a catchment that ABS ranks in the bottom 40 % of the country for socioeconomic advantage. That is not a coincidence. It is a pattern, and this pipeline exists to make that pattern visible, measurable, and actionable.

The 300,000 streets are waiting.

*300,000 Streets of Melbourne — Regen Melbourne × RMIT University
City of Darebin Pilot, 2024*